technische universität
dortmund

Bachelorarbeit

Erweiterung und reproduzierbare Evaluierung von Fast-Failover-Routingverfahren mit kanten- und knotendisjunkten Pfaden

Ammar Gamous
237440

Studiengang: Bachelor Informatik
Erstgutachter: Prof. Dr. Dr. Klaus-Tycho Förster
Zweitgutachter: Stephanie Althoff, M. Sc.

30.09.2026

Fakultät für Informatik
Lehrstuhl IV - Praktische Informatik

Abstract

Der TREE-Ansatz erweitert kantendisjunkte Pfade zu baumbasierten Fast-Failover-Routingstrukturen und wurde ursprünglich für Kantenausfälle untersucht. In dieser Arbeit wird der Ansatz auf Knotenausfälle erweitert. Zusätzlich werden knotendisjunkte Pfade als alternative Grundlage für die Konstruktion der Routingstrukturen untersucht. Dafür werden jeweils die Varianten Paths, One Tree und Multiple Trees für EDPs und NDPs umgesetzt.

Die sechs Routingverfahren werden bei zufälligen und geclusterten Kanten- und Knotenausfällen evaluiert. Die Versuche umfassen drei Erdős–Rényi-Konfigurationen sowie die Real-World-Topologien ATT-MPLS, Oteglobe und Cogentco mit jeweils 1000 Wiederholungen. Als Bewertungsmetriken werden Resilienz, durchschnittliche Hopanzahl und additiver Stretch verwendet.

Die Ergebnisse zeigen keine allgemeine Überlegenheit kanten- oder knotendisjunkter Pfade. Ihr Verhalten hängt von der Topologie und dem Fehlermodell ab. Die Baumerweiterungen erhöhen jedoch in vielen Konfigurationen die Resilienz gegenüber dem Routing über die ursprünglichen disjunkten Pfade. Innerhalb der untersuchten NDP-Verfahren erreicht Multiple Trees stets mindestens die Resilienz von One Tree und One Tree mindestens die Resilienz von Paths. Dieser Gewinn geht häufig mit einer höheren durchschnittlichen Hopanzahl und einem höheren Stretch einher.

Ergänzend wurde eine automatisierte Pipeline entwickelt, die den gesamten Evaluierungsablauf von der Konfiguration und Durchführung der Experimente bis zur Aggregation der Ergebnisse und automatischen Ploterstellung unterstützt. Das Framework kann sowohl über eine Kommandozeile als auch über eine Streamlit-basierte grafische Benutzeroberfläche bedient werden. Eine Artefaktevaluation mit drei Informatikstudierenden zeigt, dass die Umgebung im geprüften Umfang von anderen Personen verwendet und erweitert werden kann. Damit stellt sie eine dokumentierte und unter festgelegten Bedingungen wiederholbare Grundlage für weitere Fast-Failover-Experimente bereit.

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Motivation . . . 1
1.2 Problemstellung . . . 1
1.3 Ziel der Arbeit . . . 2
1.4 Forschungsfragen . . . 3
1.5 Struktur . . . 3

2 Grundlagen und Ausgangspunkt der Arbeit **4**

2.1 Netzwerke als Graphen . . . 4
2.2 Fast-Failover-Routing . . . 5
2.3 Link- und Knotenausfälle . . . 6
2.3.1 Ausfalltypen . . . 6
2.3.2 Ausfallmodelle . . . 8
2.4 Disjunkte Pfade . . . 9
2.5 Ausgangspunkt der Arbeit: TREE-Algorithmen . . . 11

3 Konzeption der Erweiterung **14**

3.1 Erweiterung auf Knotenausfälle (Node Failures) . . . 14
3.1.1 Modellierung . . . 15
3.1.2 Änderungen im Routing . . . 15
3.2 Erweiterung auf Node-Disjoint Paths . . . 16
3.2.1 Berechnung der NDPs . . . 16
3.2.2 Unterschiede zu EDPs . . . 17
3.3 Erweiterung der Algorithmen . . . 18
3.3.1 One-Tree-Algorithmus . . . 19
3.3.2 Multiple-Trees-Algorithmus . . . 21
3.3.3 Besonderheiten und Unterschiede zur EDP-Version . . . 22
3.4 Gesamtüberblick . . . 23

4 Implementierung **25**

4.1 Projektstruktur . . . 26
4.2 Implementierung der Algorithmen . . . 27
4.2.1 NDP Paths . . . 28
4.2.2 NDP One Tree . . . 29
4.2.3 NDP Multiple Trees . . . 31
4.3 Modellierung der Fehlermodelle . . . 34
4.3.1 Random Link Failure . . . 35
4.3.2 Clustered Link Failure . . . 37
4.3.3 Random Node Failure . . . 41
4.3.4 Clustered Node Failure . . . 42
4.4 Seed-basierte Steuerung für reproduzierbare Randomisierung der Versuchsszenen . . . 44
4.5 Routing . . . 49
4.5.1 Beteiligte Funktionen . . . 49
4.5.2 Reihenfolge der Routingstrukturen . . . 50
4.5.3 Routing innerhalb einer Struktur . . . 51
4.5.4 Wechsel zwischen mehreren Strukturen . . . 52
4.5.5 Vergleich mit der TREE-Referenzimplementierung . . . 54
4.6 Evaluierungskomponenten . . . 55
4.6.1 CSV-Dateien . . . 55
4.6.2 Bewertungsmetriken . . . 57
4.6.3 Ploterstellung . . . 60
4.7 Pipeline . . . 62
4.7.1 JSON-Konfiguration . . . 63
4.7.2 CLI und GUI . . . 64
4.7.3 Automatische Ausführung . . . 67
4.7.4 Automatische Plot-Erstellung . . . 69

5 Evaluation **71**
5.1 Versuchsaufbau . . . 71
5.1.1 Verwendete Graphen . . . 72

5.1.2 Umfang der Evaluation . . . 74
5.1.3 Technische Ausführungsumgebung . . . 74
5.2 Auswertungsmethodik . . . 74
5.2.1 Aggregation und Interpretation der Metriken . . . 74
5.2.2 Darstellung der Ergebnisse . . . 75
5.2.3 Vorgehen beim Algorithmusvergleich . . . 76
5.3 Ergebnisse auf Random-Graphen . . . 77
5.3.1 Random Link Failure . . . 77
5.3.2 Clustered Link Failure . . . 79
5.3.3 Random Node Failure . . . 80
5.3.4 Clustered Node Failure . . . 83
5.4 Ergebnisse auf Real-World-Graphen . . . 85
5.4.1 Random Link Failure . . . 85
5.4.2 Clustered Link Failure . . . 87
5.4.3 Random Node Failure . . . 89
5.4.4 Clustered Node Failure . . . 91
5.5 Vergleich der Algorithmen . . . 93
5.5.1 Vergleich der EDP- und NDP-Verfahren . . . 93
5.5.2 Einfluss der Baumerweiterungen . . . 94
5.5.3 Resilienz und Routeneffizienz . . . 95
5.6 Diskussion der Ergebnisse . . . 95

6 Experimentierumgebung, Reproduzierbarkeit und Erweiterbarkeit 98
6.1 Bereitstellung der Experimentierumgebung . . . 98
6.2 Aufbau und Durchführung der Artefaktevaluation . . . 99
6.3 Ergebnisse zur Bedienbarkeit und Reproduzierbarkeit . . . 100
6.4 Ergebnisse zur Erweiterbarkeit . . . 101
6.5 Überarbeitung anhand der Rückmeldungen . . . 102
6.6 Zusammenfassende Bewertung . . . 103

7 Fazit und Ausblick 105

7.1 Zusammenfassung und erreichte Ziele . . . 105
7.2 Beantwortung der Forschungsfragen . . . 106
7.2.1 Übertragung der TREE-Verfahren . . . 106
7.2.2 Vergleich der EDP- und NDP-Verfahren . . . 107
7.2.3 Einfluss der Baumerweiterungen . . . 107
7.2.4 Artefaktevaluation der Experimentierumgebung . . . 108
7.3 Grenzen der Arbeit . . . 109
7.4 Ausblick und mögliche Erweiterungen . . . 110
7.4.1 Weitere Fehlermodelle . . . 111
7.4.2 Weitere Graphtypen . . . 111
7.4.3 Optimierung der Implementierung . . . 111
7.4.4 Zusätzliche Routingalgorithmen . . . 112
7.4.5 Erweiterung der Auswertung . . . 113

Literaturverzeichnis **114**

Anhang **116**

Abbildungsverzeichnis **132**

Tabellenverzeichnis **135**

Algorithmenverzeichnis **136**

1 Einleitung

Dieses Kapitel führt in das Thema der Arbeit ein. Zunächst erläutert Abschnitt 1.1 die Motivation für die Untersuchung ausfallsicherer Routingverfahren. Anschließend beschreibt Abschnitt 1.2 die zugrunde liegende Problemstellung, bevor Abschnitt 1.3 die Ziele der Arbeit festlegt. Daraus werden in Abschnitt 1.4 die zentralen Forschungsfragen abgeleitet. Abschließend gibt Abschnitt 1.5 einen Überblick über den weiteren Aufbau der Arbeit.

1.1 Motivation

Fehler und Ausfälle sind in heutigen Computernetzen allgegenwärtig. Sie können durch Hardwaredefekte, Softwarefehler, menschliche Fehlhandlungen oder äußere Einwirkungen wie Stromausfälle und Naturkatastrophen verursacht werden. Trotzdem sollen die über das Netzwerk bereitgestellten Dienste und Anwendungen möglichst erreichbar bleiben. Daher ist die Resilienz eines Netzwerks, also seine Widerstandsfähigkeit gegenüber Ausfällen, ein wichtiger Gegenstand der Untersuchung [1].

Fällt eine Verbindung oder ein Netzwerkknoten aus, kann die aktuell verwendete Route unterbrochen werden. Eine Neuberechnung durch die Kontrollebene kann jedoch Zeit benötigen. Fast-Failover-Routingverfahren berechnen deshalb alternative Routingstrukturen im Voraus und ermöglichen eine lokale Reaktion auf Ausfälle [2].

Ein Knotenausfall unterscheidet sich von einem Kantenausfall, weil gleichzeitig alle mit dem ausgefallenen Knoten verbundenen Kanten nicht mehr verwendet werden können. Da kantendisjunkte Pfade gemeinsame innere Knoten besitzen dürfen, kann ein einzelner Knotenausfall mehrere dieser Pfade gleichzeitig unterbrechen.

Knotendisjunkte Pfade bieten hierfür eine mögliche Alternative. Sie besitzen außer dem Start- und Zielknoten keine gemeinsamen Knoten [3]. Daher ist zu untersuchen, wie sich die EDP-basierten TREE-Verfahren um NDP-basierte Varianten erweitern lassen und wie sich beide Pfadtypen bei Kanten- und Knotenausfällen verhalten.

Für einen belastbaren Vergleich ist außerdem ein einheitlicher und reproduzierbarer experimenteller Ablauf erforderlich.

1.2 Problemstellung

Die TREE-Verfahren wurden für kantendisjunkte Pfade und Link Failures entwickelt [2]. Es ist daher nicht unmittelbar klar, ob sie auch bei Knotenausfällen dieselben Eigenschaften besitzen. Ein ausgefallener Knoten entfernt gleichzeitig alle inzidenten Kanten und kann dadurch mehrere kantendisjunkte Pfade unterbrechen.

Als Alternative werden knotendisjunkte Pfade betrachtet. Ihre Verwendung erfordert jedoch Anpassungen an den bestehenden Verfahren. Insbesondere muss festgelegt werden, wie die One-Tree- und Multiple-Trees-Strukturen aus knotendisjunkten Pfaden aufgebaut werden und welche Kanten und Knoten während des Baumwachstums erneut verwendet werden dürfen.

Für die anschließende Bewertung müssen die ursprünglichen EDP-basierten und die erweiterten NDP-basierten Verfahren unter vergleichbaren Bedingungen ausgeführt werden. Dazu müssen sie innerhalb einer Wiederholung denselben Graphen, dasselbe Start-Ziel-Paar und dieselben Ausfälle erhalten. Andernfalls könnten Ergebnisunterschiede durch verschiedene Versuchsszenen statt durch die Algorithmen verursacht werden.

Die Kombination mehrerer Algorithmen, Fehlermodelle, Graphen und Parameter führt zu einer großen Anzahl von Experimenten. Neben der algorithmischen Erweiterung wird deshalb eine reproduzierbare Experimentumgebung benötigt, welche die Konfiguration, Durchführung, Speicherung und Auswertung der Versuche automatisiert.

1.3 Ziel der Arbeit

Ziel dieser Arbeit ist es, die EDP-basierten TREE-Verfahren um die Behandlung von Knotenausfällen und um NDP-basierte Varianten zu erweitern. Neben EDP Paths, EDP One Tree und EDP Multiple Trees werden dazu NDP Paths, NDP One Tree und NDP Multiple Trees umgesetzt.

Die sechs Routingverfahren sollen unter vergleichbaren Bedingungen experimentell untersucht werden. Dabei werden zufällige und geclusterte Kanten- sowie Knotenausfälle betrachtet. Die Versuche werden sowohl auf synthetischen Graphen als auch auf ausgewählten Real-World-Topologien durchgeführt.

Ein weiteres Ziel besteht in der Entwicklung einer reproduzierbaren Experimentumgebung. Sie soll die Auswahl und Konfiguration der Algorithmen, Fehlermodelle und Graphen ermöglichen und anschließend den vollständigen Versuchsablauf automatisieren. Dazu gehören die Durchführung der Experimente, die Speicherung der Rohdaten, die Berechnung der Bewertungsmetriken und die Erzeugung der Diagramme.

Die Umgebung soll außerdem verständlich dokumentiert und erweiterbar sein. Andere Personen sollen die Experimente mit denselben Einstellungen wiederholen und zusätzliche Algorithmen, Fehlermodelle oder Grapharten mit überschaubarem Aufwand ergänzen können.

1.4 Forschungsfragen

Aus der beschriebenen Problemstellung und den Zielen der Arbeit ergeben sich vier zentrale Forschungsfragen:

1. Wie können Knotenausfälle in das untersuchte TREE-Modell integriert und die EDP-basierten Verfahren auf knotendisjunkte Pfade übertragen werden?
2. Wie unterscheiden sich die EDP- und NDP-basierten Routingverfahren hinsichtlich Resilienz, durchschnittlicher Hopanzahl und durchschnittlichem Stretch?
3. Welchen Einfluss haben die Erweiterungen One Tree und Multiple Trees gegenüber dem Routing über die ursprünglichen disjunkten Pfade bei zufälligen und geclusterten Kanten- und Knotenausfällen?
4. In welchem Umfang ermöglicht das entwickelte Framework anderen Personen eine einfache Bedienung, die reproduzierbare Durchführung der Experimente und die Erweiterung um zusätzliche Algorithmen, Fehlermodelle und Grapharten?

Die ersten drei Forschungsfragen untersuchen die algorithmischen Erweiterungen und ihre Ergebnisse auf synthetischen und realen Netzwerktopologien. Die vierte Forschungsfrage betrachtet die Bedienbarkeit, Reproduzierbarkeit und Erweiterbarkeit der entwickelten Umgebung.

1.5 Struktur

Nach der Einleitung beschreibt Kapitel 2 die benötigten Grundlagen und stellt die ursprünglichen TREE-Algorithmen vor. Kapitel 3 erläutert die Konzeption der Erweiterungen auf Knotenausfälle und knotendisjunkte Pfade.

Kapitel 4 behandelt die konkrete Implementierung der Algorithmen, Fehlermodelle, Routingroutine, Evaluierungskomponenten und Pipeline. Die Ergebnisse der Experimente auf synthetischen und realen Netzwerktopologien werden in Kapitel 5 dargestellt und diskutiert.

Kapitel 6 untersucht die Reproduzierbarkeit, Bedienbarkeit und Erweiterbarkeit der entwickelten Experimentierumgebung. Kapitel 7 fasst die Ergebnisse zusammen, beantwortet die Forschungsfragen und gibt einen Ausblick auf mögliche Erweiterungen.

2 Grundlagen und Ausgangspunkt der Arbeit

In diesem Kapitel werden die theoretischen Grundlagen und der Ausgangspunkt dieser Arbeit vorgestellt. Zunächst wird in 2.1 beschrieben, wie Kommunikationsnetze als Graphen modelliert werden und welche Begriffe für die spätere Betrachtung von Routingverfahren relevant sind. Anschließend werden Fast-Failover-Routing in 2.2, Link- und Knotenausfälle in 2.3 sowie disjunkte Pfade in 2.4 erläutert. Abschließend wird in 2.5 das TREE-Verfahren als Grundlage dieser Arbeit vorgestellt, da die in dieser Arbeit entwickelten Erweiterungen direkt auf den dort beschriebenen Algorithmen für Edge-Disjoint Paths, One Tree und Multiple Trees aufbauen.

2.1 Netzwerke als Graphen

Kommunikationsnetze können als Graphen modelliert werden. Ein **Graph** beschreibt, welche Objekte miteinander verbunden sind. In dieser Arbeit stehen die **Knoten** für Netzwerkelemente, zum Beispiel Router oder Switches. Die **Kanten** beschreiben die Verbindungen zwischen diesen Elementen. Das Netzwerk wird daher als Graph $G = (V, E)$ betrachtet. Dabei bezeichnet V die Menge der Knoten und E die Menge der Kanten.

Die betrachteten Verbindungen werden als ungerichtete Kanten modelliert. Das bedeutet, dass eine Verbindung zwischen zwei Knoten in beide Richtungen verwendet werden kann. Im TREE-Paper werden die Links als symmetrische Full-Duplex-Verbindungen beschrieben und sind damit bidirektional nutzbar [2].

Für Routingverfahren sind Pfade besonders wichtig. Ein **Pfad** ist eine Folge von Knoten, bei der jeweils zwei aufeinanderfolgende Knoten durch eine Kante verbunden sind. Ein Pfad von einem Quellknoten s zu einem Zielknoten d beschreibt damit eine mögliche Route durch das Netzwerk. Die Länge eines Pfades wird durch die Anzahl der verwendeten Kanten angegeben. Im Netzwerkkontext entspricht dies der Anzahl der Hops. Je mehr Hops ein Paket benötigt, desto länger ist der Weg durch das Netzwerk.

Ein weiterer wichtiger Begriff ist die Konnektivität. Ein Graph heißt zusammenhängend, wenn zwischen jedem Paar von Knoten ein Pfad existiert. Für diese Arbeit ist besonders relevant, ob der Quellknoten s und der Zielknoten d auch nach Ausfällen noch miteinander verbunden sind. Nur wenn im verbleibenden Graphen noch ein Pfad zwischen s und d existiert, kann ein Routingverfahren das Ziel grundsätzlich erreichen.

Das TREE-Paper bezeichnet einen Graphen als k-kantenverbunden, wenn er nach dem Entfernen beliebiger $k - 1$ Kanten weiterhin zusammenhängend bleibt [2]. Diese Eigenschaft beschreibt, wie stark ein Netzwerk gegenüber Kantenausfällen verbunden ist. Für Fast-Failover-Routing ist das wichtig, weil die Anzahl der verfügbaren alternativen Wege davon abhängt, wie gut Quelle und Ziel miteinander verbunden sind.

Das beschriebene Graphmodell bildet die Grundlage für die folgenden Betrachtungen von Ausfällen, disjunkten Pfaden und Fast-Failover-Verfahren.

In der Evaluation dieser Arbeit werden verschiedene Arten von Graphen verwendet. Dazu gehören zufällig erzeugte Graphen und reale Netzwerktopologien. Zufällige Graphen, wie Erdős-Rényi-Graphen, dienen dazu, die Verfahren unter kontrollierbaren Parametern wie der Anzahl der Knoten oder der Kantenwahrscheinlichkeit zu untersuchen [4, 5]. Random-regular Graphen besitzen einen festen Knotengrad und werden von dem Framework unterstützt [6]. Reale Netzwerktopologien, wie die Topologien aus der Internet Topology Zoo, bilden dagegen existierende Netzwerkstrukturen ab [7]. Die genaue Auswahl und Parametrisierung dieser Graphen wird später im Evaluationskapitel 5 beschrieben.

2.2 Fast-Failover-Routing

Die Erreichbarkeit innerhalb von Kommunikationsnetzen soll auch beim Ausfall einzelner Komponenten erhalten bleiben. Solche Ausfälle können zum Beispiel durch defekte Leitungen, fehlerhafte Hardware, Wartungsarbeiten oder größere Störungen entstehen. Wenn ein Ausfall auftritt, muss das Netzwerk den Verkehr möglichst schnell über alternative Wege weiterleiten. Genau an dieser Stelle setzt Fast-Failover-Routing an. **Fast-Failover-Routing** bezeichnet Verfahren, die nach einem Ausfall schnell auf vorbereitete Ersatzwege umschalten, ohne zuerst eine vollständige globale Neuberechnung der Routen abzuwarten [8].

Dabei ist die Unterscheidung zwischen Control Plane und Data Plane wichtig. Die **Control Plane** ist für die Berechnung und Verwaltung von Routinginformationen zuständig. Sie entscheidet also, welche Wege grundsätzlich im Netzwerk verwendet werden sollen. Die **Data Plane** leitet dagegen die einzelnen Pakete anhand vorhandener Weiterleitungsregeln weiter. Eine vollständige Reaktion über die Control Plane kann vergleichsweise langsam sein, weil Informationen über den Ausfall verteilt und neue Routen berechnet werden müssen. Fast-Failover-Verfahren versuchen diese Verzögerung zu vermeiden, indem die Reaktion direkt in der Data Plane erfolgt [8].

Dafür werden Ersatzregeln bereits vor dem Auftreten eines Ausfalls berechnet und in den Netzwerkknoten hinterlegt. Wenn ein Paket später auf einen Ausfall trifft, kann der betroffene Knoten lokal entscheiden, wie das Paket weitergeleitet werden soll. Der Knoten muss dafür nicht das gesamte Netzwerk neu berechnen und auch nicht auf eine zentrale Instanz warten. Solche statischen Failover-Regeln werden im Voraus berechnet und an den Knoten installiert [2].

Lokales Routing bedeutet in diesem Zusammenhang, dass ein Knoten nur Informationen verwendet, die ihm lokal zur Verfügung stehen. Eine Weiterleitungsregel an einem Knoten $v \in V$ darf nur vom Quellknoten s, vom Zielknoten d, von den am Knoten sichtbaren Ausfällen und vom Eingangsport des Pakets abhängen [2]. Dadurch können

die Regeln sofort angewendet werden, sobald ein Paket ankommt. Gleichzeitig ist dieses Modell eingeschränkt, weil der Knoten keine vollständige Sicht auf alle Ausfälle im Netzwerk hat.

Die vorberechneten Failover-Regeln können im TREE-Modell als statische Routingtabellen an den Netzwerkknoten hinterlegt werden. Eine solche Tabelle ordnet einer Situation einen Ausgangsport zu und enthält neben Regeln für den Normalfall auch Regeln für lokale Ausfälle. Während des Failovers werden diese Regeln nicht verändert [2].

Die Implementierung dieser Arbeit erzeugt keine konkreten Routingtabellen, sondern simuliert das entsprechende Weiterleitungsverhalten auf den vorberechneten Pfad- und Baumstrukturen. Die dabei entstehenden Versuchsergebnisse werden anschließend in CSV-Dateien gespeichert.

An Fast-Failover-Routing ergeben sich mehrere Anforderungen. Erstens muss die Reaktion schnell sein, da Pakete nicht lange warten sollen. Zweitens sollen die Entscheidungen lokal möglich sein, damit keine globale Rekonvergenz notwendig ist [8]. Drittens sollen die Verfahren möglichst viele Ausfallsituationen überstehen und das Ziel weiterhin erreichen. Viertens sollen die entstehenden Wege nicht unnötig lang sein. Routingverfahren werden deshalb unter anderem anhand der Resilience, also des Anteils erfolgreicher Routingversuche, und anhand der Anzahl der benötigten Hops bewertet [2].

In dieser Arbeit wird dieses Fast-Failover-Modell als Grundlage verwendet. Der Fokus liegt darauf, wie sich bestehende Verfahren für Linkausfälle auf Knotenausfälle erweitern lassen und wie sich kanten- und knotendisjunkte Routingstrukturen im Vergleich verhalten.

2.3 Link- und Knotenausfälle

Für die Untersuchung von Fast-Failover-Routing ist nicht nur wichtig, dass Ausfälle auftreten können, sondern auch welche Komponenten betroffen sind und wie diese Ausfälle im Netzwerk verteilt sind. In dieser Arbeit werden deshalb zwei Aspekte unterschieden. Die Ausfalltypen beschreiben in 2.3.1, ob Links oder Knoten ausfallen. Die Ausfallmodelle beschreiben in 2.3.2 dagegen, wie die ausgefallenen Komponenten im Graphen ausgewählt werden.

2.3.1 Ausfalltypen

Fast-Failover-Routing wird eingesetzt, um Kommunikationsnetze auch bei Ausfällen einzelner Komponenten funktionsfähig zu halten. In paketvermittelten Netzwerken können dabei unterschiedliche Arten von Ausfällen auftreten, zum Beispiel Ausfälle von Verbindungen oder Ausfälle ganzer Netzwerkknoten [8]. Für diese Arbeit sind vor allem Linkausfälle und Knotenausfälle relevant.

Ein Link-Ausfall beschreibt den Ausfall einer Verbindung zwischen zwei Netzwerkknoten. Im Graphmodell entspricht ein solcher Link einer Kante. Deshalb werden die Begriffe Link-Ausfall und Kantenausfall in dieser Arbeit synonym verwendet. Fällt ein Link aus, so kann die zugehörige Kante im Graphen nicht mehr für die Weiterleitung von Paketen verwendet werden. Die beiden Endknoten der Kante bleiben dabei jedoch weiterhin funktionsfähig und können Pakete über andere verfügbare Kanten weiterleiten.

Das ursprüngliche Modell, auf dem diese Arbeit aufbaut, betrachtet Fast-Failover-Routing bei Linkausfällen. Dabei wird angenommen, dass eine Menge von Links ausfallen kann und dass die Routingverfahren versuchen, Pakete trotz dieser ausgefallenen Links weiterhin vom Quellknoten s zum Zielknoten d zu leiten [2]. Ein Routingversuch ist nur dann grundsätzlich möglich, wenn im verbleibenden Graphen noch ein Pfad zwischen s und d existiert.

Ein Knotenausfall beschreibt dagegen den Ausfall eines ganzen Netzwerkknotens, zum Beispiel eines Routers oder Switches. Ein ausgefallener Knoten kann keine Pakete mehr weiterleiten. Dadurch sind auch alle Verbindungen betroffen, die mit diesem Knoten inzident sind. Im Graphmodell bedeutet das, dass nicht nur eine einzelne Kante ausfällt, sondern dass alle Kanten, die mit dem ausgefallenen Knoten verbunden sind, nicht mehr verwendet werden können.

In dieser Arbeit werden Knotenausfälle deshalb durch das Sperren aller inzidenten Kanten eines ausgefallenen Knotens modelliert. Sei $F_V \subseteq V$ die Menge der ausgefallenen Knoten. Dann wird die Menge der dadurch betroffenen Kanten beschrieben durch

$$F_E = \{\{u, v\} \in E \mid u \in F_V \vee v \in F_V\}.$$

Diese Kanten stehen dem Routing anschließend nicht mehr zur Verfügung. Aus Sicht der Erreichbarkeit entspricht dies weitgehend dem Entfernen der ausgefallenen Knoten aus dem Graphen, solange Quelle und Ziel selbst nicht ausgefallen sind.

Abbildung 1 zeigt den Unterschied anhand desselben Graphen. Links fällt eine einzelne Kante aus. Rechts ist ein Knoten betroffen, wodurch alle zu ihm inzidenten Kanten nicht mehr verwendet werden können.

Die Auswirkungen von Kanten- und Knotenausfällen hängen auch von der verwendeten Form der Pfaddisjunktheit ab. Der Unterschied zwischen kanten- und knotendisjunkten Pfaden wird in Abschnitt 2.4 erläutert.

Diese Unterscheidung ist besonders wichtig für disjunkte Pfade. Kantendisjunkte Pfade (EDPs) teilen sich keine gemeinsamen Kanten und sind deshalb vor allem für Linkausfälle relevant. Sie können sich aber trotzdem innere Knoten teilen. Wenn ein solcher gemeinsamer innerer Knoten ausfällt, können mehrere kantendisjunkte Pfade (EDPs) gleichzeitig unbrauchbar werden. Bei knotendisjunkten Pfaden kann ein ausgefallener innerer Knoten höchstens einen Pfad der betrachteten Pfadmenge direkt unterbrechen. Die übrigen NDPs werden durch diesen Knotenausfall nicht unmittelbar betroffen. Deshalb

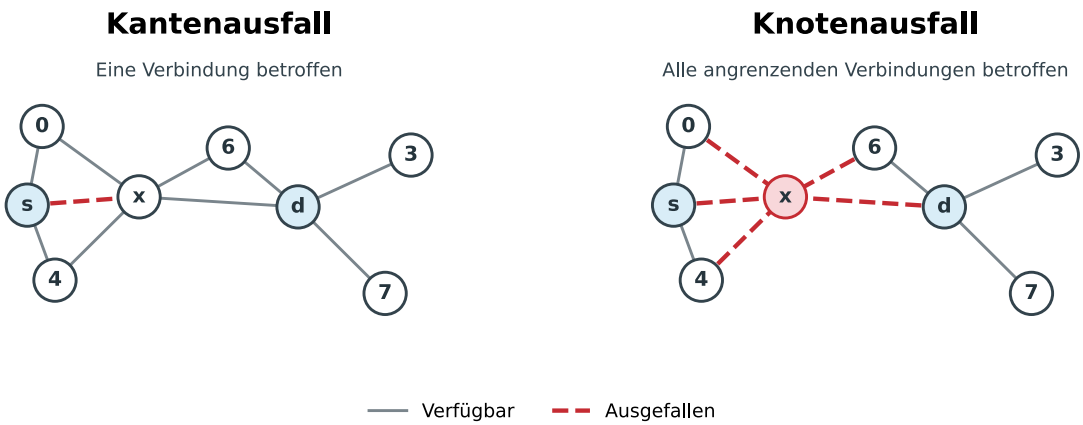


Abbildung 1: Vergleich eines Kantenausfalls (links) und eines Knotenausfalls (rechts) im selben Graphen. Beim Kantenausfall ist eine einzelne Verbindung betroffen. Beim Knotenausfall sind alle zum ausgefallenen Knoten inzidenten Kanten nicht mehr verfügbar.

werden in dieser Arbeit neben kantendisjunkten auch knotendisjunkte Routingstrukturen betrachtet [8].

2.3.2 Ausfallmodelle

Neben dem Ausfalltyp ist auch das Ausfallmodell wichtig. Der Ausfalltyp beschreibt, welche Art von Komponente ausfällt, also ein Link oder ein Knoten. Das Ausfallmodell beschreibt dagegen, wie die ausgefallenen Komponenten im Graphen ausgewählt werden. Das TREE-Paper unterscheidet für Linkausfälle zwischen zufälligen und geclusterten Ausfällen [2]. Diese beiden Modelle werden in dieser Arbeit auch auf Knotenausfälle übertragen.

Bei zufälligen Ausfällen werden die betroffenen Komponenten zufällig aus dem Graphen ausgewählt. Bei Random Link Failures werden also Links zufällig entfernt. Bei Random Node Failures werden entsprechend Knoten zufällig als ausgefallen markiert, wobei Quelle s und Ziel d nicht als ausgefallene Knoten gewählt werden. Dieses Modell verteilt die Ausfälle über den Graphen und untersucht damit, wie robust die Verfahren gegenüber allgemein verteilten Ausfällen sind.

Bei geclusterten Ausfällen treten die Ausfälle dagegen räumlich konzentriert auf. Im TREE-Paper werden Clustered Link Failures um den Zielknoten d modelliert. Dabei fallen Links in der Nähe von d mit einer Wahrscheinlichkeit p_f aus. Zusätzlich können sich Ausfälle auf benachbarte Kanten ausbreiten, wobei die Wahrscheinlichkeit pro Distanzschritt um p_f^{Δ} reduziert wird. In den Experimenten des TREE-Papers wird $p_f^{\Delta} = 0.3$ verwendet [2]. Dieses Modell soll regionale Ausfälle abbilden, wie sie zum Beispiel durch Naturkatastrophen, Erdbeben oder großflächige Stromausfälle entstehen können

[9, 10]. Solche Ereignisse können dazu führen, dass mehrere Netzwerkverbindungen in einem größeren Gebiet um ein Zentrum, zum Beispiel ein Epizentrum, gleichzeitig betroffen sind [11, 12].

In dieser Arbeit wird dieses geclusterte Modell analog auf Knotenausfälle übertragen. Bei Clustered Node Failures werden Knoten in der Nähe des Zielknotens d mit einer höheren Wahrscheinlichkeit als ausgefallen markiert. Für weiter entfernte Knoten wird die Ausfallwahrscheinlichkeit schrittweise reduziert. Auch hier werden Quelle s und Ziel d selbst nicht als ausgefallene Knoten gewählt. Dadurch kann untersucht werden, wie die Routingverfahren reagieren, wenn nicht nur einzelne unabhängige Komponenten ausfallen, sondern mehrere Ausfälle in einem zusammenhängenden Bereich auftreten.

Aus der Kombination von Ausfalltyp und Ausfallmodell ergeben sich die vier in dieser Arbeit betrachteten Fehlertypen: Random Link Failures, Clustered Link Failures, Random Node Failures und Clustered Node Failures. Die genaue Parametrisierung dieser Modelle, etwa die Anzahl der zufälligen Ausfälle oder die Startwahrscheinlichkeit und Abnahme bei geclusterten Ausfällen, wird im Evaluationskapitel 5 beschrieben.

2.4 Disjunkte Pfade

Alternative Pfade zwischen einem Quellknoten (s) und einem Zielknoten (d) bilden eine wesentliche Grundlage des Fast-Failover-Routings. Ein einzelner Pfad reicht bei einem Ausfall oft nicht aus, weil eine ausgefallene Kante oder ein ausgefallener Knoten den gesamten Pfad unbenutzbar machen kann. Deshalb werden mehrere Pfade betrachtet, die möglichst unabhängig voneinander sind. Solche Pfade heißen disjunkte Pfade.

Ein Pfad $P = (v_0, v_1, \ldots, v_\ell)$ mit $v_0 = s$ und $v_\ell = d$ verbindet den Quellknoten s mit dem Zielknoten d. Die Menge der Knoten eines Pfades wird mit $V(P) = \{v_0, v_1, \ldots, v_\ell\}$ bezeichnet, die Kantenmenge mit $E(P) = \{\{v_0, v_1\}, \{v_1, v_2\}, \ldots, \{v_{\ell-1}, v_\ell\}\}$. Die Länge des Pfades entspricht der Anzahl der verwendeten Kanten und beträgt damit ℓ. Die inneren Knoten eines Pfades sind alle Knoten außer dem Quell- und Zielknoten, also

$$V_{\text{int}}(P) = \{v_0, v_1, \ldots, v_\ell\} \setminus \{v_0, v_\ell\}.$$

Diese Unterscheidung ist wichtig, weil Quelle und Ziel bei mehreren Pfaden gemeinsam auftreten dürfen. Entscheidend ist, ob sich die Pfade auf ihren Kanten oder auf ihren inneren Knoten überschneiden.

Eine Menge von Pfaden $P_{gesamt} = \{P_1, P_2, \ldots, P_k\}$ zwischen s und d heißt kantendisjunkt, wenn keine zwei Pfade eine gemeinsame Kante verwenden.
Für alle $i, j \in \{1, \ldots, k\}$ mit $i \neq j$ gilt also:

$$E(P_i) \cap E(P_j) = \emptyset.$$

Im Netzwerkkontext entsprechen kantendisjunkte Pfade den sogenannten Edge-Disjoint Paths (EDPs). Die Existenz und Berechnung solcher Pfade ist ein klassisches Problem

der Graphentheorie und wurde unter anderem in der Arbeit von Suurballe und Tarjan untersucht, die effiziente Algorithmen zur Bestimmung kürzester kantendisjunkter Pfade bereitstellt [13]. Auch in der Netzwerk- und Routingliteratur werden EDPs als Standardmodell für robuste Mehrpfadkommunikation verwendet [14].

Da sich kantendisjunkte Pfade (EDPs) keine Kanten teilen, ist diese Eigenschaft besonders für Linkausfälle relevant. Fällt eine Kante auf einem Pfad aus, so betrifft dieser Ausfall nicht direkt die anderen kantendisjunkten Pfade. Diese Eigenschaft wird in der Literatur häufig als Grundlage für zuverlässiges Routing und Traffic Engineering verwendet [14].

Kantendisjunkte Pfade (EDPs) dürfen sich jedoch innere Knoten teilen. Das bedeutet, dass zwei EDPs zwar unterschiedliche Kanten verwenden, aber trotzdem über denselben Zwischenknoten laufen können. Für reine Linkausfälle ist das zunächst unproblematisch. Bei Knotenausfällen kann ein solcher gemeinsamer Zwischenknoten jedoch kritisch sein, weil der Ausfall dieses Knotens mehrere Pfade gleichzeitig unbrauchbar machen kann.

Eine stärkere Form der Disjunktheit sind knotendisjunkte Pfade. Eine Menge von Pfaden $P_{gesamt} = \{P_1, P_2, \ldots, P_k\}$ zwischen s und d heißt knotendisjunkt, wenn sich keine zwei Pfade einen inneren Knoten teilen.
Für alle $i, j \in \{1, \ldots, k\}$ mit $i \neq j$ gilt also:

$$V_{\text{int}}(P_i) \cap V_{\text{int}}(P_j) = \emptyset.$$

Im Netzwerkkontext entsprechen diese Pfade den knotendisjunkten Pfaden engl. Node-Disjoint Paths (NDPs). Quelle s und Ziel d dürfen dabei gemeinsam verwendet werden, da alle betrachteten Pfade zwischen denselben Endpunkten verlaufen.

Abbildung 2 veranschaulicht beide Pfadtypen auf demselben Graphen. Die EDPs verwenden unterschiedliche Kanten, können jedoch gemeinsame innere Knoten besitzen. Bei den NDPs treten solche gemeinsamen inneren Knoten nicht auf.

Der Unterschied zwischen EDPs und NDPs ist für diese Arbeit wesentlich. EDPs schützen vor allem gegen den Ausfall einzelner Links, weil kein Link in mehreren Pfaden vorkommt. NDPs schützen zusätzlich gegen den Ausfall innerer Knoten, weil auch diese nicht mehrfach verwendet werden. Damit sind NDPs eine strengere Bedingung als EDPs. Für einfache Pfade zwischen demselben Quell- und Zielknoten impliziert Knotendisjunktheit stets Kantendisjunktheit, da ein gemeinsamer innerer Knoten zwangsläufig mehrere Kanten in beiden Pfaden verbinden würde. Umgekehrt gilt dies nicht: Zwei Pfade können kantendisjunkt sein und sich trotzdem einen inneren Knoten teilen, wenn dieser Knoten mindestens vier Kanten besitzt. Knotenausfälle können daher stärkere Auswirkungen auf die vorhandenen Routen haben als einzelne Linkausfälle [15].

Knotendisjunkte Pfade besitzen außer dem Startknoten s und dem Zielknoten d keine gemeinsamen Knoten. In Kommunikationsnetzen können sie als voneinander unabhängige Arbeits- und Ersatzpfade eingesetzt werden. Dadurch soll verhindert werden, dass der Ausfall eines inneren Knotens gleichzeitig mehrere verfügbare Routen unterbricht.

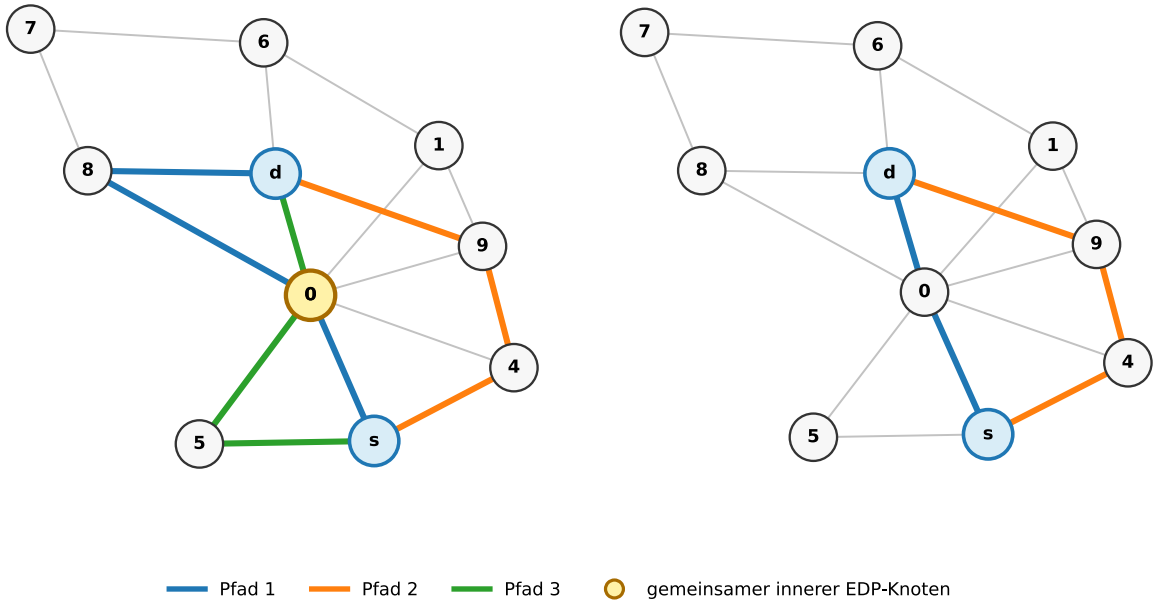


Abbildung 2: Kantendisjunkte Pfade (links) und knotendisjunkte Pfade (rechts) im selben Graphen. Die Farben unterscheiden die einzelnen Pfade.

Knotendisjunkte Pfade bilden daher eine mögliche Grundlage für zuverlässige und ausfallsichere Routingverfahren [16].

Das ursprüngliche TREE-Verfahren verwendet EDPs als Ausgangsstrukturen [2]. Diese Arbeit ergänzt NDP-basierte Varianten und untersucht beide Pfadtypen bei Kanten- und Knotenausfällen. Beide Pfadtypen bilden damit die Ausgangsstrukturen für die in dieser Arbeit verglichenen Routingverfahren.

Für die Berechnung der disjunkten Pfade wird in der Implementierung NetworkX verwendet. NetworkX stellt Funktionen zur Berechnung von kantendisjunkten und knotendisjunkten Pfaden bereit. Für EDPs wird die Funktion `edge_disjoint_paths` verwendet, für NDPs die Funktion `node_disjoint_paths` [17]. Die so berechneten Pfade bilden anschließend die Grundlage für die in dieser Arbeit betrachteten Routingverfahren.

2.5 Ausgangspunkt der Arbeit: TREE-Algorithmen

Ausgangspunkt dieser Arbeit ist das Verfahren *TREE – Tree Routing to Extend Edge Disjoint Paths* von Schweiger et al. [2]. Reine EDP-Verfahren können nur so viele kantendisjunkte Routen verwenden, wie es die Kantenkonnektivität zwischen Quelle und Ziel erlaubt. Weitere Kanten des Netzwerks bleiben dabei möglicherweise ungenutzt.

TREE erweitert deshalb ausgewählte EDPs zu Baumstrukturen, um zusätzliche lokale Ausweichmöglichkeiten zu schaffen.

Als einfachste Ausgangsvariante kann ein Routingverfahren nur die berechneten EDPs verwenden. Diese Variante wird in dieser Arbeit als *EDP Paths* bezeichnet. Dabei werden mehrere kantendisjunkte Pfade zwischen s und d vorberechnet. Beim Routing wird zunächst ein Pfad verwendet. Trifft ein Paket auf einen ausgefallenen Link, wird zurückgegangen und ein anderer Pfad ausprobiert. Dieser Ansatz ist einfach und liefert oft kurze Pfade, kann aber lange Umwege erzeugen, wenn ein Ausfall erst nahe am Ziel erkannt wird und das Paket dann weit zurücklaufen muss.

Die Blätter dieser Bäume liegen an Knoten, die benachbart zum Ziel d sind. Dadurch kann ein Paket innerhalb des Baumes mittels Tiefensuche weitergeleitet werden und bei einem Ausfall lokal in einen anderen Ast wechseln. Im Idealfall muss das Paket dadurch nicht bis zur Quelle zurücklaufen, sondern kann eine nähere Umleitung zum Ziel verwenden [2].

Die erste TREE-Variante heißt *One Tree*. Dabei wird nur ein einzelner EDP zu einem Baum erweitert und wird dafür der längste der berechneten EDPs ausgewählt. Dieser Pfad wird als Grundlage des Baumes verwendet und anschließend um weitere erreichbare Knoten und Kanten erweitert. Danach wird der Baum durch Trunkierungsverfahren bereinigt. Äste, die nicht zu einem Knoten führen, der mit d benachbart ist, werden entfernt. Das Ergebnis ist ein Baum, dessen relevante Blätter jeweils eine direkte Verbindung zum Ziel besitzen. One Tree soll zusätzliche Ausweichmöglichkeiten schaffen, aber gleichzeitig die gute Pfadlänge der ursprünglichen EDPs möglichst erhalten [2].

Die zweite TREE-Variante heißt *Multiple Trees*. Hier wird nicht nur ein einzelner Pfad erweitert, sondern es werden mehrere Bäume auf Basis der EDPs konstruiert. Die EDPs werden dabei zunächst absteigend nach ihrer Länge sortiert, sodass längere Pfade zuerst erweitert werden. Im Unterschied zu One Tree arbeitet Multiple Trees mit Kantenmarkierungen, weil mehrere Baumstrukturen im Netzwerk entstehen. Das Ziel d selbst ist nicht Teil der Bäume; stattdessen enden die Bäume an Knoten, die mit d benachbart sind. Auch hier werden anschließend redundante Äste entfernt [2].

Das Routing auf diesen Strukturen erfolgt im TREE-Paper mit einem Verfahren, das als *Depth First Routing* beschrieben wird. Zunächst werden die EDPs berechnet und für die Baumkonstruktion absteigend nach Länge sortiert. Danach wird EDPs, One Tree oder Multiple Trees angewendet und die entstandenen Bäume bei den erweiterten Tree-Varianten werden gekürzt. Für das eigentliche Routing werden die ursprünglichen EDPs anschließend aufsteigend nach Länge sortiert, sodass kurze Wege zuerst ausprobiert werden. Das Paket wird dann entlang der Pfade beziehungsweise Bäume in Tiefensuchreihenfolge weitergeleitet, bis das Ziel d erreicht wird oder keine nutzbare Weiterleitung mehr möglich ist [2].

Für die Reihenfolge innerhalb eines Baumes verwendet TREE zusätzlich eine lokale Rangordnung. An Knoten mit mehreren möglichen Ästen wird bevorzugt der Ast gewählt,

der die kürzeste Entfernung zu einem Blatt unterhalb dieses Astes besitzt. Dadurch soll beim Routing möglichst früh ein kurzer Weg in Richtung Ziel gefunden werden. Für One Tree wird außerdem zuerst der ursprüngliche EDP-Ast ausprobiert, bevor zusätzliche Baumäste verwendet werden. Dadurch kann One Tree die Hop-Count-Qualität der EDPs erhalten, solange die ursprünglichen EDPs das Ziel noch erreichen [2].

Die Routingregeln können als statische Routingtabellen umgesetzt werden. Da die EDPs und Baumstrukturen vorab berechnet werden, müssen die Regeln nur an den einzelnen Knoten installiert werden. Die Weiterleitung kann dann lokal anhand des Eingangsports und der am Knoten sichtbaren Ausfälle entschieden werden. Damit bleibt das Verfahren im Fast-Failover-Modell, in dem keine globale Neuberechnung während des Ausfalls notwendig ist [2].

Das ursprüngliche TREE-Verfahren betrachtet Linkausfälle und verwendet EDPs als Ausgangsstrukturen. Knotenausfälle und NDP-basierte Varianten gehören nicht zum untersuchten Modell. Das Paper nennt außerdem den zusätzlichen Routingtabellenbedarf der Baumstrukturen als möglichen Nachteil [2]. Diese Arbeit ergänzt das Verfahren um Knotenausfälle sowie NDP-basierte Varianten von Paths, One Tree und Multiple Trees.

Dieser Abschnitt beschreibt das Verfahren des TREE-Papers. Die vorliegende Implementierung übernimmt dessen grundlegendes Routingprinzip, verwendet jedoch eine eigene deterministische Reihenfolge der Strukturen und Baumäste. Die Unterschiede werden in Abschnitt 4.5 erläutert.

3 Konzeption der Erweiterung

Die im TREE-Ansatz vorgestellten Fast-Failover-Routingverfahren verwenden kantendisjunkte Pfade und wurden ursprünglich für den Umgang mit Link Failures entwickelt [2]. In dieser Arbeit werden die bestehenden Verfahren erweitert, um neben ausgefallenen Kanten auch ausgefallene Knoten berücksichtigen zu können. Darüber hinaus werden knotendisjunkte Pfade als zusätzliche Pfadgrundlage untersucht, auf deren Basis die bestehenden Pfad-, One-Tree- und Multiple-Trees-Strukturen analog zu den EDP-basierten Varianten berechnet werden. Ziel ist es, die vorhandenen Algorithmen möglichst systematisch zu übertragen, sodass die ursprünglichen EDP-basierten Verfahren und die neu entwickelten NDP-basierten Varianten unter vergleichbaren Bedingungen betrachtet werden können.

Dieses Kapitel beschreibt die Konzeption der dafür vorgenommenen Erweiterungen. Zunächst wird in Abschnitt 3.1 erläutert, wie die Knotenausfälle (Node Failures) im verwendeten Graphmodell abgebildet werden und welche Änderungen sich daraus für das Routing ergeben. Anschließend behandelt Abschnitt 3.2 die Berechnung von knotendisjunkten Pfaden (NDPs) und grenzt diese von kantendisjunkten Pfaden (EDPs) ab. Darauf aufbauend beschreibt Abschnitt 3.3 die Übertragung des One-Tree- und des Multiple-Trees-Algorithmus auf knotendisjunkte Pfade (NDPs). Abschließend gibt Abschnitt 3.4 einen Überblick über die daraus entstehenden Routingverfahren und deren Einordnung in die Gesamtarchitektur.

3.1 Erweiterung auf Knotenausfälle (Node Failures)

Die ursprünglichen TREE-Verfahren wurden für die Behandlung von Linkausfälle (Link Failures) entwickelt. Ein solcher Ausfall betrifft eine einzelne Kante, die während des Routings nicht mehr verwendet werden kann [2]. Bei einem Knotenausfall fällt dagegen ein vollständiger Knoten zusammen mit allen an ihn angrenzenden Kanten aus. Dadurch kann ein einzelner Knotenausfall gleichzeitig mehrere vorberechnete Routingpfade oder Zweige einer Baumstruktur beeinträchtigen. Für die Erweiterung muss daher zunächst festgelegt werden, wie ausgefallene Knoten im bereits vorhandenen Modell für Kantenausfälle abgebildet werden.

Abschnitt 3.1.1 beschreibt zunächst, wie ausgefallene Knoten im bestehenden Modell für Kantenausfälle abgebildet werden. Anschließend erklärt Abschnitt 3.1.2, wie die daraus abgeleitete Menge ausgefallener Kanten in die vorhandenen Routingabläufe integriert wird.

3.1.1 Modellierung

Das Netzwerk wird weiterhin als ungerichteter Graph $G = (V, E)$ betrachtet. Dabei bezeichnet V die Menge der Knoten und E die Menge der Kanten. Die Menge der ausgefallenen Knoten wird mit F_V bezeichnet. Quelle s und Ziel d werden von dieser Menge ausgeschlossen.

Der Ausfall der Quelle oder des Ziels würde unabhängig vom verwendeten Routingverfahren zwangsläufig zu einem Fehlschlag führen. Solche Fälle ermöglichen daher keinen aussagekräftigen Vergleich der untersuchten Verfahren und werden nicht berücksichtigt.

Ein ausgefallener Knoten wird durch den gleichzeitigen Ausfall aller seiner inzidenten Kanten modelliert. Dabei handelt es sich um sämtliche Kanten, die den ausgefallenen Knoten direkt mit einem seiner Nachbarn verbinden. Aus der Menge F_V wird deshalb die Menge ausgefallener Kanten $F_E(F_V)$ abgeleitet.

Diese Menge $F_E(F_V)$ enthält somit genau die Kanten, die zu mindestens einem ausgefallenen Knoten inzident sind. Falls zwei ausgefallene Knoten durch eine gemeinsame Kante verbunden sind, wird diese Kante aufgrund der Mengendarstellung nur einmal in $F_E(F_V)$ aufgenommen. Ein Node Failure wird im vorhandenen Fehlermodell damit als eine zusammengehörige Gruppe gleichzeitig ausgefallener Kanten dargestellt.

Das nach den Ausfällen verfügbare Netzwerk kann mathematisch als

$$\big(V \setminus F_V,\ E \setminus F_E(F_V)\big)$$

aufgefasst werden. In der verwendeten Darstellung werden die ausgefallenen Knoten und ihre Kanten jedoch nicht tatsächlich aus dem gespeicherten ursprünglichen Graphen entfernt. Stattdessen werden sämtliche inzidenten Kanten in die an das Routing übergebene Ausfallmenge aufgenommen und während der Weiterleitung als nicht verfügbar behandelt.

Ein ausgefallener Knoten besitzt dadurch im für das Routing verfügbaren Netzwerk keine nutzbare inzidente Kante. Er kann weder erreicht noch verlassen werden und ist somit hinsichtlich des Routingverlaufs isoliert. Für die Erreichbarkeit zwischen nicht ausgefallenen Knoten ist diese Darstellung gleichbedeutend mit seiner Entfernung aus dem Netzwerk. Gleichzeitig kann das bereits vorhandene Modell zur Verarbeitung ausgefallener Kanten weiterverwendet werden.

3.1.2 Änderungen im Routing

Für die Behandlung von Knotenausfällen ist kein eigener Routingablauf erforderlich. Vor dem Routing wird die in Abschnitt 3.1.1 definierte Menge ausgefallener Knoten F_V in die zugehörige Kantenmenge $F_E(F_V)$ überführt. Anschließend wird diese Kantenmenge genauso verarbeitet wie bei einem Link Failure.

Da alle inzidenten Kanten eines ausgefallenen Knotens gesperrt sind, kann dieser Knoten während des Routings weder erreicht noch verlassen werden. Das jeweilige Verfahren verwendet stattdessen die nächste verfügbare Alternative seiner vorberechneten Pfad- oder Baumstruktur.

Die Routingstrukturen werden auf dem ursprünglichen Graphen G berechnet, ohne die für den jeweiligen Routinglauf festgelegten Ausfälle zu berücksichtigen. Während des Routings werden die vorberechneten Pfade und Baumstrukturen weder neu berechnet noch an die aufgetretenen Ausfälle angepasst.

Auch die simulierten Routingentscheidungen bleiben lokal. An einem Knoten werden lediglich die dort erkennbaren inzidenten Kantenausfälle und die vorberechneten Weiterleitungsregeln berücksichtigt. Eine globale Neuberechnung der Routingstrukturen findet nicht statt [2]. Auf diese Weise bleiben die grundlegenden Annahmen des TREE-Modells erhalten, während zusätzlich Knotenausfälle untersucht werden können. Die Implementierung erzeugt dabei keine konkreten Routingtabellen, sondern simuliert das lokale Weiterleitungsverhalten auf den vorberechneten Pfad- und Baumstrukturen.

Zwischen Link und Node Failures besteht dennoch ein wesentlicher Unterschied. Ein einzelner Link Failure unterbricht genau eine Kante. Ein Node Failure kann dagegen gleichzeitig mehrere Kanten und damit auch mehrere vorberechnete Routingpfade oder Baumzweige unterbrechen.

Die unterschiedlichen Auswirkungen von Kanten- und Knotendisjunktheit auf Knotenausfälle motivieren die in Abschnitt 3.2 betrachtete Erweiterung auf Node-Disjoint Paths. Eine ausführliche Gegenüberstellung beider Pfadtypen erfolgt in Abschnitt 3.2.2.

3.2 Erweiterung auf Node-Disjoint Paths

Die ursprünglichen TREE-Verfahren verwenden Edge-Disjoint Paths als Pfadgrundlage [2]. Als zusätzliche Pfadgrundlage werden in dieser Arbeit Node-Disjoint Paths untersucht. Ihre Verwendung soll zeigen, wie sich eine stärkere Trennung der Basispfade auf das Routing bei Knotenausfällen auswirkt. Die Berechnung der NDPs wird in Abschnitt 3.2.1 beschrieben. Ihre Unterschiede zu EDPs und die daraus entstehenden Vor- und Nachteile werden anschließend in Abschnitt 3.2.2 gegenübergestellt.

3.2.1 Berechnung der NDPs

Zwei Pfade P_i und P_j sind knotendisjunkt, wenn sie außer s und d keine weiteren Knoten gemeinsam besitzen. Für alle unterschiedlichen Pfade einer Menge von NDPs gilt daher:

$$NDPs = \{P_1, P_2, \ldots, P_k\} \texttt{ mit } V_{\text{int}}(P_i) \cap V_{\text{int}}(P_j) = \emptyset \texttt{ für alle } i, j \in \{1, \ldots, k\} \texttt{ mit } i \neq j$$

Die gemeinsame Verwendung von s und d ist notwendig, da sämtliche Pfade dieselbe Quelle mit demselben Ziel verbinden. Genau genommen handelt es sich daher um intern knotendisjunkte Pfade (NDPs).

Für ihre Berechnung wird die von NetworkX bereitgestellte Funktion `node_disjoint_paths` verwendet. Sie führt eine flussbasierte Berechnung auf einem gerichteten Hilfsgraphen durch. Durch eine geeignete Aufteilung und Kapazitätsbeschränkung der Knoten wird sichergestellt, dass jeder innere Knoten in höchstens einem der berechneten Pfade vorkommt. Auf diese Weise wird eine größtmögliche Menge paarweise intern knotendisjunkter Pfade zwischen s und d bestimmt [18].

Die Flussberechnung maximiert dabei in erster Linie die Anzahl der knotendisjunkten Pfade. Sie garantiert jedoch nicht, dass die einzelnen Pfade oder ihre Gesamtlänge minimal sind. Daher kann die berechnete Pfadmenge auch längere Wege enthalten, obwohl für einen einzelnen Pfad möglicherweise eine kürzere Route vorhanden wäre.

Die NDPs werden wie die zuvor verwendeten EDPs als Folgen von Knoten dargestellt. Dadurch können sie ohne Änderung ihrer grundlegenden Darstellung als Ausgangsstrukturen für das direkte Pfadrouting sowie für den One-Tree- und den Multiple-Trees-Algorithmus verwendet werden. Die konkrete Verwendung der NDPs innerhalb dieser Verfahren wird in Abschnitt 3.3 beschrieben.

Der Aufruf der NetworkX-Funktion, die Verarbeitung der zurückgegebenen Pfade und deren Sortierung werden im Implementierungskapitel 4 näher erläutert.

3.2.2 Unterschiede zu EDPs

Der wesentliche Unterschied zwischen EDPs und NDPs liegt in der geforderten Form der Disjunktheit. Für zwei Edge-Disjoint Paths P_i und P_j wird lediglich verlangt, dass ihre Kantenmengen disjunkt sind. Formal gilt:

$$EDPs = \{P_1, P_2, \ldots, P_k\} \texttt{ mit } E(P_i) \cap E(P_j) = \emptyset \texttt{ für alle } i, j \in \{1, \ldots, k\} \texttt{ mit } i \neq j$$

Gemeinsame innere Knoten sind dabei erlaubt. Zwei EDPs können deshalb denselben Knoten über unterschiedliche eingehende und ausgehende Kanten durchlaufen, ohne ihre Kantendisjunktheit zu verletzen. Ein Ausfall dieses Knotens kann folglich beide Pfade gleichzeitig unterbrechen.

Bei NDPs ist die Disjunktheitsbedingung strenger. Da zwei NDPs keine gemeinsamen inneren Knoten besitzen dürfen, können sie auch keine gemeinsame Kante verwenden. Eine Menge paarweise knotendisjunkter Pfade ist damit zugleich kantendisjunkt. Umgekehrt ist jedoch nicht jede Menge kantendisjunkter Pfade auch knotendisjunkt.

Dieser Unterschied ist in Abbildung 2 veranschaulicht: Die EDPs können gemeinsame innere Knoten besitzen, während die NDPs nur den Quell- und Zielknoten gemeinsam verwenden.

Für die maximal mögliche Anzahl der Pfade zwischen s und d gilt daher:

$$k_{\mathrm{NDP}}(s, d) \leq k_{\mathrm{EDP}}(s, d).$$

Die stärkere Unabhängigkeit der NDPs ist insbesondere für Node Failures von Bedeutung. Da jeder innere Knoten in höchstens einem NDP enthalten ist, kann der Ausfall eines einzelnen inneren Knotens höchstens einen der vorberechneten NDPs direkt unterbrechen. Dabei wird entsprechend der in Abschnitt 3.1 beschriebenen Modellierung vorausgesetzt, dass Quelle s und Ziel d nicht als ausgefallene Knoten ausgewählt werden. Bei EDPs kann ein einzelner Knotenausfall dagegen mehrere Pfade gleichzeitig betreffen, wenn diese den ausgefallenen Knoten gemeinsam verwenden.

Die strengere Disjunktheitsbedingung bringt jedoch auch mögliche Nachteile mit sich. Zwischen derselben Quelle und demselben Ziel können weniger NDPs als EDPs vorhanden sein. Dadurch stehen den NDP-basierten Verfahren möglicherweise weniger vorberechnete Ausweichpfade zur Verfügung. Beim Multiple-Trees-Algorithmus kann dies außerdem dazu führen, dass weniger Bäume erzeugt werden.

Darüber hinaus können für die Herstellung der Knotendisjunktheit längere Routen erforderlich sein, wenn kürzere Wege innere Knoten gemeinsam verwenden würden. Da die verwendete Flussberechnung die Anzahl der NDPs maximiert, aber ihre individuelle oder gesamte Länge nicht minimiert, lässt sich aus der Knotendisjunktheit allein kein Vorteil hinsichtlich der Hopzahl ableiten.

NDPs bieten somit eine stärkere Trennung der vorberechneten Basispfade gegenüber Knotenausfällen, während EDPs die vorhandenen Knoten und Kanten weniger eingeschränkt nutzen können. Ob die stärkere Trennung mögliche Nachteile wie eine geringere Pfadanzahl oder längere Routen ausgleicht, wird in der späteren Evaluation empirisch untersucht.

Die Knotendisjunktheit bezieht sich zunächst ausschließlich auf die berechneten Basispfade. Werden diese anschließend zu Baumstrukturen erweitert, müssen die ergänzten Zweige nicht ebenfalls knotendisjunkt sein. Diese Übertragung der NDPs auf die TREE-Algorithmen und die daraus entstehenden Einschränkungen werden im folgenden Abschnitt 3.3 erläutert.

3.3 Erweiterung der Algorithmen

Die ursprünglichen TREE-Verfahren verwenden kantendisjunkte Pfade (EDPs) als Grundlage für das Routing. Während das einfache Pfadverfahren direkt entlang dieser Pfade

routet, erweitern der One-Tree- und der Multiple-Trees-Algorithmus die vorberechneten Pfade um zusätzliche Baumzweige. Dadurch können weitere Kanten des Netzwerks als Ausweichmöglichkeiten genutzt werden [2].

Für die Erweiterung auf knotendisjunkten Pfaden (NDPs) werden die bisher verwendeten EDPs durch die in Abschnitt 3.2 beschriebenen NDPs ersetzt. Die grundlegende Arbeitsweise der Algorithmen bleibt dabei erhalten. Dies betrifft insbesondere die Verwendung der disjunkten Pfade als Ausgangsstrukturen, ihre Erweiterung zu Bäumen, den Ausschluss des Zielknotens aus den gespeicherten Bäumen sowie deren anschließende Trunkierung. Auch das bestehende Routingverfahren kann grundsätzlich weiterverwendet werden, da EDPs und NDPs jeweils als Folgen von Knoten dargestellt werden.

Aus dieser Übertragung entstehen neben dem direkten NDP-Pfadverfahren die beiden neuen Varianten NDP One Tree und NDP Multiple Trees. Im Folgenden werden ihre wesentlichen konzeptionellen Eigenschaften beschrieben. Die genaue Umsetzung der einzelnen Verarbeitungsschritte wird anschließend im Implementierungskapitel 4 erläutert.

Abschnitt 3.3.1 beschreibt zunächst, wie einer der längsten NDPs zu einer One-Tree-Struktur erweitert und gemeinsam mit den unveränderten NDPs für das Routing verwendet wird. Anschließend erklärt Abschnitt 3.3.2, wie aus jedem berechneten NDP eine eigene Baumstruktur erzeugt und beim Routing eingesetzt wird. Abschnitt 3.3.3 fasst abschließend die Besonderheiten der NDP-basierten Varianten sowie ihre Unterschiede zu den ursprünglichen EDP-basierten Verfahren zusammen.

3.3.1 One-Tree-Algorithmus

Beim One-Tree-Algorithmus wird genau einer der berechneten NDPs zu einem Baum erweitert. Hierfür werden die Pfade zunächst nach ihrer Länge geordnet. Ein durch die deterministische Sortierung ausgewählter längster NDP dient als Basispfad des Baumes. Die übrigen NDPs bleiben als unveränderte Pfadstrukturen erhalten. Dieses Vorgehen entspricht grundsätzlich der ursprünglichen EDP-basierten One-Tree-Variante [2].

Der ausgewählte Basispfad verbindet die Quelle s mit dem Ziel d. Der Zielknoten selbst wird jedoch nicht in den gespeicherten Baum aufgenommen. Der Basispfad innerhalb des Baumes endet daher zunächst am vorletzten Knoten des NDPs, der unmittelbar mit d verbunden ist.

Ausgehend von den inneren Knoten des Basispfades wird der Baum um weitere Knoten und Kanten des Graphen ergänzt. Die Quelle dient nicht als Ausgangspunkt des Baumwachstums.

Nach seiner Erweiterung wird der Baum trunkiert. Dabei bleiben nur solche Kanten erhalten, die auf einem Weg von s zu einem Baumknoten liegen, der direkt mit d verbunden

ist. Zweige, die keine Möglichkeit zum Erreichen des Ziels bieten, werden entfernt. Der Zielknoten bleibt außerhalb des gespeicherten Baumes. Während des Routings kann der Baum über einen geeigneten Ausgangsknoten verlassen werden, sofern dessen direkte Verbindung zu d verfügbar ist.

Abbildung 3 zeigt die resultierenden One-Tree-Strukturen für beide Pfadgrundlagen. Jeweils der längste Basispfad wird erweitert, während die übrigen Pfade unverändert bleiben.

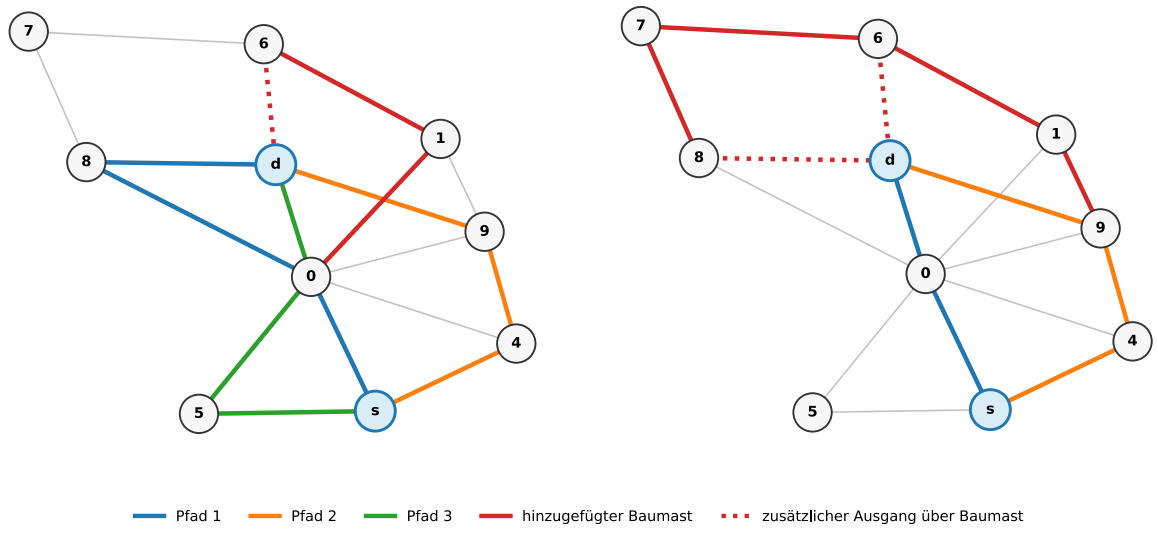


Abbildung 3: EDP One Tree (links) und NDP One Tree (rechts) nach Erweiterung und Trunkierung. Zusätzliche Baumkanten sind rot dargestellt; rot gepunktete Kanten kennzeichnen zusätzliche Verbindungen von Ausgangsknoten zum Ziel d. Diese Zielverbindungen gehören nicht zum gespeicherten Baum.

Beim simulierten Routing werden zunächst die nicht zu einem Baum erweiterten NDPs in aufsteigender Reihenfolge ihrer Länge betrachtet. Der als Baumgrundlage ausgewählte NDP wird nicht zusätzlich als unveränderter Pfad geroutet, sondern bleibt als bevorzugter Basispfad in der anschließend betrachteten One-Tree-Struktur enthalten. Innerhalb des Baumes wird die Weiterleitung durch ein ranggeordnetes tiefenorientiertes Verfahren simuliert. Bei jeder Verzweigung erhält zunächst der Nachfolger auf dem ursprünglichen Basispfad Vorrang. Ist die entsprechende Kante ausgefallen oder führt der gewählte Zweig nicht erfolgreich zum Ziel, wird das Zurückkehren entlang der bereits durchlaufenen Kanten simuliert und anschließend ein anderer Zweig untersucht.

Das One-Tree-Verfahren orientiert sich damit zunächst am Verhalten des direkten NDP-Routings. Die zusätzlichen Baumzweige werden insbesondere dann relevant, wenn die ursprünglichen Pfade durch Ausfälle unterbrochen sind. Da der Baum jedoch auch über ergänzte Ausgangsknoten verlassen werden kann, muss die tatsächlich zurückgelegte Route nicht in jedem Fall mit dem ursprünglichen NDP übereinstimmen.

3.3.2 Multiple-Trees-Algorithmus

Beim Multiple-Trees-Algorithmus dient jeder berechnete NDP als Basispfad für einen eigenen Baum. Die Anzahl der erzeugten Bäume entspricht damit grundsätzlich der Anzahl der zwischen s und d berechneten NDPs.

Für die Konstruktion werden die NDPs ebenfalls nach ihrer Länge geordnet und beginnend mit dem längsten Pfad verarbeitet. Jedem Basispfad werden anschließend zusätzliche, noch nicht verwendete Kanten zugeordnet. Die kürzeren Basispfade bleiben dabei vollständig in ihren jeweiligen Bäumen erhalten. Da ihre Erweiterung später erfolgt, stehen für sie tendenziell weniger unbenutzte Kanten zur Verfügung. Ihre Baumstrukturen können dadurch näher an den ursprünglichen kurzen Pfaden bleiben [2].

Wie beim One-Tree-Algorithmus wird der Zielknoten d nicht in die gespeicherten Bäume aufgenommen. Jeder Basispfad endet innerhalb seines Baumes daher an einem Nachbarknoten von d. Die Bäume werden anschließend um zusätzliche Knoten und Kanten erweitert und danach einzeln trunkiert. Erhalten bleiben nur die Teile eines Baumes, die zu einem möglichen Ausgang in Richtung des Zielknotens führen.

Bei der Konstruktion werden die Kanten der Basispfade sowie die nachträglich hinzugefügten Kanten jeweils einem Baum zugeordnet. Eine zusätzliche Kante darf nur dann verwendet werden, wenn sie keinem anderen Baum zugewiesen wurde. Die gespeicherten Baumstrukturen bleiben dadurch untereinander kantendisjunkt. Diese Eigenschaft bezieht sich auf die Bäume ohne den Zielknoten d. Die abschließenden Verbindungen von den Ausgangsknoten zu d gehören nicht zu den gespeicherten Bäumen. Außerdem können unterschiedliche Bäume gemeinsame Knoten enthalten, sofern sie diese über verschiedene Kanten erreichen.

Abbildung 4 zeigt die Erweiterung aller Basispfade zu Bäumen. Auch bei der NDP-basierten Variante können nach dem Wachstum gemeinsame innere Knoten auftreten, da zwischen den Bäumen Kanten, jedoch keine Knoten reserviert werden.

Da bei diesem Algorithmus jeder berechnete NDP zu einem eigenen Baum erweitert wird, werden keine NDPs zusätzlich als unveränderte Pfade geroutet. Beim simulierten Routing werden die Bäume in aufsteigender Reihenfolge der Länge ihrer Basispfade betrachtet. Innerhalb jedes Baumes wird die Weiterleitung mithilfe eines ranggeordneten tiefenorientierten Verfahrens simuliert. Anders als beim One-Tree-Algorithmus erhält der ursprüngliche Basispfad dabei keinen besonderen Vorrang. An einer Verzweigung wird zunächst der Zweig mit der geringsten vorberechneten Entfernung zu einem möglichen Ausgangsknoten gewählt, der unmittelbar mit d verbunden ist. Ist die nächste Kante ausgefallen oder führt der untersuchte Zweig nicht zum Ziel, wird das Zurückkehren entlang der bereits durchlaufenen Kanten simuliert und anschließend ein anderer verfügbarer Zweig untersucht. Erst wenn alle nutzbaren Zweige eines Baumes ohne Erfolg geprüft wurden und die simulierte Weiterleitung zur Quelle s zurückgekehrt ist, wird mit dem nächsten Baum fortgefahren.

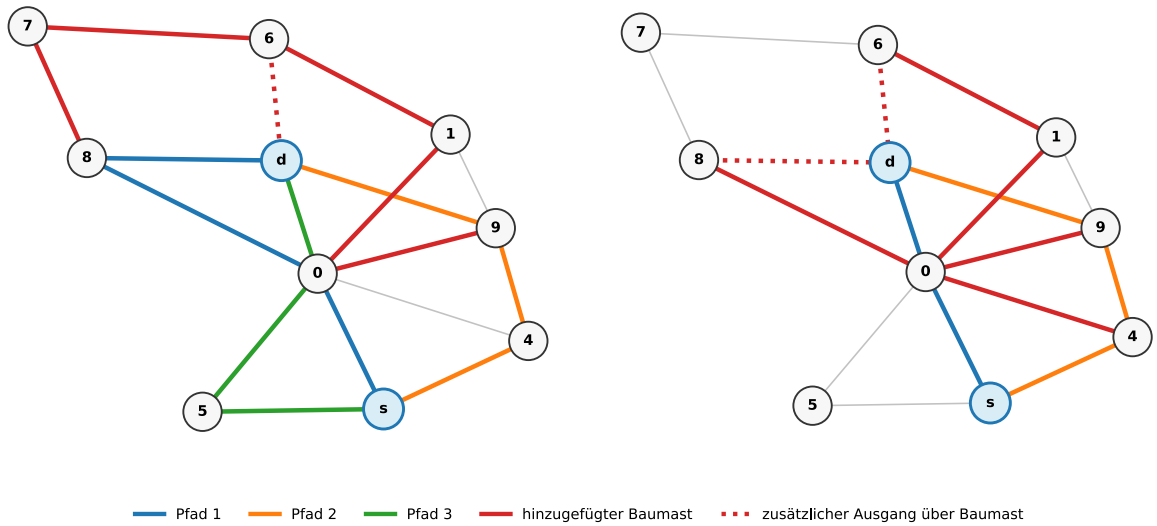


Abbildung 4: EDP Multiple Trees (links) und NDP Multiple Trees (rechts) nach Erweiterung und Trunkierung. Zusätzliche Baumkanten sind rot dargestellt; rot gepunktete Kanten kennzeichnen zusätzliche Verbindungen zum Ziel d. Diese Zielverbindungen gehören nicht zu den gespeicherten Bäumen.

Da mehrere Basispfade zu Bäumen erweitert werden, können alternative Baumzweige in mehreren Routingstrukturen entstehen. Wie viele zusätzliche nutzbare Wege nach der Trunkierung verbleiben, hängt jedoch vom jeweiligen Graphen ab. Das Durchlaufen zusätzlicher Zweige kann außerdem zu einer größeren Hopzahl und einem höheren Stretch führen.

3.3.3 Besonderheiten und Unterschiede zur EDP-Version

Der wesentliche Unterschied zu den ursprünglichen TREE-Algorithmen liegt in der verwendeten Pfadgrundlage. Die EDP-basierten Verfahren beginnen mit kantendisjunkten Pfaden, während die neuen Varianten knotendisjunkte Pfade als Basispfade verwenden. Die grundlegenden Regeln für die Erweiterung, Trunkierung und Verwendung der Baumstrukturen bleiben dagegen erhalten.

Da Knotendisjunktheit eine strengere Bedingung als Kantendisjunktheit darstellt, kann die Anzahl der gefundenen NDPs geringer als die Anzahl der EDPs zwischen derselben Quelle und demselben Ziel sein. Beim One-Tree-Algorithmus können dadurch weniger unveränderte Ausweichpfade zur Verfügung stehen. Beim Multiple-Trees-Algorithmus kann die geringere Pfadanzahl außerdem dazu führen, dass entsprechend weniger Bäume erzeugt werden.

Die Knotendisjunktheit gilt zunächst für die berechneten NDP-Basispfade. Beim One-Tree-Algorithmus schützt die Implementierung die Kanten und inneren Knoten der

übrigen NDPs, sodass diese nicht in den wachsenden Baum aufgenommen werden. Die innerhalb des One Trees entstehenden alternativen Routen sind jedoch nicht untereinander knotendisjunkt, da sie gemeinsame Abschnitte des Baumes verwenden können.

Beim Multiple-Trees-Algorithmus werden dagegen nur die Kanten global reserviert. Unterschiedliche Bäume können deshalb gemeinsame Knoten enthalten, solange sie keine gespeicherten Kanten gemeinsam verwenden. Ein Knotenausfall kann daher mehrere Multiple-Trees-Strukturen gleichzeitig beeinträchtigen.

Die Verwendung von NDPs verbessert somit gezielt die Trennung der ursprünglichen Basispfade gegenüber Knotenausfällen. Sie garantiert jedoch nicht, dass auch alle nachträglich erzeugten Baumzweige knotendisjunkt sind. Die Auswirkungen dieses Unterschieds auf Resilienz, Hopzahl und Stretch werden später in der Evaluation untersucht.

Für die Verarbeitung von Node Failures sind innerhalb der Baumalgorithmen keine zusätzlichen konzeptionellen Änderungen erforderlich. Wie in Abschnitt 3.1 beschrieben, wird ein ausgefallener Knoten durch den Ausfall aller seiner inzidenten Kanten modelliert. Das Routing behandelt diese Kanten als nicht verfügbar und versucht entsprechend der jeweiligen Routingstruktur einen anderen Zweig, Pfad oder Baum.

Das folgende Implementierungskapitel 4 beschreibt die konkrete Umsetzung dieser Verfahren. Dazu gehören die verwendeten Datenstrukturen, die deterministische Sortierung der Basispfade, das Wachstum und die Trunkierung der Bäume sowie der Ablauf des Routings bei Kanten- und Knotenausfällen.

3.4 Gesamtüberblick

Die in den vorangehenden Abschnitten beschriebenen Erweiterungen werden in einer gemeinsamen Architektur zusammengeführt. Sie übernimmt die Trennung zwischen der Vorberechnung der Routingstrukturen und der lokalen Weiterleitung aus dem TREE-Modell [2]. Konzeptionell gliedert sich die Architektur in die Berechnung der Basispfade, den Aufbau der Routingstrukturen, die Abbildung der Ausfälle und die anschließende Durchführung des Routings. Dadurch können die verwendete Pfadgrundlage, die Form der Routingstruktur und der untersuchte Ausfalltyp weitgehend unabhängig voneinander gewählt werden.

Ausgangspunkt ist ein ungerichteter Graph $G = (V, E)$ mit einer Quelle s und einem Ziel d. Für dieses Quelle-Ziel-Paar wird zunächst eine Menge disjunkter Basispfade berechnet. Als Pfadgrundlage können entweder Edge-Disjoint Paths oder Node-Disjoint Paths verwendet werden. Beide Pfadtypen besitzen dieselbe grundlegende Darstellung als Folgen von Knoten. Die nachfolgenden Verarbeitungsschritte können daher unabhängig davon ausgeführt werden, ob die Basispfade kanten- oder knotendisjunkt sind. Lediglich die Berechnung und die Disjunktheitseigenschaften der Ausgangspfade unterscheiden sich.

Auf Grundlage dieser Pfade wird eine von drei Routingstrukturen erzeugt. Beim direkten Pfadverfahren werden die berechneten Basispfade unverändert verwendet. Beim One-Tree-Verfahren wird der längste Basispfad zu einer einzelnen Baumstruktur erweitert, während die übrigen Pfade erhalten bleiben. Beim Multiple-Trees-Verfahren dient dagegen jeder Basispfad als Grundlage für einen eigenen Baum. In beiden Baumverfahren ist das Ziel d nicht Bestandteil der Bäume. Nach der Erweiterung werden Zweige entfernt, die nicht zu einem an d angrenzenden Knoten führen. Durch die Kombination der beiden Pfadtypen mit den drei Strukturvarianten entstehen insgesamt sechs Verfahren: EDP Paths, EDP One Tree, EDP Multiple Trees, NDP Paths, NDP One Tree und NDP Multiple Trees.

Die Erzeugung des Ausfallszenarios ist von der Berechnung der Routingstrukturen getrennt. Bei Link Failures wird unmittelbar eine Menge ausgefallener Kanten festgelegt. Bei Node Failures werden zunächst ausgefallene Knoten ausgewählt und anschließend, wie in Abschnitt 3.1 beschrieben, in die Menge ihrer inzidenten Kanten überführt. Ob die Link beziehungsweise Node Failures zufällig oder geclustert erzeugt werden, beeinflusst somit die Auswahl der Ausfälle, nicht jedoch die Schnittstelle zum Routing. Alle sechs Verfahren erhalten letztlich eine einheitliche Menge nicht verfügbarer Kanten.

Die Routingstrukturen werden auf dem ursprünglichen Graphen ohne Berücksichtigung des jeweiligen Ausfallszenarios vorbereitet. Während eines Routinglaufs werden sie weder neu berechnet noch verändert. Das Paket folgt abhängig vom ausgewählten Verfahren den vorberechneten Pfaden oder Baumzweigen. Trifft es auf eine ausgefallene Kante, verwendet das Verfahren die jeweils vorgesehene Alternative oder kehrt innerhalb der Routingstruktur zurück. Die Weiterleitungsentscheidungen beruhen dabei weiterhin nur auf den vorberechneten Regeln und den am aktuellen Knoten erkennbaren inzidenten Ausfällen.

Für einen Versuchsfall können alle ausgewählten Verfahren auf demselben Graphen, mit demselben Quelle-Ziel-Paar und demselben Ausfallszenario ausgeführt werden. Ein einzelner Routinglauf liefert zunächst, ob das Ziel erreicht wurde, sowie die dabei benötigte Hopzahl. Aus den wiederholten Routingläufen werden anschließend die Resilienz, die durchschnittliche Hopzahl und der durchschnittliche Stretch bestimmt. Die gemeinsame Eingabe und die einheitliche Behandlung der Ausfälle schaffen damit vergleichbare Bedingungen für die spätere empirische Untersuchung.

Die Architektur trennt somit drei zentrale Vergleichsdimensionen voneinander: die Wahl zwischen EDPs und NDPs, die Wahl zwischen direktem Pfadrouting, One Tree und Multiple Trees sowie die Wahl des Ausfalltyps und des Ausfallmodells. Eine Erweiterung auf Node Failures erfordert dadurch keine eigenen Routingverfahren, und die Verwendung von NDPs erfordert keine gesonderte Ausfallbehandlung. Diese Trennung bildet zugleich die Grundlage für die modulare Implementierung und die automatisierte Evaluationspipeline, die in den folgenden Kapiteln beschrieben werden.

4 Implementierung

Dieses Kapitel beschreibt die konkrete Umsetzung der in Kapitel 3 entwickelten Erweiterungen. Der Schwerpunkt liegt auf den neu hinzugefügten NDP-Varianten und ihrer Einbindung in das bestehende Framework, sodass die EDP- und NDP-basierten Verfahren unter denselben Versuchsbedingungen ausgeführt und miteinander verglichen werden können. Neben den Routingalgorithmen umfasst die Implementierung daher auch die Erzeugung der Ausfälle, die Speicherung und Auswertung der Ergebnisse sowie die Automatisierung des gesamten Versuchsablaufs.

Zunächst wird in Abschnitt 4.1 die Projektstruktur erläutert und die Aufteilung der zentralen Funktionen auf die einzelnen Module beschrieben. Anschließend wird in Abschnitt 4.2 die Implementierung der Verfahren NDP Paths, NDP One Tree und NDP Multiple Trees vorgestellt. Dabei werden insbesondere die Berechnung der knotendisjunkten Pfade sowie der Aufbau und die Trunkierung der Routingstrukturen betrachtet.

Daraufhin beschreibt Abschnitt 4.3 die Umsetzung der vier untersuchten Kombinationen aus Ausfalltyp und Ausfallmodell. Diese umfassen zufällige und geclusterte Kantenausfälle sowie zufällige und geclusterte Knotenausfälle.

Abschnitt 4.4 behandelt die Seed-basierte Steuerung der Randomisierung. Hier wird erläutert, wie die Erzeugung synthetischer Graphen, die Auswahl der Start- und Zielknoten sowie die Bestimmung der Ausfälle reproduzierbar durchgeführt werden. Dadurch kann bei gleicher Konfiguration und gleichem Seed dieselbe Folge von Versuchsszenen erneut erzeugt werden.

Das eigentliche Routing wird in Abschnitt 4.5 beschrieben. Der Abschnitt erläutert die Reihenfolge der Pfad- und Baumstrukturen, die Tiefensuche innerhalb einer Routingstruktur, den Umgang mit ausgefallenen Kanten sowie die Zählung der Hops und Rückwege.

Anschließend werden in Abschnitt 4.6 die Evaluierungskomponenten vorgestellt. Hierzu gehören die Speicherung der einzelnen Versuchsergebnisse in CSV-Dateien, ihre Aggregation anhand der Metriken Resilience, Average Hops und Average Stretch sowie die Erzeugung der zugehörigen Diagramme.

Abschließend beschreibt Abschnitt 4.7 die entwickelte Pipeline. Diese verbindet die Auswahl und Parametrisierung der Algorithmen, Ausfallmodelle und Graphen mit der Durchführung und Auswertung der Experimente. Die Pipeline kann sowohl über die Kommandozeile als auch über eine grafische Bedienoberfläche verwendet werden. Dadurch lässt sich der vollständige Ablauf von der JSON-Konfiguration über die Erzeugung der CSV-Dateien bis zur automatischen Erstellung der Plots wiederholbar ausführen.

4.1 Projektstruktur

Der technische Teil des Repositories ist entsprechend den Arbeitsschritten 1 bis 7 gegliedert. Diese Aufteilung dokumentiert die schrittweise Entwicklung von der Reproduktion der ursprünglichen TREE-Verfahren bis zu einer automatisierten und erweiterbaren Versuchsumgebung. Die einzelnen Entwicklungsstände bleiben dadurch voneinander getrennt und können nachträglich nachvollzogen werden.

Die ursprüngliche TREE-Implementierung befindet sich im Verzeichnis `external/TREE/`. In `schritt1_reproduktion/` wurden zunächst die ursprünglichen EDP-basierten Verfahren reproduziert. Schritt 2 ergänzt die Behandlung von Knotenausfällen. In Schritt 3 wurden knotendisjunkte Pfade hinzugefügt. Schritt 4 enthält den gemeinsamen Vergleich der EDP- und NDP-basierten Routingverfahren. Die automatisierte Versuchsumgebung wurde anschließend in Schritt 5 erstellt.

Die für die Experimente verwendete Implementierung befindet sich vollständig im Verzeichnis `schritt5_pipeline_automatisierung/`. Dieser Ordner kann unabhängig von den vorherigen Schritten verwendet werden. Er enthält die benötigten Python-Module, Konfigurationen und Real-World-Graphen. Dadurch muss die Pipeline während der Ausführung nicht auf Dateien aus den vorherigen Schritten zugreifen.

Der Schritt-5-Ordner enthält die folgenden Bestandteile:

- `pipeline.py` steuert den vollständigen Versuchsablauf.
- `gui.py` stellt eine grafische Bedienoberfläche bereit.
- `src/` enthält die Algorithmen, Fehlermodelle und die Durchführung einzelner Experimente.
- `evaluation/` enthält die Auswertung der CSV-Dateien und die Erstellung der Plots.
- `config/` enthält vorbereitete JSON-Konfigurationen.
- `data/topology_zoo/` enthält die verwendeten Real-World-Graphen.
- `Dokumentation/` enthält Anleitungen und Beschreibungen der einzelnen Module.

Die zentrale Komponente ist `pipeline.py`. Dieses Modul liest eine JSON-Konfiguration ein und erstellt daraus die geplanten Experimente. Danach führt es die Experimente aus und startet die Auswertung. Die Pipeline kann direkt über die Kommandozeile oder über die grafische Oberfläche aufgerufen werden.

Die grafische Oberfläche in `gui.py` enthält keine eigene Experimentlogik. Sie ermöglicht die Auswahl der Algorithmen, Fehlermodelle, Graphen und Parameter. Aus dieser Auswahl wird eine JSON-Konfiguration erstellt. Anschließend wird dieselbe Pipeline gestartet, die auch bei einer Ausführung über die Kommandozeile verwendet wird. Dadurch führen beide Bedienungsarten denselben Versuchsablauf aus.

Die eigentliche Experimentlogik ist auf drei Module unter `src/` verteilt. `routing_algorithms.py` enthält die sechs untersuchten Routingverfahren. Dazu gehören die EDP- und NDP-basierten Varianten von Paths, One Tree und Multiple Trees. Das Modul `failures.py` enthält die zufälligen und geclusterten Link- und Knotenausfälle. `experiment.py` erzeugt die Graphen, wählt Start- und Zielknoten aus und führt die ausgewählten Algorithmen mit den erzeugten Ausfällen aus. Die Ergebnisse werden danach in CSV-Dateien gespeichert.

Die Auswertung erfolgt durch `evaluation/result_analysis.py`. Dieses Modul liest die erzeugten CSV-Dateien ein und fasst die einzelnen Wiederholungen zusammen. Anschließend erstellt es die ausgewählten Vergleichsplots.

Für die Konfiguration stehen mit `pilot.json` und `full.json` zwei vorbereitete Dateien zur Verfügung. `pilot.json` ist für kurze Testläufe vorgesehen. `full.json` enthält die vollständigen Graph-Konfigurationen und verwendet 1000 Wiederholungen. Eigene Konfigurationen können auf diesen Dateien aufgebaut werden.

Alle Ergebnisse eines Laufs werden unter `Output/<Laufname>/` gespeichert. Der Unterordner `raw/` enthält die Rohdaten der einzelnen Experimente. Die zusammengefasste CSV-Datei wird unter `summaries/` gespeichert. Die erzeugten Abbildungen befinden sich unter `plots/`. Zusätzlich speichert die Pipeline die tatsächlich verwendete Konfiguration und ein Manifest des Laufs. Dadurch können die Ergebnisse später der verwendeten Konfiguration zugeordnet werden.

Die Verzeichnisse `schritt6_reproduzierbarkeit/` und `schritt7_erweiterung/` ergänzen die technische Implementierung. Sie enthalten die Unterlagen für das Nutzerfeedback sowie eine Anleitung zur Erweiterung der Umgebung um zusätzliche Graphen, Fehlermodelle und Routingalgorithmen.

4.2 Implementierung der Algorithmen

Die Routingalgorithmen sind im Modul `src/routing_algorithms.py` implementiert. Das Modul enthält sowohl die ursprünglichen EDP-basierten Verfahren als auch die in dieser Arbeit ergänzten NDP-basierten Varianten.

Für die NDP-Verfahren wurde keine vollständig getrennte Implementierung erstellt. Stattdessen verwenden die EDP- und NDP-Varianten gemeinsame Funktionen. Der Parameter `path_type` legt fest, ob kanten- oder knotendisjunkte Pfade berechnet werden. Der Parameter `structure_type` bestimmt anschließend, ob die Pfade direkt verwendet oder zu One-Tree- beziehungsweise Multiple-Trees-Strukturen erweitert werden.

Die Funktion `run_algorithms(...)` berechnet die NDPs einer Versuchsszene einmal und speichert sie in einem Zwischenspeicher. Danach erhalten NDP Paths, NDP One Tree und NDP Multiple Trees dieselbe Pfadmenge. Die Ergebnisse der drei Verfahren basieren dadurch auf denselben knotendisjunkten Pfaden.

Eine Routingstruktur wird durch die Klasse `RoutingStructure` beschrieben. Sie enthält den ursprünglichen Pfad, die daraus erzeugte Pfad- oder Baumstruktur und mögliche Ausgänge zum Zielknoten. Der Zielknoten selbst wird nicht als Teil des gespeicherten Baums abgelegt. Stattdessen speichert die Struktur die Knoten, von denen das Ziel direkt erreicht werden kann.

Das Ergebnis eines Routingversuchs wird als `RoutingResult` zurückgegeben. Dieses enthält den Namen des Algorithmus, den Erfolg des Routings, die Anzahl der benötigten Hops und die tatsächlich besuchten Kanten. Zusätzlich werden die Anzahl der berechneten Pfade und die Anzahl der vorbereiteten Routingstrukturen gespeichert.

Die folgenden Unterabschnitte erklären die Implementierung der drei NDP-basierten Routingverfahren. Abschnitt 4.2.1 beschreibt, wie die knotendisjunkten Pfade direkt für das Routing verwendet werden. Anschließend erklärt Abschnitt 4.2.2 die Erweiterung des längsten NDP zu einer One-Tree-Struktur. Abschnitt 4.2.3 beschreibt abschließend, wie aus allen berechneten NDPs mehrere Baumstrukturen erzeugt werden.

4.2.1 NDP Paths

NDP Paths verwendet die berechneten knotendisjunkten Pfade direkt für das Routing. Die Pfade werden durch die Funktion `get_disjoint_paths(...)` erzeugt. Für den Pfadtyp `ndp` ruft diese Funktion `nx.node_disjoint_paths(...)` von NetworkX auf. Listing 1 zeigt, wie die Implementierung zwischen EDPs und NDPs unterscheidet.

```
74 def get_disjoint_paths(graph: nx.Graph, source, destination, path_type:
       str) -> list[list]:
75     """Berechnet EDPs oder intern knotendisjunkte NDPs, kürzeste zuerst.""
       "
76     if path_type == "edp":
77         paths = list(nx.edge_disjoint_paths(graph, source, destination))
78     elif path_type == "ndp":
79         paths = list(nx.node_disjoint_paths(graph, source, destination))
80     else:
81         raise ValueError(f"Unknown path type: {path_type}")
82     return sorted((list(path) for path in paths), key=lambda path: (len(
       path), tuple(map(str, path))))
```

Listing 1: Berechnung und Sortierung der EDPs und NDPs

Die berechneten Pfade dürfen gemeinsame Start- und Zielknoten besitzen. Ihre inneren Knoten müssen jedoch verschieden sein. Der Ausfall eines inneren Knotens kann deshalb höchstens einen der berechneten NDPs unterbrechen.

Ein Beispiel der berechneten Pfade zeigt Abbildung 2, die EDPs und NDPs auf demselben Graphen gegenüberstellt.

NetworkX gibt die Pfade nicht zwingend in einer festen Reihenfolge zurück. Aus diesem Grund werden sie nach ihrer Länge sortiert. Bei Pfaden gleicher Länge wird zusätzlich die Folge ihrer Knoten verwendet. Diese zweite Sortierung sorgt dafür, dass die Reihenfolge bei gleicher Versuchsszene stabil bleibt.

Für NDP Paths erzeugt `build_structures(...)` aus jedem NDP eine eigene Routingstruktur. Der Zielknoten d wird dabei aus der gespeicherten Struktur entfernt. Der vorletzte Knoten des Pfads wird stattdessen als Ausgang zum Ziel gespeichert.

Die erzeugten Pfadstrukturen werden anschließend an die gemeinsame Routingroutine übergeben. Ihre Reihenfolge, der Wechsel zwischen mehreren Strukturen und die Behandlung ausgefallener Kanten werden ausführlich in Abschnitt 4.5 beschrieben.

4.2.2 NDP One Tree

NDP One Tree verwendet ebenfalls alle berechneten NDPs. Es wird jedoch nur der längste NDP zu einem Baum erweitert. Die übrigen NDPs bleiben unverändert und werden weiterhin als einzelne Ausweichpfade verwendet.

Dazu sortiert `build_structures(...)` zunächst alle NDPs nach ihrer Länge. Der letzte Eintrag dieser sortierten Liste ist der längste Pfad. Bei einem Gleichstand wird die stabile Knotenreihenfolge verwendet. Dadurch ist eindeutig festgelegt, welcher Pfad zum One Tree erweitert wird.

Aus dem ausgewählten NDP wird zuerst das Grundgerüst des Baums erzeugt. Dieses besteht aus dem gesamten Pfad mit Ausnahme des Zielknotens d. Die Kante zwischen dem vorletzten Pfadknoten und d wird ebenfalls nicht in den Baum eingefügt. Der vorletzte Knoten kann später als Ausgang zum Ziel verwendet werden.

Die Funktion `_grow_tree(...)` erweitert dieses Grundgerüst. Das Wachstum beginnt an den inneren Knoten des NDP. Die Knoten werden vom ersten inneren Knoten nach s bis zum letzten inneren Knoten vor d verarbeitet. Der Startknoten ist Teil des Baums, wird aber nicht als Ausgangspunkt für neue Äste verwendet.

Während des Wachstums werden benachbarte Knoten zum Baum hinzugefügt. Ein Knoten wird nicht erneut aufgenommen, wenn er bereits im Baum enthalten ist. Der Zielknoten wird ebenfalls übersprungen. Dadurch bleibt die erzeugte Struktur ein Baum.

Die übrigen NDPs müssen als eigenständige Ausweichpfade erhalten bleiben. Deshalb werden ihre Kanten und inneren Knoten vor dem Baumwachstum geschützt. Der One Tree darf diese Bereiche nicht übernehmen. Diese Einschränkung ist für die NDP-Variante besonders wichtig, da die inneren Knoten der übrigen NDPs nicht Teil des erweiterten Baums werden sollen.

Nach dem Wachstum können Äste vorhanden sein, über die das Ziel nicht erreicht werden kann. Die Funktion `truncate_tree(...)` entfernt diese Äste. Erhalten bleiben nur Wege, die vom Startknoten zu einem Knoten mit einer direkten Verbindung zu d

führen. Diese Knoten werden als Ausgänge des Baums gespeichert. Das Ergebnis dieser Konstruktion ist beispielhaft in Abbildung 3 dargestellt. Die rechte Teilabbildung zeigt den erweiterten und trunkierten NDP One Tree sowie die unveränderten übrigen NDPs. Listing 2 fasst die zentralen Schritte zusammen. Es zeigt die Auswahl des längsten Pfads, den Schutz der übrigen Pfade, das Baumwachstum und die anschließende Trunkierung.

```
    trees_by_path = {}
    if structure_type == "one_tree":
        tree = _base_tree(longest)
        # Wie in der TREE-Konstruktion bleiben die übrigen disjunkten
    Pfade
        # eigenständige Ausweichstrukturen. Ihre Kanten und inneren Knoten
        # dürfen deshalb nicht in den längsten, zum Baum erweiterten Pfad
        # aufgenommen werden.
        reserved_path_edges = {
            _edge_key(a, b)
            for path in ascending
            for a, b in zip(path[:-2], path[1:-1])
        }
        protected_path_nodes = {
            node
            for path in ascending[:-1]
            for node in path[1:-1]
            if node not in tree
        }
        _grow_tree(
            graph,
            tree,
            destination,
            longest[1:-1],
            reserved_path_edges,
            protected_path_nodes,
        )
        grown_graph = tree.copy() if retain_growth_stage else None
        tree, exits = truncate_tree(graph, tree, source, destination)
        trees_by_path[tuple(longest)] = RoutingStructure(
            tree,
            tuple(longest),
            exits,
            True,
            grown_graph,
        )
```

Listing 2: Erzeugung der One-Tree-Struktur aus dem längsten Pfad

Der erzeugte One Tree wird gemeinsam mit den übrigen unveränderten NDPs an die Routingroutine übergeben. Die Reihenfolge dieser Strukturen wird in Abschnitt 4.5.2 beschrieben. Die Bevorzugung des ursprünglichen NDP innerhalb des One Tree und das Backtracking in zusätzlichen Baumästen werden in Abschnitt 4.5.3 erläutert.

Algorithmus 1 Extend – NDP One Tree

Require: Graph G, Startknoten s, Zielknoten d, berechnete NDPs
1: $pathToExtend \leftarrow$ längster NDP als Knotenfolge
2: $reservedEdges \leftarrow$ Kanten aller NDPs ohne ihre letzte Kante nach d
3: $reservedNodes \leftarrow$ innere Knoten aller übrigen NDPs
4: $tree \leftarrow pathToExtend$ ohne d und ohne die letzte Kante nach d
5: **for all** $u \in inner(pathToExtend)$, von s nach d **do**
6: $nodes \leftarrow [u]$
7: $it \leftarrow 0$
8: **while** $it < length(nodes)$ **do**
9: $neighbors \leftarrow$ Nachbarknoten von $nodes[it]$
10: **for all** $v \in neighbors$ **do**
11: **if** $v \neq d$ und v nicht in $tree$ und v nicht in $reservedNodes$ und Kante $(nodes[it], v)$ nicht in $reservedEdges$ **then**
12: Füge v zu $nodes$ hinzu
13: Füge v und die Kante $(nodes[it], v)$ zu $tree$ hinzu
14: Füge $(nodes[it], v)$ zu $reservedEdges$ hinzu
15: **end if**
16: **end for**
17: $it \leftarrow it + 1$
18: **end while**
19: **end for**
20: Trunkiere $tree$ mit Algorithmus 3
21: **return** übrige NDPs und $tree$, aufsteigend nach Pfadlänge geordnet

4.2.3 NDP Multiple Trees

NDP Multiple Trees erzeugt aus jedem berechneten NDP einen eigenen Baum. Damit unterscheidet sich das Verfahren von NDP One Tree, bei dem nur der längste Pfad erweitert wird.

Vor dem Baumwachstum werden alle NDPs nach ihrer Länge sortiert. Für die Erweiterung werden sie anschließend in umgekehrter Reihenfolge verarbeitet. Der längste NDP wird somit zuerst erweitert. Danach folgen die kürzeren Pfade.

Für jeden NDP wird mit `_base_tree(...)` ein eigenes Baumgrundgerüst erzeugt. Wie bei NDP One Tree enthält dieses Grundgerüst den Zielknoten nicht. Die inneren Knoten des jeweiligen NDP dienen als Startpunkte für das Baumwachstum.

Die Kanten aller NDP-Grundgerüste werden vor der Erweiterung global reserviert. Wird während des Wachstums eine neue Kante zu einem Baum hinzugefügt, wird auch diese Kante reserviert. Ein später erzeugter Baum kann sie danach nicht erneut verwenden. Dadurch verwenden die erzeugten Bäume keine gemeinsamen Baumkanten.

Nach dem Wachstum wird jeder Baum einzeln trunkiert. Äste ohne Verbindung zu einem möglichen Ausgang nach d werden entfernt. Das Ergebnis ist eine Liste von Baumstrukturen. Jede dieser Strukturen ist einem der ursprünglichen NDPs zugeordnet. Die Umsetzung ist in Listing 3 dargestellt. Das Listing zeigt die globale Reservierung der Kanten, die Verarbeitung der Pfade vom längsten zum kürzesten und die abschließende Anordnung der erzeugten Strukturen.

```
    else:
        # Alle Ausgangspfade sind bereits Teil der Strukturen und werden deshalb
        # vor der Erweiterung global reserviert (Algorithmus 2).
        globally_used = {
            _edge_key(a, b)
            for path in ascending
            for a, b in zip(path[:-2], path[1:-1])
        }
        for path in reversed(ascending):
            tree = _base_tree(path)
            _grow_tree(
                graph,
                tree,
                destination,
                path[1:-1],
                globally_used,
            )
            grown_graph = tree.copy() if retain_growth_stage else None
            tree, exits = truncate_tree(graph, tree, source, destination)
            trees_by_path[tuple(path)] = RoutingStructure(
                tree,
                tuple(path),
                exits,
                True,
                grown_graph,
            )

    structures = []
    for path in ascending:
        key = tuple(path)
        if key in trees_by_path:
            structures.append(trees_by_path[key])
        else:
            structures.append(RoutingStructure(_base_tree(path), key, (path[-2],), False))
    return structures
```

Listing 3: Erzeugung einer Baumstruktur für jeden disjunkten Pfad

Die fertigen Baumstrukturen werden anschließend an die gemeinsame Routingroutine übergeben. Ihre Reihenfolge beim Routing wird in Abschnitt 4.5.2 beschrieben. Die Priori-

sierung der Baumäste anhand ihrer Entfernung zu einem Ausgang und das Backtracking behandelt Abschnitt 4.5.3.

Die Knotendisjunktheit gilt zunächst für die NDPs, aus denen die Bäume erzeugt werden. Bei der Erweiterung der Multiple Trees werden global die verwendeten Kanten, nicht jedoch alle verwendeten Knoten reserviert. Zwei gewachsene Bäume können deshalb zusätzliche Knoten gemeinsam verwenden, obwohl ihre NDP-Grundpfade intern knotendisjunkt sind. Diese Eigenschaft ist bei der Interpretation der NDP-Multiple-Trees-Ergebnisse zu berücksichtigen.

Abbildung 4 veranschaulicht die erzeugten Baumstrukturen. Die rechte Teilabbildung zeigt, dass trotz knotendisjunkter Basispfade gemeinsame innere Knoten zwischen den erweiterten Bäumen auftreten können.

Algorithmus 2 Extend – NDP Multiple Trees

Require: Graph G, Startknoten s, Zielknoten d, berechnete NDPs
1: $NDPs \leftarrow$ alle NDPs, absteigend nach Pfadlänge geordnet
2: $usedEdges \leftarrow$ Kanten aller NDPs ohne ihre letzte Kante nach d
3: **for all** $pathToExtend \in NDPs$ **do**
4: $tree \leftarrow pathToExtend$ ohne d und ohne die letzte Kante nach d
5: **for all** $u \in inner(pathToExtend)$, von s nach d **do**
6: $nodes \leftarrow [u]$
7: $it \leftarrow 0$
8: **while** $it < length(nodes)$ **do**
9: $neighbors \leftarrow$ Nachbarknoten von $nodes[it]$
10: **for all** $v \in neighbors$ **do**
11: **if** $v \neq d$ und v nicht in $tree$ und Kante $(nodes[it], v)$ nicht in $usedEdges$ **then**
12: Füge v zu $nodes$ hinzu
13: Füge v und die Kante $(nodes[it], v)$ zu $tree$ hinzu
14: Füge $(nodes[it], v)$ zu $usedEdges$ hinzu
15: **end if**
16: **end for**
17: $it \leftarrow it + 1$
18: **end while**
19: **end for**
20: Trunkiere $tree$ mit Algorithmus 3
21: Speichere $tree$ für $pathToExtend$
22: **end for**
23: **return** alle Bäume, aufsteigend nach der Länge ihrer NDPs geordnet

Die Trunkierung ist nicht auf die NDP-basierten Erweiterungen beschränkt. Dasselbe Verfahren wird für EDP One Tree, EDP Multiple Trees, NDP One Tree und NDP Multiple

Trees verwendet. Nach dem vollständigen Wachstum eines Baums werden alle Äste entfernt, die nicht von s zu einem erreichbaren Ausgangsknoten führen. Als Ausgang gilt ein Knoten des Baums, der im ursprünglichen Graphen direkt mit d verbunden ist. Bei Multiple Trees wird diese Trunkierung für jeden erzeugten Baum einzeln durchgeführt.

Algorithmus 3 Truncation der Baumstruktur

Require: Graph G, Baum $tree$, Startknoten s, Zielknoten d
1: $exits \leftarrow$ Knoten in $tree$, die in G mit d verbunden sind
2: $validExits \leftarrow \emptyset$
3: $keepEdges \leftarrow \emptyset$
4: **for all** $x \in exits$ **do**
5: **if** x in $tree$ von s erreichbar ist **then**
6: $pathToExit \leftarrow$ eindeutiger Baumweg von s nach x
7: Füge x zu $validExits$ hinzu
8: Füge alle Kanten aus $pathToExit$ zu $keepEdges$ hinzu
9: **end if**
10: **end for**
11: Entferne alle Kanten und Knoten, die nicht zu $keepEdges$ gehören
12: **if** $validExits \neq \emptyset$ **then**
13: Behalte s im Baum
14: **end if**
15: **return** $tree$ und $validExits$

4.3 Modellierung der Fehlermodelle

Die Fehlermodelle sind im Modul `src/failures.py` implementiert. Das Modul unterscheidet zwischen Link- und Knotenausfällen. Für beide Ausfallarten stehen ein zufälliges und ein geclustertes Modell zur Verfügung. Daraus entstehen die vier untersuchten Kombinationen Random Link, Clustered Link, Random Node und Clustered Node.

Die Funktion `select_failures(...)` bildet den gemeinsamen Einstiegspunkt. Sie erhält den Graphen, den Start- und Zielknoten, das ausgewählte Fehlermodell und einen Zufallszahlengenerator. Als Ergebnis gibt sie die ausgefallenen Kanten und Knoten zurück.

Die Routingalgorithmen verarbeiten Ausfälle intern als Kanten. Bei einem Knotenausfall werden deshalb alle Kanten, die mit dem ausgefallenen Knoten verbunden sind, ermittelt und in die Ausfallmenge aufgenommen. Der ausgefallene Knoten wird zusätzlich gespeichert, damit die Anzahl der Knotenausfälle in den CSV-Dateien erfasst werden kann. Der ursprüngliche Graph wird dabei nicht verändert.

Die unterschiedliche Wirkung eines Kanten- und eines Knotenausfalls ist in Abbildung 1 dargestellt. Beim Knotenausfall werden alle inzidenten Kanten als nicht verfügbar behandelt.

Die Auswahl der Ausfälle erfolgt einmal pro Versuchsszene. Danach erhalten alle sechs Routingalgorithmen denselben Graphen, dieselben Start- und Zielknoten und dieselbe Ausfallmenge. Dadurch können ihre Ergebnisse direkt miteinander verglichen werden.

Die folgenden Unterabschnitte erklären die Implementierung der vier Fehlermodelle. Abschnitt 4.3.1 behandelt zufällige Linkausfälle. Abschnitt 4.3.2 beschreibt geclusterte Linkausfälle. Die zufälligen und geclusterten Knotenausfälle werden anschließend in den Abschnitten 4.3.3 und 4.3.4 erklärt.

4.3.1 Random Link Failure

Random Link Failure wählt eine bestimmte Anzahl von Kanten aus dem gesamten Graphen aus. Die Verarbeitung besteht aus zwei Schritten. Zunächst berechnet `experiment.py` die gewünschte Anzahl der Linkausfälle. Anschließend wählt `failures.py` die entsprechenden Kanten aus.

Der Parameter `fail_factor` gibt die Anzahl der Ausfälle nicht direkt an. Er wird als Skalierungsfaktor für die lokale Konnektivität zwischen dem Start- und Zielknoten verwendet und wird als f_{factor} bezeichnet. Bei einem Linkausfall entspricht diese Konnektivität der Anzahl der berechneten EDPs. Die Anzahl der Linkausfälle wird wie folgt berechnet:

$$F = \text{round}\left(\max(|\text{EDPs}| - 1, 0) * f_{\text{factor}}\right).$$

Dabei bezeichnet $|\text{EDPs}|$ die Anzahl der berechneten kantendisjunkten Pfade. Durch die Verwendung von $\max(|\text{EDPs}| - 1, 0)$ kann die berechnete Anzahl nicht negativ werden.

Listing 4 zeigt die Berechnung in `experiment.py`. Für Linkausfälle wird `connectivity = len(edps)` gesetzt. Da das zufällige Fehlermodell eine feste Ausfallanzahl verwendet, liefert `uses_failure_count(...)` in diesem Fall den Wert `True`.

```
419             # Link- und Node-Ausfälle skalieren mit ihrer passenden
    lokalen
420             # Konnektivität; alle sechs Algorithmen erhalten dieselbe
    Auswahl.
421             connectivity = len(edps) if failure_type == "link" else len(
    ndps)
422             nr_failures = (
423                 round(max(connectivity - 1, 0) * fail_factor)
424                 if uses_failure_count(failure_model)
425                 else 0
```

Listing 4: Berechnung der gewünschten Anzahl zufälliger Ausfälle

Die berechnete Anzahl wird anschließend als `nr_failures` an `select_failures(...)` übergeben. Dort werden zunächst alle Kanten durch `sorted_edges(...)` stabil sortiert. Danach wählt `rng.sample(...)` die Ausfälle aus dieser Kantenliste aus.

Listing 5 zeigt diese Auswahl. Der Ausdruck `min(nr_failures, len(edges))` verhindert, dass mehr Kanten ausgewählt werden, als im Graphen vorhanden sind.

```
184     if failure_model == "random" and failure_type == "link":
185         edges = sorted_edges(graph.edges)
186         return rng.sample(edges, min(nr_failures, len(edges))), []
```

Listing 5: Auswahl der zufälligen Linkausfälle

Die tatsächlich ausgewählte Anzahl der Linkausfälle ergibt sich damit aus:

$$|F_E| = \min\left(F, |E|\right),$$

wobei $|E|$ die Anzahl aller Kanten des Graphen bezeichnet. Setzt man die Berechnung von F ein, ergibt sich die vollständige Formel:

$$|F_E| = \min\left(\text{round}\left(\max\left(|\text{EDPs}| - 1, 0\right) * f_{\text{factor}}\right), |E|\right).$$

Besitzt eine Versuchsszene beispielsweise vier EDPs und wird `fail_factor` $= 0{,}8$ verwendet, ergibt sich:

$$\begin{aligned} F &= \text{round}\left((4-1) \cdot 0{,}8\right) \\ &= \text{round}(2{,}4) \\ &= 2. \end{aligned}$$

In dieser Versuchsszene werden somit zwei zufällig ausgewählte Kanten als ausgefallen markiert. Nur wenn der Graph weniger als zwei Kanten enthalten würde, würde `min(...)` die Anzahl entsprechend reduzieren.

Bei Random Link Failure sind Kanten zum Start- oder Zielknoten nicht geschützt. Sie sind Teil der sortierten Kantenliste und können ebenfalls ausgewählt werden. Bei gleicher Versuchsszene und gleichem Seed führt die stabile Sortierung dazu, dass dieselben Kanten ausgewählt werden.

Die Berechnung der gewünschten Anzahl zufälliger Linkausfälle orientiert sich an der TREE-Referenzimplementierung. Die vorliegende Implementierung ergänzt mit

`max(...)` eine Absicherung gegen negative Ausfallzahlen und begrenzt die tatsächliche Auswahl mit `min(...)` auf die Anzahl der vorhandenen Kanten. Zusätzlich werden die Kanten vor der Seed-gesteuerten Zufallsauswahl stabil sortiert. Dadurch bleibt die Auswahl bei gleicher Versuchsszene und gleichem Seed reproduzierbar.

4.3.2 Clustered Link Failure

Clustered Link Failure erzeugt Linkausfälle in mehreren aufeinanderfolgenden Runden. Die Ausbreitung beginnt bei den Kanten, die mit dem Zielknoten d verbunden sind. Danach werden weitere Kanten aus der Umgebung der aktuellen Kandidaten betrachtet.

Das Modell verwendet die Parameter `failure_start` und `failure_drop`. Zur Vereinfachung der mathematischen Darstellung werden diese im Folgenden als

$$f_{\text{start}} := \texttt{failure_start} \qquad \text{und} \qquad f_{\text{drop}} := \texttt{failure_drop}$$

bezeichnet. f_{start} bestimmt den Auswahlanteil der ersten Runde. f_{drop} legt fest, um welchen Wert dieser Anteil nach jeder Runde reduziert wird.

Damit der Auswahlanteil einen gültigen Wert darstellt, muss

$$0 < f_{\text{start}} \leq 1 \qquad \text{und} \qquad 0 < f_{\text{drop}} \leq 1$$

gelten. Werte außerhalb dieses Bereichs werden vor der Erzeugung der Ausfälle abgewiesen.

```
if not 0.0 < failure_start <= 1.0:
    raise ValueError(
        "failure_start must be greater than 0 and less than or equal to 1"
    )
if not 0.0 < failure_drop <= 1.0:
    raise ValueError(
        "failure_drop must be greater than 0 and less than or equal to 1"
    )
```

Listing 6: Prüfung der Parameter des Clustered-Failure-Modells

Sei $E_{C_i} \subseteq E$ die Menge der Kantenkandidaten in Runde i. Ihre Anzahl wird mit $|E_{C_i}|$ bezeichnet. Die erste Runde erhält den Index $i = 0$. Die Anzahl der in Runde i auszuwählenden Kanten ergibt sich direkt aus:

$$F_i = \lfloor (f_{\text{start}} - i * f_{\text{drop}}) * |E_{C_i}| \rfloor , \qquad i \geq 0.$$

Dabei bezeichnet F_i die Anzahl der in Runde i ausgewählten Kanten. Sie ist von der gesamten Ausfallmenge F_E zu unterscheiden.

Eine Runde wird nur ausgeführt, solange

$$f_{\text{start}} - i * f_{\text{drop}} > 0$$

gilt und Kantenkandidaten vorhanden sind. Die Formel entspricht der Implementierung, in der die aktuelle Wahrscheinlichkeit nach jeder Runde um `failure_drop` reduziert wird.

Die erste Kandidatenmenge E_{C_0} wird aus `graph.edges(destination)` gebildet. Sie enthält somit Kanten, die mit d verbunden sind. Die Bedingung `if source not in edge` entfernt bereits in dieser Runde alle Kanten zum Startknoten s.

Beim Aufbau der folgenden Kandidatenmengen gilt eine andere Bedingung. Kanten zu s und Kanten zu d werden mit `continue` übersprungen. Dadurch können Kanten zu d nur in der ersten Runde ausgewählt werden. Kanten zu s sind in allen Runden geschützt.

Listing 7 zeigt beide Bedingungen. Die Bildung von `incidents` belegt, dass Zielkanten in der ersten Runde zulässig sind. Die spätere Bedingung mit `source` und `destination` zeigt ihren Ausschluss beim Aufbau der nächsten Kandidatenmenge.

```
114     incidents = [
115         edge
116         for edge in sorted_edge_sequence(graph.edges(destination))
117         if source not in edge
118     ]
119     probability = failure_start
120 
121     while probability > 0.0 and incidents:
122         count = math.floor(probability * len(incidents))
123         for edge in rng.sample(incidents, count):
124             key = edge_key(edge)
125             if key not in seen_failed:
126                 failed.append(edge)
127                 seen_failed.add(key)
128 
129         next_incidents = []
130         seen_next = set()
131         for _, outer_node in incidents:
132             for edge in sorted_edge_sequence(graph.edges(outer_node)):
133                 if source in edge or destination in edge:
134                     continue
135                 key = edge_key(edge)
136                 if key not in seen_next:
137                     next_incidents.append(edge)
138                     seen_next.add(key)
139         incidents = sorted_edge_sequence(next_incidents)
140         probability -= failure_drop
```

Listing 7: Kandidatenauswahl und Endpunktschutz bei Clustered Link Failure

Die Kandidaten der nächsten Runde werden aus allen Kanten der aktuellen Kandidatenmenge E_{C_i} aufgebaut. Es ist dabei nicht entscheidend, welche dieser Kanten tatsächlich als ausgefallen ausgewählt wurden. Die Ausbreitung folgt somit der gesamten Kandidatenschicht und nicht nur den ausgefallenen Kanten.

Die Mengen `seen_failed` und `seen_next` verhindern, dass dieselbe ungerichtete Kante mehrfach gespeichert wird. `edge_key(...)` normalisiert eine Kante dafür als `frozenset`. Außerdem werden die Kandidaten vor der Seed-gesteuerten Auswahl stabil sortiert.

Ein Beispiel zeigt den Ablauf. Angenommen, die erste Kandidatenmenge E_{C_0} enthält nach dem Schutz von s fünf Kanten. Damit gilt

$$|E_{C_0}| = 5.$$

Es werden

$$f_{\text{start}} = 0{,}6 \qquad \text{und} \qquad f_{\text{drop}} = 0{,}3$$

verwendet. Für die erste Runde mit $i = 0$ ergibt sich:

$$\begin{aligned} F_0 &= \lfloor (0{,}6 - 0 \cdot 0{,}3) \cdot 5 \rfloor \\ &= \lfloor 3{,}0 \rfloor \\ &= 3. \end{aligned}$$

Damit werden drei der fünf Kandidaten zufällig als ausgefallen ausgewählt. Für die Erzeugung von E_{C_1} werden trotzdem alle fünf Kanten aus E_{C_0} betrachtet. Dazu werden ihre äußeren Endknoten bestimmt und deren inzidente Kanten untersucht.

Kanten zum Zielknoten d sind nur in der ersten Runde Teil der Kandidatenmenge. Beim Aufbau aller weiteren Kandidatenmengen werden sie ausgeschlossen. Bereits in der ersten Runde ausgewählte Kanten bleiben ausgefallen, während die übrigen inzidenten Kanten von d in späteren Runden geschützt sind.

Angenommen, nach der Erweiterung, Filterung und Entfernung von Duplikaten enthält E_{C_1} acht Kanten. Damit gilt

$$|E_{C_1}| = 8.$$

Für die zweite Runde mit $i = 1$ ergibt sich:

$$\begin{aligned} F_1 &= \lfloor (0{,}6 - 1 \cdot 0{,}3) \cdot 8 \rfloor \\ &= \lfloor 0{,}3 \cdot 8 \rfloor \\ &= \lfloor 2{,}4 \rfloor \\ &= 2. \end{aligned}$$

In der zweiten Runde werden zwei weitere Kanten ausgewählt. Für eine mögliche dritte Runde mit $i = 2$ wäre der Auswahlanteil

$$0{,}6 - 2 \cdot 0{,}3 = 0.$$

Die dritte Runde wird deshalb nicht mehr ausgeführt. In diesem Beispiel werden insgesamt fünf Kanten als ausgefallen in F wie folgend gespeichert:

$$\begin{aligned} F &= F_0 + F_1 \\ &= 3 + 2 \\ &= 5. \end{aligned}$$

Die Auswahl wird nur durchgeführt, solange der aktuelle Anteil positiv ist. Wird beispielsweise $f_{\text{start}} = 0{,}8$ und $f_{\text{drop}} = 0{,}3$ gewählt, werden nacheinander die Anteile $0{,}8$, $0{,}5$ und $0{,}2$ verwendet. Nach der dritten Runde ergibt die Reduktion den Wert $-0{,}1$. Dieser Wert wird nicht mehr für eine Auswahl verwendet, da die Schleifenbedingung nur positive Anteile zulässt:

```
while probability > 0.0 and incidents:
    # ...
    probability -= failure_drop
```

Listing 8: Beendigung der Clusterbildung bei nicht positivem Auswahlanteil

Die Berechnung der Anzahl geclusterter Linkausfälle pro Runde wurde aus der TREE-Referenzimplementierung übernommen.[1] Beide Implementierungen berechnen die Ausfallzahl einer Runde nach dem gleichen Prinzip.

Die vorliegende Implementierung verändert jedoch den Aufbau der Kandidatenlisten. Kanten zum Startknoten werden in allen Runden geschützt, während Kanten zum Zielknoten nur in der ersten Runde zugelassen werden. Zusätzlich werden ungerichtete Kanten normalisiert und die Kandidaten vor der Seed-gesteuerten Auswahl stabil sortiert. Die Berechnungsformel ist damit unverändert, die Menge der möglichen Ausfälle kann sich gegenüber der Referenzimplementierung jedoch unterscheiden.

[1] Vgl. `external/TREE/src/kResilienceTrees.py`, Z. 663–675.

4.3.3 Random Node Failure

Random Node Failure verwendet dasselbe Berechnungsschema wie Random Link Failure. Anstelle von Kanten werden jedoch Knoten zufällig ausgewählt. Außerdem wird die Anzahl der Ausfälle nicht aus den EDPs, sondern aus den berechneten NDPs bestimmt.

Mit $|\mathrm{NDPs}|$ wird die Anzahl der knotendisjunkten Pfade zwischen s und d bezeichnet. Die Knotenkandidaten bestehen aus allen Knoten des Graphen außer dem Startknoten s und dem Zielknoten d. Ihre Anzahl wird im Folgenden mit

$$|V| := \text{Anzahl der zulässigen Knotenkandidaten}$$

bezeichnet. Die tatsächliche Anzahl der ausgefallenen Knoten ergibt sich aus

$$|F_V| = \min\left(\mathrm{round}\left(\max\left(|\mathrm{NDPs}| - 1, 0\right) * f_{\mathrm{factor}}\right), |V|\right).$$

Dabei bezeichnet F_V die Menge der ausgefallenen Knoten. Die Begrenzung auf $|V|$ verhindert, dass mehr Knoten ausgewählt werden, als zulässige Kandidaten vorhanden sind.

Besitzt eine Versuchsszene beispielsweise vier NDPs und gilt $f_{\mathrm{factor}} = 0{,}8$, dann werden

$$\mathrm{round}\left((4-1) \cdot 0{,}8\right) = 2$$

Knoten ausgewählt, sofern mindestens zwei Kandidaten vorhanden sind.

Gegenüber Random Link Failure unterscheiden sich die Berechnung und die Auswahl durch die folgenden Schritte:

```
connectivity = len(ndps)

candidates = unique_sorted_nodes(
    node for node in graph
    if node not in (source, destination)
)
nodes = rng.sample(
    candidates,
    min(nr_failures, len(candidates))
)

return (
    node_failures_to_edges(graph, nodes),
    unique_sorted_nodes(nodes),
)
```

Listing 9: Unterschiede zwischen Random Node und Random Link Failure

Die Kandidaten werden vor der Zufallsauswahl stabil sortiert. Dadurch wird bei gleichem Seed dieselbe Knotenauswahl erzeugt. Der Start- und der Zielknoten sind nicht in der Kandidatenliste enthalten und können daher nicht ausfallen.

Nach der Auswahl bestimmt `node_failures_to_edges(...)` alle Kanten, die mit mindestens einem ausgefallenen Knoten verbunden sind. Ein Knotenausfall wird somit als gleichzeitiger Ausfall aller inzidenten Kanten des Knotens modelliert.

4.3.4 Clustered Node Failure

Clustered Node Failure überträgt die schichtweise Ausbreitung von Clustered Link Failure auf Knoten. Die erste Kandidatenmenge besteht aus den Nachbarknoten des Zielknotens d. Der Start- und der Zielknoten werden davon ausgeschlossen.

Mit

$$|V_{C_i}| := \text{Anzahl der Knotenkandidaten in Runde } i$$

wird die Größe der jeweiligen Kandidatenmenge bezeichnet. Die Anzahl der in Runde i ausgewählten Knoten ergibt sich aus

$$F_i = \lfloor (f_{\text{start}} - i * f_{\text{drop}}) * |V_{C_i}| \rfloor , \qquad i \geq 0.$$

Die Zählung beginnt bei $i = 0$. In der ersten Runde wird daher unmittelbar f_{start} verwendet. Für $f_{\text{start}} = 0{,}6$ und fünf Knotenkandidaten ergibt sich beispielsweise

$$F_0 = \lfloor 0{,}6 \cdot 5 \rfloor = 3.$$

Enthält die zweite Runde acht Kandidaten und gilt $f_{\text{drop}} = 0{,}3$, werden

$$F_1 = \lfloor (0{,}6 - 0{,}3) \cdot 8 \rfloor = 2$$

weitere Knoten ausgewählt. Danach ist der Auswahlanteil null und die Ausbreitung wird beendet.

Die Berechnung des Auswahlanteils, die Verwendung von `math.floor(...)` und die Abbruchbedingung entsprechen Clustered Link Failure. Die Unterschiede betreffen daher hauptsächlich den Aufbau der Kandidatenmenge und die Umwandlung der Knotenausfälle in Kantenausfälle:

```
candidates = unique_sorted_nodes(
    node for node in graph.neighbors(destination)
    if node not in (source, destination)
```

```
    )

    # ...

    for node in candidates:
        for neighbor in sorted(
            graph.neighbors(node),
            key=node_sort_key,
        ):
            if (
                neighbor not in (source, destination)
                and neighbor not in failed
            ):
                next_candidates.append(neighbor)

    candidates = unique_sorted_nodes(next_candidates)

    # ...

    nodes = clustered_node_failures(
        graph,
        source,
        destination,
        failure_start,
        failure_drop,
        rng,
    )
    return node_failures_to_edges(graph, nodes), nodes
```

Listing 10: Unterschiede zwischen Clustered Node und Clustered Link Failure

Die Kandidaten der nächsten Runde werden aus den Nachbarn aller aktuellen Kandidaten außer s und d gebildet, auch wenn die Endknoten direkt mit den Kandidaten benachbart sind. Die Ausbreitung ist daher nicht ausschließlich auf die tatsächlich ausgefallenen Knoten beschränkt. Die Ausfälle befinden sich schichtweise im Umfeld von d, müssen aber nicht zwingend einen zusammenhängenden Teilgraphen bilden.

Start- und Zielknoten bleiben in allen Runden wie vorher beschrieben geschützt. Bereits ausgefallene Knoten werden nicht erneut als Kandidaten aufgenommen. Die Funktion `unique_sorted_nodes(...)` entfernt außerdem Duplikate und stellt eine stabile Reihenfolge her.

Für f_{start} und f_{drop} gelten dieselben zulässigen Bereiche und Abbruchbedingungen wie bei Clustered Link Failure. Nach der Auswahl werden alle inzidenten Kanten der ausgefallenen Knoten bestimmt und an die Routingalgorithmen übergeben.

Clustered Node Failure ist die in dieser Arbeit vorgenommene Übertragung des geclusterten Linkausfallmodells auf Knotenausfälle.

4.4 Seed-basierte Steuerung für reproduzierbare Randomisierung der Versuchsszenen

Die Durchführung der Experimente enthält mehrere zufallsabhängige Schritte. Dazu gehören die Erzeugung synthetischer Graphen, die Auswahl von Start- und Zielknoten sowie die Bestimmung der ausgefallenen Kanten beziehungsweise Knoten. Damit die Ergebnisse trotz dieser Zufallseinflüsse reproduzierbar bleiben, werden sämtliche Zufallsentscheidungen durch einen in der Konfiguration festgelegten Seed gesteuert. Der verwendete Seed wird zusammen mit den übrigen Versuchsparametern im Manifest des Pipeline-Laufs gespeichert. Listing 11 zeigt die dafür wesentlichen Anweisungen aus `write_manifest(...)` in gekürzter Form.

```
manifest = {
    "python": sys.version,
    "platform": platform.platform(),
    "config_path": str(Path(config_path).resolve()),
    "runs": int(config["runs"]),
    "seed": config.get("seed"),
    "resume": bool(config.get("resume", False)),
    # ...
}

with manifest_path.open("w", encoding="utf-8") as manifest_file:
    json.dump(manifest, manifest_file, indent=2)
```

Listing 11: Speicherung reproduzierbarkeitsrelevanter Metadaten (Ausschnitt aus `pipeline.py`)

Abbildung 5 fasst zusammen, wie der konfigurierte Seed von der Pipeline bis zur konkreten Versuchsszene weitergegeben wird.

Zu Beginn einer Versuchsreihe erzeugt die Implementierung mit `random.Random(seed)` einen zentralen Pseudozufallszahlengenerator. Für jede Wiederholung wird daraus ein eigener `graph_seed` abgeleitet. Dieser wird bei synthetischen Datensätzen an den jeweiligen NetworkX-Graphgenerator übergeben. Wie Listing 12 zeigt, wird bei einem Real-World-Datensatz dagegen die eingelesene GraphML-Topologie kopiert. In diesem Fall beeinflusst der Seed nicht die Topologie selbst, sondern die nachfolgende Auswahl der Start- und Zielknoten sowie der Ausfälle.

```
388     master_rng = random.Random(seed)
389     loaded_graph = nx.Graph(nx.read_graphml(graphml_path)) if graphml_path
        else None
390     effective_graph_type = "graphml" if graphml_path else graph_type
391     effective_param = graph_name or Path(graphml_path).stem if
       graphml_path else graph_param
392
393     for rep in range(1, runs + 1):
```

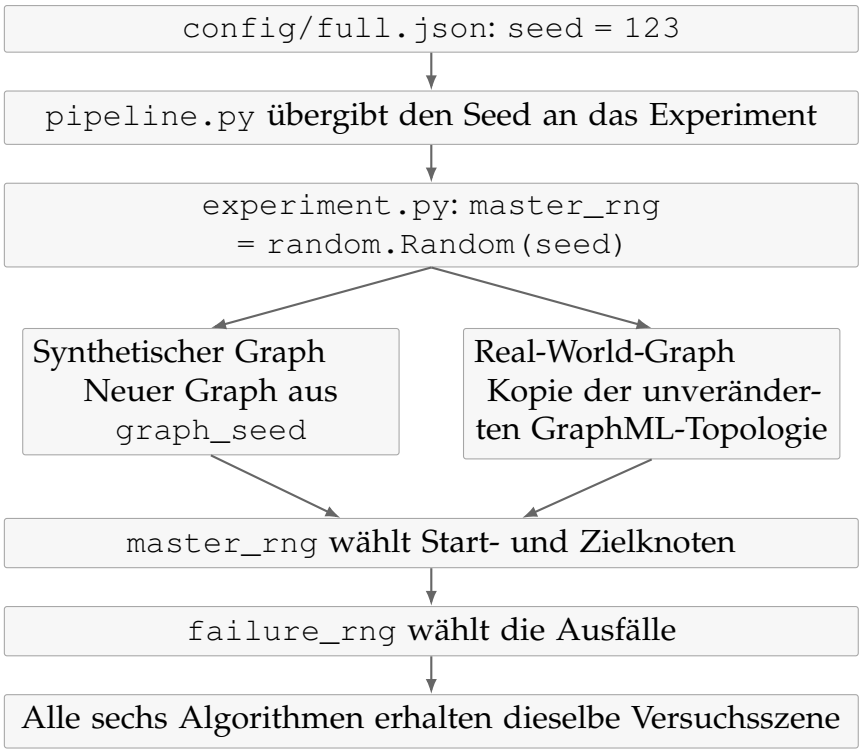


Abbildung 5: Seed-gesteuerte Erzeugung einer reproduzierbaren Versuchsszene

```
394         for _ in range(1000):
395             graph_seed = master_rng.randrange(2**32)
396             graph = loaded_graph.copy() if loaded_graph is not None else
    create_graph(
397                 graph_type, nodes, graph_param, graph_seed
398             )
399             try:
400                 source, destination = choose_endpoints(graph, master_rng)
```

Listing 12: Seed-gesteuerte Erzeugung einer Versuchsszene

Die Auswahl von Start- und Zielknoten erfolgt ebenfalls pseudozufällig. Zunächst werden die Zusammenhangskomponenten und ihre Knoten stabil sortiert. Anschließend werden alle Knotenpaare bestimmt, die derselben Komponente angehören und nicht direkt durch eine Kante verbunden sind. Aus dieser stabil geordneten Kandidatenliste wählt der vom Seed initialisierte Zufallsgenerator ein Paar aus. Diese Schritte sind in Listing 13 dargestellt. Die Sortierung stellt sicher, dass derselbe Zufallsindex bei gleichem Graphen wieder demselben Start-Ziel-Paar zugeordnet wird.

```
225 def choose_endpoints(graph, rng):
226     """Wählt ein verbundenes, nicht direkt benachbartes s-d-Paar."""
```

```
    # Nur Paare innerhalb derselben Zusammenhangskomponente betrachten. Das
    # vermeidet bei größeren Real-World-Graphen eine Pfadsuche für jedes der
    # O(n^2) möglichen Knotenpaare.
    components = [
        tuple(sorted(component, key=node_sort_key))
        for component in nx.connected_components(graph)
    ]
    components = sorted(
        components,
        key=lambda component: tuple(node_sort_key(node) for node in component),
    )
    candidates = [
        pair
        for component in components
        for pair in combinations(component, 2)
        if not graph.has_edge(*pair)
    ]
    if not candidates:
        raise nx.NetworkXNoPath("No connected non-adjacent endpoint pair")
    return rng.choice(candidates)
```

Listing 13: Stabile und reproduzierbare Auswahl von Start- und Zielknoten

Für die Ausfälle wird ein separater Pseudozufallszahlengenerator erzeugt. Sein Schlüssel setzt sich aus dem globalen Seed, der Wiederholungsnummer, dem Ausfalltyp, dem Fehlermodell sowie den jeweils verwendeten Parametern `failFactor` und `failureStart` zusammen. Dadurch ist die Ausfallauswahl unabhängig von der Ausführungsreihenfolge der sechs Routingalgorithmen. Listing 14 zeigt sowohl die Erzeugung dieses Generators als auch seine Übergabe an `select_failures(...)`.

```
            failure_rng = random.Random(
                f"{seed}:{rep}:{failure_type}:{failure_model}:"
                f"{seed_parameter_token(fail_factor)}:"
                f"{seed_parameter_token(failure_start)}"
            )
            failed_edges, failed_nodes = select_failures(
                graph,
                source,
                destination,
                failure_type,
                failure_model,
                nr_failures,
                failure_start if uses_cluster_parameters(failure_model) else 0.0,
                failure_drop,
                failure_rng,
```

```
442                 )
```

Listing 14: Konfigurationsabhängiger Zufallsgenerator für die Ausfallauswahl

Numerische Failure-Parameter werden zuvor durch `seed_parameter_token(...)` kanonisch dargestellt. Dadurch ergeben beispielsweise `1` und `1.0` denselben Zufallsschlüssel. Die dafür verwendete Normalisierung ist in Listing 15 dargestellt.

```
254 def seed_parameter_token(value):
255     """Normalisiert numerische Parameter für reproduzierbare RNG-Schlüssel
    .."""
256     if value == "":
257         return ""
258     try:
259         decimal = Decimal(str(value))
260     except (InvalidOperation, ValueError):
261         return str(value)
262     if decimal == decimal.to_integral_value():
263         return str(int(decimal))
264     return format(decimal.normalize(), "f")
```

Listing 15: Kanonische Darstellung numerischer Failure-Parameter

Vor der eigentlichen Zufallsauswahl werden außerdem die Kanten- und Knotenkandidaten stabil sortiert. Random Link Failures verwenden eine sortierte Kantenliste; Random Node Failures verwenden eine sortierte Liste der zulässigen Knoten. Listing 16 stellt nur die dafür wesentlichen, im Quellcode nicht unmittelbar aufeinanderfolgenden Anweisungen dar.

```
def sorted_edges(edges):
    return [
        stable_edge_tuple(edge)
        for edge in sorted(edges, key=edge_sort_key)
    ]

def unique_sorted_nodes(nodes):
    return sorted(dict.fromkeys(nodes), key=node_sort_key)

# ...

if failure_model == "random" and failure_type == "link":
    edges = sorted_edges(graph.edges)
    return rng.sample(edges, min(nr_failures, len(edges))), []

if failure_model == "random" and failure_type == "node":
    candidates = unique_sorted_nodes(
        node for node in graph if node not in (source, destination)
    )
    nodes = rng.sample(candidates, min(nr_failures, len(candidates)))
```

Listing 16: Stabile Kandidatenlisten für zufällige Ausfälle (Ausschnitt aus `failures.py`)

Auf diese Weise erhalten alle sechs Routingalgorithmen innerhalb einer Wiederholung denselben Graphen, dasselbe Start-Ziel-Paar und dieselbe Ausfallmenge. Dieser Ablauf ist in Abbildung 5 dargestellt; die konkrete Erzeugung des ausfallabhängigen Zufallsgenerators zeigt Listing 14. Ergebnisunterschiede lassen sich damit auf das jeweilige Routingverfahren und nicht auf unterschiedliche Zufallsszenen zurückführen.

Auch beim Fortsetzen einer unterbrochenen Versuchsreihe bleibt die Zufallsfolge erhalten. Bereits vollständig gespeicherte Wiederholungen werden erkannt. Die für den Zustand von `master_rng` maßgeblichen Zufallsoperationen werden für diese Wiederholungen erneut ausgeführt, die zugehörigen Ergebniszeilen jedoch nicht erneut geschrieben. Dadurch erreicht der zentrale Zufallszahlengenerator denselben Zustand wie bei einer durchgängigen Ausführung. Die entscheidenden Schritte dieser Resume-Verarbeitung fasst Listing 17 zusammen.

```
if resume:
    completed_reps = min(
        completed_rep_prefix(csv_path, algorithms)
        for csv_path in paths.values()
    )
    # ...
else:
    completed_reps = 0

master_rng = random.Random(seed)

for rep in range(1, runs + 1):
    # Diese Zufallsoperationen werden auch für gespeicherte
    # Wiederholungen erneut ausgeführt.
    graph_seed = master_rng.randrange(2**32)
    # ...
    source, destination = choose_endpoints(graph, master_rng)

    if rep <= completed_reps:
        continue
```

Listing 17: Rekonstruktion der Zufallsfolge beim Fortsetzen (Ausschnitt aus `experiment.py`)

Ein gleicher Seed bedeutet folglich nicht, dass in allen Wiederholungen derselbe Graph und dieselben Start- und Zielknoten verwendet werden. Er legt stattdessen eine deterministische Folge unterschiedlicher Versuchsszenen fest. Eine Änderung des Seeds führt normalerweise zu einer anderen, wiederum reproduzierbaren Folge von Graphen,

Start- und Zielknoten sowie Ausfällen. Für eine vollständige Wiederholbarkeit müssen zusätzlich dieselbe Konfiguration, dieselbe Codeversion sowie kompatible Python- und NetworkX-Versionen verwendet werden. Abbildung 5 verdeutlicht somit die Seed-Kette, während die Listings 12 bis 17 ihre wesentlichen Implementierungsschritte belegen.

4.5 Routing

Nach der Berechnung der disjunkten Pfade und dem Aufbau der Routingstrukturen folgt das eigentliche Routing. Dabei wird geprüft, ob ein Algorithmus den Zielknoten d trotz der ausgewählten Ausfälle erreicht. Zusätzlich werden die tatsächlich durchlaufenen Kanten als Hops gezählt.

Die sechs Algorithmen verwenden denselben Routingkern. Sie unterscheiden sich durch den verwendeten Pfadtyp und die daraus aufgebaute Struktur. Die EDP-Verfahren verwenden kantendisjunkte Pfade, während die NDP-Verfahren knotendisjunkte Pfade verwenden. Für beide Pfadtypen stehen die Varianten Paths, One Tree und Multiple Trees zur Verfügung.

Innerhalb einer Versuchsszene erhalten alle Algorithmen denselben Graphen, dieselben Start- und Zielknoten sowie dieselbe Ausfallmenge. Der Routingkern verarbeitet die Ausfälle einheitlich als ausgefallene Kanten. Bei einem Knotenausfall bestimmt die Funktion `node_failures_to_edges(...)` deshalb zuvor alle inzidenten Kanten des ausgefallenen Knotens. Ein Knotenausfall entspricht beim Routing somit dem gleichzeitigen Ausfall aller Kanten dieses Knotens.

Zunächst gibt Abschnitt 4.5.1 einen Überblick über die am Routing beteiligten Funktionen. Anschließend beschreibt Abschnitt 4.5.2 die Reihenfolge, in der die Pfad- und Baumstrukturen verwendet werden. Das Routing innerhalb einer einzelnen Struktur wird in Abschnitt 4.5.3 erläutert. Darauf aufbauend behandelt Abschnitt 4.5.4 den Wechsel zwischen mehreren Routingstrukturen und die Addition ihrer Hop-Anzahlen. Abschließend vergleicht Abschnitt 4.5.5 die verwendete Routingroutine mit der TREE-Referenzimplementierung.

4.5.1 Beteiligte Funktionen

Die Routingausführung verteilt sich auf mehrere Funktionen:

- `run_algorithms(...)` führt alle ausgewählten Algorithmen auf derselben Versuchsszene aus. EDPs und NDPs werden dabei getrennt zwischengespeichert. Dadurch verwenden alle Verfahren desselben Pfadtyps dieselben Ausgangspfade.
- `get_disjoint_paths(...)` berechnet die EDPs oder NDPs und sortiert sie aufsteigend nach ihrer Länge. Bei gleicher Länge wird eine stabile Sortierung anhand der Knotendarstellung verwendet.

- `build_structures(...)` erzeugt aus den berechneten Pfaden reine Pfadstrukturen, One Tree oder Multiple Trees.
- `route_algorithm(...)` führt einen Algorithmus aus, prüft dessen Routingstrukturen nacheinander und addiert die dabei entstandenen Hops.
- `route_structure(...)` führt die geordnete Tiefensuche innerhalb einer einzelnen Pfad- oder Baumstruktur aus.
- `_rooted_children(...)` wurzelt die aktuelle Struktur am Startknoten s und bestimmt die Eltern- und Kindbeziehungen.
- `_distance_to_exit(...)` berechnet die minimale Baumdistanz von einem Teilbaum zu einem möglichen Ausgang nach d.
- `_edge_key(...)` normalisiert ungerichtete Kanten. Dadurch wird eine ausgefallene Kante unabhängig von ihrer gespeicherten Richtung erkannt.

Das Ergebnis eines Algorithmus wird als `RoutingResult` zurückgegeben. Diese Datenklasse enthält den Erfolgsstatus, die Hop-Anzahl, die chronologische Kantenfolge sowie die Anzahl der vorbereiteten Pfade und Routingstrukturen.

4.5.2 Reihenfolge der Routingstrukturen

Die disjunkten Pfade werden zunächst aufsteigend nach ihrer Länge sortiert. Bei den reinen Pfadverfahren wird jeder Pfad als eigene Routingstruktur verwendet. Das Routing beginnt mit dem kürzesten Pfad. Kann dieser das Ziel nicht erreichen, wird der nächste Pfad geprüft.

Beim One-Tree-Verfahren bleiben die kürzeren Pfade beziehungsweise übrige Pfade eigenständige Ausweichstrukturen. Nur der längste Pfad wird zu einem Baum erweitert. Das Routing prüft deshalb zunächst die kürzeren Pfade und zuletzt den Baum des längsten Pfads. Innerhalb dieses Baums wird der ursprüngliche Pfad bevorzugt. Zusätzliche Baumäste werden verwendet, wenn dieser Pfad aufgrund eines Ausfalls nicht zum Ziel führt.

Bei Multiple Trees bildet jeder disjunkte Pfad das Grundgerüst eines eigenen Baums. Die Bäume werden während der Konstruktion vom längsten zum kürzesten Ausgangspfad erweitert. Dabei werden die im Baum gespeicherten Kanten aller Ausgangspfade sowie die während des Wachstums hinzugefügten Kanten global reserviert. Die abschließenden Kanten zum Zielknoten d gehören nicht zu den gespeicherten Bäumen und sind daher nicht Teil dieser Reservierung.

Jeder Baum wird unmittelbar nach seinem Wachstum trunkiert und gespeichert, bevor der nächste Baum konstruiert wird. Beim Trunkieren werden alle Äste entfernt, die keinen Ausgang zum Ziel besitzen. Während des Wachstums reservierte Kanten bleiben auch dann global reserviert, wenn sie anschließend aus dem Baum entfernt werden. Sie können deshalb nicht beim Wachstum eines späteren Baums verwendet werden.

Eine globale Reservierung von Knoten findet dagegen nicht statt. Bei EDP Multiple Trees dürfen unterschiedliche Bäume Knoten gemeinsam verwenden, solange ihre gespeicherten Baumkanten verschieden sind. Bei NDP Multiple Trees sind die ursprünglichen Ausgangspfade intern knotendisjunkt. Die während der Erweiterung hinzugefügten Äste können jedoch Knoten enthalten, die auch in einem anderen Baum vorkommen. Die vollständigen erweiterten Bäume sind daher nicht zwingend global knotendisjunkt.

Nach der Konstruktion der Multiple Trees existieren keine zusätzlichen reinen Pfadstrukturen. Beim Routing werden ausschließlich die erzeugten Bäume getestet. Zuerst wird der Baum des kürzesten Ausgangspfads und zuletzt der Baum des längsten Ausgangspfads verwendet.

4.5.3 Routing innerhalb einer Struktur

Eine vorbereitete Pfad- oder Baumstruktur wird durch die Datenklasse `RoutingStructure` beschrieben. Das Attribut `graph` enthält den Pfad beziehungsweise den trunkierten Baum. Der Zielknoten d ist nicht Bestandteil dieses gespeicherten Graphen.

Das Attribut `exits` enthält stattdessen alle Strukturknoten, die im ursprünglichen Graphen direkt mit d verbunden sind. Erreicht das Routing einen solchen Ausgangsknoten und ist dessen Kante zu d funktionsfähig, wird diese Kante durchlaufen und das Routing erfolgreich beendet.

Das Attribut `original_path` speichert den EDP beziehungsweise NDP, aus dem die Struktur entstanden ist. Beim One Tree wird diese Information verwendet, um den ursprünglichen Pfad innerhalb des Baums zu bevorzugen. `is_tree` kennzeichnet, ob es sich um einen gewachsenen Baum oder einen unveränderten Pfad handelt. Für die Erstellung von Abbildungen kann `grown_graph` zusätzlich den Baumzustand vor dem Trunkieren enthalten.

Die jeweilige Struktur wird am Startknoten s gewurzelt und anschließend durch eine geordnete Tiefensuche durchlaufen. Die Reihenfolge der Kindknoten wird durch drei Kriterien bestimmt:

1. Beim One Tree wird der nächste Knoten des ursprünglichen Pfads bevorzugt.
2. Danach wird der Teilbaum mit der kleinsten Entfernung zu einem möglichen Ausgang nach d gewählt.
3. Bei gleicher Entfernung entscheidet die stabile Sortierung nach der Knotendarstellung.

Die entsprechende Sortierung ist in Listing 18 dargestellt.

```
ordered = sorted(
    children.get(node, []),
    key=lambda child: (
```

```
            0 if (
                prefer_original_path
                and original_successor.get(node) == child
            ) else 1,
            _distance_to_exit(
                children, exits, child, distance_cache
            ),
            str(child),
        ),
    )
```

Listing 18: Bestimmung der Astreihenfolge beim Routing

Ausgefallene Kanten werden übersprungen. Führt ein besuchter Ast nicht zum Ziel, kehrt das Routing über die zuvor durchlaufene Kante zurück. Dieses Backtracking ist in der rekursiven Funktion VISIT in Algorithmus 4 dargestellt.

Jede in `route` eingetragene Kante entspricht einem Hop. Rückwege werden ebenfalls eingetragen und erhöhen daher die Hop-Anzahl. Eine ausgefallene Kante wird dagegen nicht durchlaufen und nicht als Hop gezählt.

4.5.4 Wechsel zwischen mehreren Strukturen

Kann eine Struktur das Ziel nicht erreichen, prüft `route_algorithm(...)` die nächste Struktur. Die Hops der bereits getesteten Strukturen werden addiert.

Der vollständige Ablauf ist in Algorithmus 4 dargestellt. Die äußere Schleife wechselt zwischen den vorbereiteten Routingstrukturen. Die rekursive Funktion VISIT führt dagegen die geordnete Tiefensuche innerhalb der jeweils aktuellen Struktur aus.

Algorithmus 4 Routing über vorbereitete Pfad- und Baumstrukturen

Require: Geordnete Routingstrukturen $structures$, Startknoten s, Zielknoten d, ausgefallene Kanten $failedEdges$

$totalHops \leftarrow 0$
$fullRoute \leftarrow [\,]$
for all $structure \in structures$ **do**
    $tree \leftarrow$ Graph der aktuellen Struktur
    if s nicht in $tree$ **then**
        Fahre mit der nächsten Struktur fort
    end if
    Wurzele $tree$ in s und bestimme seine Kindknoten
    $exits \leftarrow$ Ausgangsknoten der aktuellen Struktur
    $route \leftarrow [\,]$
    $success \leftarrow$ VISIT(s)
    $totalHops \leftarrow totalHops + length(route)$
    Hänge $route$ an $fullRoute$ an
    if $success$ **then**
        return $true$, $totalHops$, $fullRoute$
    end if
end for
return $false$, -1, $fullRoute$

function VISIT(u)
    if $u \in exits$ und Kante (u, d) nicht in $failedEdges$ **then**
        Hänge (u, d) an $route$ an
        return $true$
    end if
    $children \leftarrow$ Kindknoten von u, geordnet nach
    (1) ursprünglichem Pfad bei One Tree,
    (2) Abstand zum nächsten Ausgang und
    (3) stabiler Knotendarstellung
    for all $v \in children$ **do**
        if Kante (u, v) in $failedEdges$ **then**
            Fahre mit dem nächsten Kindknoten fort
        end if
        Hänge (u, v) an $route$ an
        if VISIT(v) **then**
            return $true$
        end if
        Hänge die Rückkante (v, u) an $route$ an
    end for
    return $false$
end function

Erzeugt die erste Struktur beispielsweise die erfolglose Bewegung $(s, a), (a, s)$ und erreicht die zweite Struktur das Ziel über $(s, c), (c, d)$, ergibt sich die Hop-Anzahl

$$h = 2 + 2 = 4.$$

Scheitern alle Strukturen, erhält das Ergebnis den Hop-Wert -1. Die durchlaufene Kantenfolge bleibt zwar zur Diagnose erhalten, der Fehlschlag wird jedoch nicht als kurze erfolgreiche Route in die Auswertung übernommen.

4.5.5 Vergleich mit der TREE-Referenzimplementierung

Die Routingroutine orientiert sich an der TREE-Referenzimplementierung. Wie dort werden die vorbereiteten Strukturen nacheinander durch eine geordnete Tiefensuche geprüft. Ausgefallene Äste werden nicht betreten, Rückwege beim Backtracking als Hops gezählt und die Hop-Anzahlen mehrerer getesteter Strukturen addiert. Ein vollständiger Fehlschlag wird in beiden Implementierungen mit `hops = -1` gekennzeichnet.

Bei den reinen EDP-Verfahren testen beide Implementierungen die Pfade vom kürzesten zum längsten. Unterschiede bestehen vor allem bei der Reihenfolge der Bäume und ihrer Äste. Die TREE-Referenz berechnet mit `rankTree(...)` eine Rangfolge der möglichen Wege zum Ziel. `getTreeOrder(...)` ordnet die Bäume anhand dieser Rangfolge. Innerhalb eines Baums verändert `rankDfs(...)` die DFS-Reihenfolge, damit günstig eingeschätzte Richtungen zuerst besucht werden.

Die vorliegende Implementierung ordnet die Strukturen dagegen nach der Länge ihres ursprünglichen EDPs beziehungsweise NDPs. Innerhalb einer Struktur wird mit `_distance_to_exit(...)` die minimale Baumdistanz zum nächsten Ausgang berücksichtigt. Beim One Tree wird zusätzlich der ursprüngliche disjunkte Pfad bevorzugt. Eine Sortierung nach der Knotendarstellung löst verbleibende Gleichstände deterministisch auf.

Die Rangordnung der TREE-Referenz kann Vorteile bei der Hop-Anzahl besitzen, da Bäume und Äste anhand ihrer Entfernung zu einem Ausgang priorisiert werden. Die vorliegende Implementierung verwendet eine einfachere deterministische Ordnung, die einheitlich auf EDP- und NDP-basierte Verfahren angewendet werden kann. Ihr Vorteil liegt vor allem in der Reproduzierbarkeit, Nachvollziehbarkeit und Erweiterbarkeit. Eine Aussage darüber, welche Reihenfolge insgesamt zu einer geringeren durchschnittlichen Hop-Anzahl führt, ist ohne einen gesonderten kontrollierten Vergleich nicht möglich.

Auch die Behandlung ausgefallener Kanten ist unterschiedlich umgesetzt. Die TREE-Referenz entfernt mit `removeFails(...)` den zugehörigen Abschnitt aus der vorbereiteten DFS-Folge. Die vorliegende Implementierung überspringt den betroffenen Ast während der rekursiven Tiefensuche. In beiden Fällen wird der hinter der ausgefallenen Kante liegende Teilbaum nicht betreten.

Die Routingprinzipien sind damit vergleichbar, die konkrete Besuchsreihenfolge kann jedoch von der TREE-Referenzimplementierung abweichen. Die NDP-basierten Verfahren und die Behandlung von Knotenausfällen sind Erweiterungen der vorliegenden Arbeit und nicht Bestandteil der ursprünglichen TREE-Referenzimplementierung.

Die erzeugten Routingergebnisse bilden die Grundlage für die in Abschnitt 4.6 beschriebenen Evaluierungskomponenten.

4.6 Evaluierungskomponenten

Nach der Durchführung der Routingversuche müssen die Einzelergebnisse gespeichert, zusammengefasst und dargestellt werden. Diese Aufgaben werden von den Evaluierungskomponenten übernommen. Zunächst schreibt `experiment.py` die Ergebnisse jeder Versuchsszene in CSV-Dateien. Anschließend berechnet `result_analysis.py` daraus die Metriken Resilience, Average Hops und Average Stretch. Die Pipeline speichert die aggregierten Ergebnisse in einer Summary-CSV und verwendet dieselben aggregierten Daten zur Erzeugung der Vergleichsplots.

Der Abschnitt ist in drei Teile gegliedert. Zunächst beschreibt Abschnitt 4.6.1 die Struktur und Speicherung der Rohdaten- und Summary-CSV-Dateien. Anschließend erläutert Abschnitt 4.6.2 die Berechnung der Bewertungsmetriken Resilience, Average Hops und Average Stretch. Abschließend behandelt Abschnitt 4.6.3 die automatische Erstellung und Speicherung der Vergleichsplots.

4.6.1 CSV-Dateien

Die Ergebnisse werden zunächst als Rohdaten im langen CSV-Format gespeichert. Eine Zeile beschreibt das Ergebnis eines Algorithmus in einer bestimmten Wiederholung und unter einer bestimmten Ausfallkonfiguration. Dadurch erhalten alle ausgewählten Algorithmen für dieselbe Versuchsszene jeweils eine eigene Ergebniszeile.

Zu den gespeicherten Angaben gehören unter anderem die Nummer der Wiederholung, der Algorithmus, der Pfadtyp, das Fehlermodell, die Graphparameter sowie die Start- und Zielknoten. Zusätzlich werden die Anzahl der berechneten Pfade und die ausgewählten Ausfallparameter gespeichert.

Für die spätere Auswertung sind insbesondere die folgenden Spalten relevant:

- `rep`: Nummer der Wiederholung,
- `algorithm`: ausgeführter Routingalgorithmus,
- `failureType`: Link- oder Knotenausfall,
- `failureModel`: zufälliger oder geclusterter Ausfall,
- `nrFailedEdges`: Anzahl der ausgefallenen Kanten,

- `nrFailedNodes`: Anzahl der ausgefallenen Knoten,
- `success`: Erfolg oder Misserfolg des Routings,
- `hops`: Anzahl der ausgeführten Routingschritte,
- `spLength`: Länge des kürzesten verbleibenden Pfads,
- `stretch`: zusätzliche Routingschritte gegenüber diesem kürzesten Pfad.

Bei einem Linkausfall enthält `nrFailedEdges` die direkt ausgewählten Kanten und `nrFailedNodes` den Wert null. Bei einem Knotenausfall enthält `nrFailedNodes` die Anzahl der ausgefallenen Knoten. `nrFailedEdges` gibt in diesem Fall die Anzahl ihrer betroffenen inzidenten Kanten an. Die beiden Spalten besitzen daher unterschiedliche Bedeutungen und können bei einem Knotenausfall verschiedene Werte enthalten.

Die wesentlichen Ergebniswerte werden in `create_instance_rows(...)` folgendermaßen erzeugt:

```
{
    "rep": rep,
    "algorithm": result.algorithm,
    "nrPaths": result.nr_paths,
    "nrFailedEdges": len(failed_edges),
    "nrFailedNodes": len(failed_nodes),
    "spLength": shortest,
    "success": int(result.success),
    "hops": result.hops,
    "stretch": (
        result.hops - shortest
        if result.success and shortest >= 0
        else ""
    ),
}
```

Listing 19: Erzeugung der wesentlichen CSV-Ergebniswerte

Ein erfolgreiches Routing wird in `success` mit dem Wert 1 gespeichert. Bei einem Misserfolg wird der Wert 0 verwendet und `hops` erhält den Wert -1. Für fehlgeschlagene Routings wird kein Stretch-Wert eingetragen. Dadurch können diese Werte später gezielt von der Mittelwertbildung ausgeschlossen werden.

Für jede Kombination aus Fehlertyp, Fehlermodell und Failure-Parameter wird eine eigene Rohdaten-CSV-Datei erzeugt. Die Dateien werden innerhalb des Ausgabeordners unter

`raw/<Dataset-Name>/`

gespeichert. Die Pipeline liest anschließend alle Rohdaten-Dateien ein und ergänzt jede Zeile um den zugehörigen Dataset-Schlüssel.

Neben den Rohdaten erzeugt die Pipeline eine Summary-CSV. Darin werden die Wiederholungen nach Dataset, Fehlertyp, Fehlermodell, Failure-Parameter und Algorithmus zusammengefasst. Die Summary enthält die berechneten Metriken sowie die Anzahl der vorhandenen und erfolgreichen Versuchsszenen. Sie wird unter

`summaries/<Run-Name>_summary.csv`

gespeichert.

4.6.2 Bewertungsmetriken

Für die Bewertung der Routingalgorithmen werden die drei Metriken Resilience, Average Hops und Average Stretch verwendet. Die Berechnung erfolgt getrennt für jede Kombination aus Dataset, Fehlertyp, Fehlermodell, Failure-Parameter und Algorithmus. Die Ergebnisse verschiedener Algorithmen werden daher nicht vor der Berechnung miteinander vermischt.

Resilience: Sei R die Anzahl der Wiederholungen einer solchen Konfiguration. Für jede Wiederholung r wird der Erfolgswert

$$x_r = \begin{cases} 1, & \text{wenn das Routing das Ziel erreicht,} \\ 0, & \text{wenn das Routing fehlschlägt} \end{cases}$$

gespeichert. Die Resilience beschreibt den Anteil der erfolgreichen Routings:

$$\text{Resilience} = \frac{1}{R} \sum_{r=1}^{R} x_r.$$

Da x_r nur die Werte null und eins annehmen kann, entspricht diese Berechnung dem Verhältnis zwischen der Anzahl erfolgreicher Routings und der Gesamtzahl der Wiederholungen:

$$\text{Resilience} = \frac{\text{successes}}{\text{samples}}.$$

Der Wert liegt zwischen null und eins. Ein Wert von 1 bedeutet, dass der Algorithmus in allen Versuchsszenen erfolgreich war. Ein Wert von 0 bedeutet, dass keine Route das Ziel erreicht hat.

Die Resilience wird damit wie im TREE-Paper als Routing-Erfolgsrate berechnet. Während die Referenzauswertung den Wert teilweise als Prozentwert zwischen 0 und 100 darstellt, verwendet die vorliegende Implementierung einen Wert zwischen 0 und 1.

Average Hops: Für die Berechnung von Average Hops werden ausschließlich erfolgreiche Routings des jeweils betrachteten Algorithmus verwendet. Die Menge dieser Wiederholungen wird durch

$$\mathcal{R}^+ = \{r \in \{1, \ldots, k\} \mid x_r = 1\}$$

beschrieben.

Sei h_r die Anzahl der ausgeführten Routingschritte einer erfolgreichen Wiederholung. Dann ergibt sich Average Hops aus

$$\mathrm{AverageHops} = \frac{1}{|\mathcal{R}^+|} \sum_{r=1}^{k} h_r \quad \texttt{mit} \quad |\mathcal{R}^+| > 0.$$

Die Hop-Anzahl umfasst alle während des Routings tatsächlich durchlaufenen Kanten. Dazu gehören neben dem Weg zum Ziel auch Rückwege aus Sackgassen und Wege innerhalb zuvor erfolglos getesteter Routingstrukturen. Ausgefallene Kanten werden nicht durchlaufen und daher nicht als Hop gezählt.

Scheitern sämtliche Routingstrukturen eines Algorithmus, wird für diese Wiederholung der Wert -1 gespeichert. Dieser Wert geht nicht in die Berechnung von Average Hops ein.

Die aktuelle Implementierung verwendet für jeden Algorithmus dessen eigene Menge $\mathcal{R}^+$ und $|\mathcal{R}^+|$ gibt die Anzahl ihrer Elemente an. Ein Algorithmus mit höherer Resilience kann deshalb zusätzliche und möglicherweise schwierigere Versuchsszenen in seinem Hop-Mittelwert enthalten. Die Berechnung unterscheidet sich damit vom kontrollierten Hop-Vergleich des TREE-Papers, bei dem nur Wiederholungen betrachtet werden, in denen das EDP-Verfahren erfolgreich war.

Average Stretch: Für die Berechnung des Stretch wird zunächst die Länge ℓ_r des kürzesten noch verfügbaren Pfads bestimmt. Dazu werden die ausgefallenen Kanten aus einer Kopie des Graphen entfernt. Anschließend berechnet NetworkX die kürzeste Pfadlänge zwischen s und d.

Der Stretch einer erfolgreichen Wiederholung ist in dieser Implementierung als additive Differenz definiert:

$$\mathrm{Stretch}_r = h_r - \ell_r.$$

Average Stretch ergibt sich entsprechend aus

$$\text{AverageStretch} = \frac{1}{|\mathcal{R}^+|} \sum_{r=1}^{k} (h_r - \ell_r) \quad \texttt{mit} \quad |\mathcal{R}^+| > 0.$$

Ein Stretch von null bedeutet, dass die ausgeführte Route genauso lang wie der kürzeste verbleibende Pfad war. Ein positiver Wert gibt an, wie viele zusätzliche Routingschritte der Algorithmus gegenüber der kürzesten noch möglichen Route benötigt hat.

Stretch erleichtert damit die Unterscheidung zwischen einer unvermeidbaren Pfadverlängerung durch die Ausfälle und zusätzlichen Umwegen des Routingalgorithmus. Er wird in dieser Implementierung nicht als Verhältnis, sondern als Differenz berechnet. Wie bei Average Hops werden nur erfolgreiche Routings des jeweils betrachteten Algorithmus berücksichtigt.

Aggregation der Metriken Die Aggregation der drei Metriken ist in Listing 20 dargestellt:

```
raw["hopsSuccessful"] = raw["hops"].where(
    raw["success"] == 1
)
raw["stretchSuccessful"] = raw["stretch"].where(
    raw["success"] == 1
)

# ...

.agg(
    resilience=("success", "mean"),
    avgHops=("hopsSuccessful", "mean"),
    avgStretch=("stretchSuccessful", "mean"),
    samples=("success", "size"),
    successes=("success", "sum"),
)
```

Listing 20: Aggregation der Bewertungsmetriken

Der Ausdruck `mean` bezeichnet den arithmetischen Mittelwert. Die Funktion `agg(...)` führt die angegebenen Berechnungen getrennt für jede zuvor gebildete Ergebnisgruppe aus.

Die Hilfsspalten `hopsSuccessful` und `stretchSuccessful` enthalten nur Werte erfolgreicher Routings. Bei einem fehlgeschlagenen Routing wird stattdessen der fehlende Wert `NaN` eingetragen. Pandas ignoriert solche Werte bei der Berechnung von `mean`. Dadurch werden fehlgeschlagene Routings nicht in Average Hops und Average Stretch einbezogen.

Die Aggregation `size` zählt alle Zeilen einer Gruppe und liefert damit die Anzahl `samples`. Die Aggregation `sum` addiert die binären Erfolgswerte und liefert die Anzahl `successes`.

Das folgende Beispiel zeigt die Berechnung für fünf Wiederholungen:

Wiederholung	`success`	`hopsSuccessful`	`stretchSuccessful`
1	1	4	1
2	0	`NaN`	`NaN`
3	1	6	2
4	1	8	4
5	0	`NaN`	`NaN`

Aus den fünf Wiederholungen ergeben sich drei erfolgreiche Routings. Die Resilience beträgt daher

$$\text{Resilience} = \frac{1+0+1+1+0}{5} = \frac{3}{5} = 0{,}6.$$

Da bei Average Hops nur die drei erfolgreichen Werte berücksichtigt werden, ergibt sich

$$\text{AverageHops} = \frac{4+6+8}{3} = 6.$$

Average Stretch wird entsprechend durch

$$\text{AverageStretch} = \frac{1+2+4}{3} \approx 2{,}33$$

berechnet. Außerdem liefert die Aggregation

$$\text{samples} = 5 \qquad \text{und} \qquad \text{successes} = 3.$$

Falls kein Routing einer Gruppe erfolgreich ist, enthalten `hopsSuccessful` und `stretchSuccessful` ausschließlich fehlende Werte. In diesem Fall können Average Hops und Average Stretch nicht berechnet werden und bleiben in der Summary als `NaN` gespeichert.

4.6.3 Ploterstellung

Die Plots werden aus der aggregierten Ergebnistabelle erzeugt, die im Arbeitsspeicher vorliegt und anschließend auch als Summary-CSV gespeichert wird. Die Summary-Datei wird für die Ploterstellung daher nicht erneut eingelesen. Auf der horizontalen Achse

steht der jeweilige Failure-Parameter. Bei Random Failure ist dies der `failFactor`; bei Clustered Failure wird `failureStart` verwendet. Beide Werte werden bei der Aggregation in der gemeinsamen Spalte `failureParameter` zusammengeführt.

Ein Dataset beschreibt eine Graphkonfiguration. Bei synthetischen Random-Graphen gehören dazu die Graphart und ihre Parameter, beispielsweise n und p. Für jede Wiederholung wird daraus eine neue Graphinstanz erzeugt. Ein Real-World-Dataset bezeichnet dagegen eine feste GraphML-Topologie, die in jeder Wiederholung erneut verwendet wird. Zusätzlich kann jedes Dataset eigene Bereiche für `failFactor`, `failureStart` und `failureDrop` festlegen.

Für jede Kombination aus Fehlermodell, Fehlertyp und ausgewählter Metrik wird eine eigene Abbildung erzeugt. Ein Teilplot zeigt dabei die Ergebnisse genau eines Datasets für diese Failure-Kombination und Metrik. Die dargestellten Kurven entsprechen den ausgewählten Routingalgorithmen. Abhängig von der Konfiguration kann ein Teilplot daher zwischen einer und sechs Algorithmuskurven enthalten.

Die Darstellung unterscheidet die Algorithmen durch feste Farben und Beschriftungen. EDP-basierte Verfahren werden mit durchgezogenen Linien und NDP-basierte Verfahren mit gestrichelten Linien dargestellt:

```
ax.plot(
    values["failureParameter"],
    values[metric],
    marker="o",
    linewidth=2,
    color=ALGORITHM_COLORS[algorithm],
    label=ALGORITHM_LABELS[algorithm],
    linestyle=(
        "--"
        if algorithm.startswith("ndp_")
        else "-"
    ),
)
```

Listing 21: Darstellung der Algorithmen in den Vergleichsplots

CLI und GUI verwenden dieselbe automatische Aufteilung der Vergleichsplots. Bis zu drei Datasets derselben Graphkategorie werden in einer gemeinsamen Vergleichsabbildung nebeneinander dargestellt. Für alle Teilplots einer Abbildung gelten dieselbe Failure-Kombination und dieselbe Metrik. Eine Abbildung kann somit beispielsweise Average Hops bei Random Node Failure für drei unterschiedliche Random-Graph-Datasets vergleichen.

Eine Vergleichsabbildung besteht aus einem, zwei oder höchstens drei nebeneinander angeordneten Teilplots. Werden mehr als drei Datasets einer Graphkategorie ausgewählt, erzeugt die Pipeline automatisch weitere Plotgruppen. Synthetische Random-Graphen und

Real-World-Graphen werden dabei nicht in derselben Abbildung kombiniert. Dadurch besitzen die über CLI und GUI erzeugten Vergleichsplots bei gleicher Dataset-Auswahl denselben Aufbau.

Jeder Plot wird in zwei Formaten gespeichert:

- PNG für die Anzeige in der GUI und die schnelle Betrachtung,
- SVG für eine verlustfreie Einbindung in die Ausarbeitung.

Die Dateien werden nach Plotgruppe getrennt unter

```
plots/random_graphs/
plots/real_world/
```

abgelegt. Der Dateiname setzt sich aus Fehlermodell, Fehlertyp und Metrik zusammen. Ein Plot für Average Hops bei zufälligen Knotenausfällen erhält beispielsweise den Namen

```
random_node_avgHops.png
```

Vor der Ploterstellung prüft die Pipeline, ob für jede Konfiguration alle ausgewählten Algorithmen und die erwartete Anzahl an Wiederholungen vorhanden sind. Zusätzlich wird kontrolliert, ob alle Resilience-Werte im zulässigen Bereich zwischen null und eins liegen. Dadurch werden unvollständige Versuchsreihen erkannt, bevor aus ihnen Plots erzeugt werden.

4.7 Pipeline

Die einzelnen Experimentfunktionen können unabhängig voneinander aufgerufen werden. Für eine vollständige Evaluation müssten dadurch jedoch viele Kombinationen aus Graphen, Fehlermodellen, Failure-Parametern und Algorithmen manuell gestartet und anschließend ausgewertet werden. Die in Schritt 5 entwickelte Pipeline automatisiert diesen Ablauf.

Die Datei `pipeline.py` bildet den Kern der Automatisierung. Sie liest eine Konfiguration ein, erstellt daraus einen Experimentplan, führt die Experimente aus und erzeugt anschließend die zusammengefassten Ergebnisse und Plots. Die Pipeline kann sowohl über die Kommandozeile als auch über die grafische Bedienoberfläche gestartet werden.

Der Abschnitt ist in vier Teile gegliedert. Zunächst beschreibt Abschnitt 4.7.1 den Aufbau und die Verwendung der JSON-Konfiguration. Anschließend erläutert Abschnitt 4.7.2 die Bedienung der Pipeline über die Kommandozeile und die grafische Benutzeroberfläche. Der automatische Ablauf der Experimente wird in Abschnitt 4.7.3 dargestellt. Abschließend behandelt Abschnitt 4.7.4 die Aggregation der Ergebnisse und die automatische Erzeugung der Vergleichsplots.

4.7.1 JSON-Konfiguration

Die Einstellungen eines Pipeline-Laufs werden in einer JSON-Datei gespeichert. Dadurch müssen Parameter nicht direkt im Python-Code verändert werden. Eine Konfiguration enthält unter anderem den Namen des Laufs, die Anzahl der Wiederholungen, den Seed, die Algorithmen, die Fehlertypen, die Metriken und die verwendeten Datasets.

Listing 22 zeigt den grundsätzlichen Aufbau anhand einer vereinfachten Fassung der Konfiguration `full.json`. Die Listen der Failure-Parameter wurden für die Darstellung gekürzt.

```
{
  "run_name": "full_runs1000",
  "title": "Step 5 CLI",
  "runs": 1000,
  "seed": 123,
  "resume": true,
  "algorithms": [
    "edp_paths",
    "edp_one_tree",
    "edp_multiple_trees",
    "ndp_paths",
    "ndp_one_tree",
    "ndp_multiple_trees"
  ],
  "failures": [
    "random_link",
    "random_node",
    "clustered_link",
    "clustered_node"
  ],
  "metrics": [
    "resilience",
    "avgHops",
    "avgStretch"
  ],
  "failure_starts": [0.3, 0.4, 0.5, 0.6,
                     0.7, 0.8, 0.9],
  "failure_drop": 0.3,

  "datasets": [
    {
      "name": "synthetic_n25_p015",
      "type": "synthetic",
      "graph_type": "erdos_renyi",
      "nodes": 25,
      "p": 0.15,
      "fail_factors": [0.8, 1.6, 2.4]
    }
  ]
```

```
}
```

Listing 22: Gekürzte JSON-Konfiguration eines Pipeline-Laufs

Synthetische Datasets werden durch ihren Graphentyp und die zugehörigen Parameter beschrieben. Für einen Erdős-Rényi-Graphen werden beispielsweise die Knotenzahl n und die Kantenwahrscheinlichkeit p angegeben. Bei einem Random-Regular-Graphen wird anstelle von p der Knotengrad `degree` verwendet.

Real-World-Graphen besitzen den Dataset-Typ `graphml`. Ihre Konfiguration enthält einen Pfad zu einer GraphML-Datei. Relative Datenpfade werden vom Schritt-5-Ordner aus aufgelöst. Dadurch findet die Pipeline die mitgelieferten Topologien auch dann, wenn der gesamte Schritt-5-Ordner an eine andere Stelle kopiert wird.

Die Failure-Parameter können global oder innerhalb eines einzelnen Datasets angegeben werden. Dataset-spezifische Werte besitzen Vorrang vor den globalen Werten. Dadurch können beispielsweise verschiedene `fail_factors`-Bereiche für unterschiedliche Graphgrößen in einem einzigen Pipeline-Lauf verwendet werden.

Nicht angegebene Werte werden durch Standardwerte ergänzt. Dazu gehören unter anderem fünf Wiederholungen, der Seed 123, aktiviertes Fortsetzen und alle sechs Algorithmen. Vor einem regulären Lauf speichert die Pipeline die aufbereitete Konfiguration als `used_config.json` im Ausgabeordner. Allgemeine Parameter und Dataset-Parameter, die über die Kommandozeile überschrieben wurden, sind darin enthalten. Die tatsächlich ausgeführten Algorithmen, Fehlermodelle und Jobs werden zusätzlich in der `manifest.json` dokumentiert.

4.7.2 CLI und GUI

Die Pipeline kann direkt über die Kommandozeile gestartet werden. Der folgende Befehl führt die vollständige in `full.json` beschriebene Pipeline aus:

```
python schritt5_pipeline_automatisierung/pipeline.py --config
    schritt5_pipeline_automatisierung/config/full.json
```

Listing 23: Start der Pipeline über die Kommandozeile

Die JSON-Datei bildet dabei die Basiskonfiguration. Häufig benötigte Parameter können beim Aufruf überschrieben werden. Dazu gehören beispielsweise `-runs`, `-seed`, `-algorithms`, `-failures`, `-datasets`, `-fail-factor`, `-failure-start` und `-failure-drop`. Für synthetische Graphen können außerdem `-graph-type`, `-nodes`, `-p` und `-degree` verwendet werden.

Ein verkürzter Testlauf kann beispielsweise folgendermaßen gestartet werden:

```
python schritt5_pipeline_automatisierung/pipeline.py \
  --config schritt5_pipeline_automatisierung/config/pilot.json \
  --run-name quick_test \
  --runs 10 \
  --failures random_link random_node \
  --fail-factor 0.8 1.6 2.4
```

Listing 24: Überschreiben einzelner Parameter über die CLI

Mit `-dry-run` kann der erzeugte Experimentplan angezeigt werden, ohne Experimente oder Plots auszuführen. In der GUI wird der Plan als Tabelle dargestellt. Sie enthält für jeden geplanten Job unter anderem das Dataset, die Failure-Kombination, die Anzahl der Wiederholungen, die Graphparameter und den vorgesehenen Ausgabeordner. Ein Job bezeichnet dabei eine Kombination aus Dataset und Failure-Modell. Sind mehrere Werte für `failFactor` oder `failureStart` eingestellt, kann ein Job mehrere Rohdaten-CSVs erzeugen.

Wie Abbildung 6 zeigt, kann die dargestellte Plantabelle über die Download-Funktion von Streamlit als CSV-Datei gespeichert werden. Die heruntergeladene Datei dient zur Kontrolle der vorgesehenen Jobs und enthält noch keine Ergebnisse der Routingexperimente.

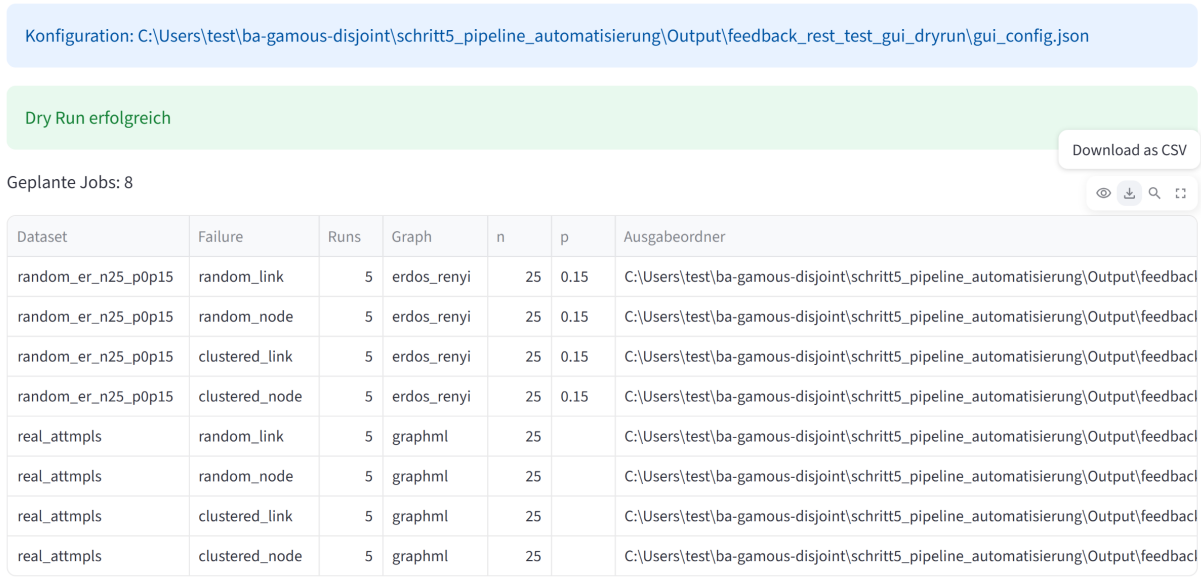


Abbildung 6: Dry-Run-Ausgabe der GUI mit den geplanten Experimentjobs und der Möglichkeit, den Experimentplan als CSV-Datei herunterzuladen

Der Modus `-preflight` führt zusätzliche Vorabprüfungen durch. Kontrolliert werden unter anderem die ausgewählten Algorithmen und Metriken, die Graphparameter, die zulässigen Wertebereiche der Failure-Parameter, die vorhandenen GraphML-Dateien, die

benötigten Plotangaben und die Schreibbarkeit des Ausgabeordners. Bei erfolgreicher Prüfung zeigt die GUI die Anzahl der Datasets und Jobs sowie den vorgesehenen Ausgabeordner an. Ein Beispiel dafür ist in Abbildung 7 dargestellt.

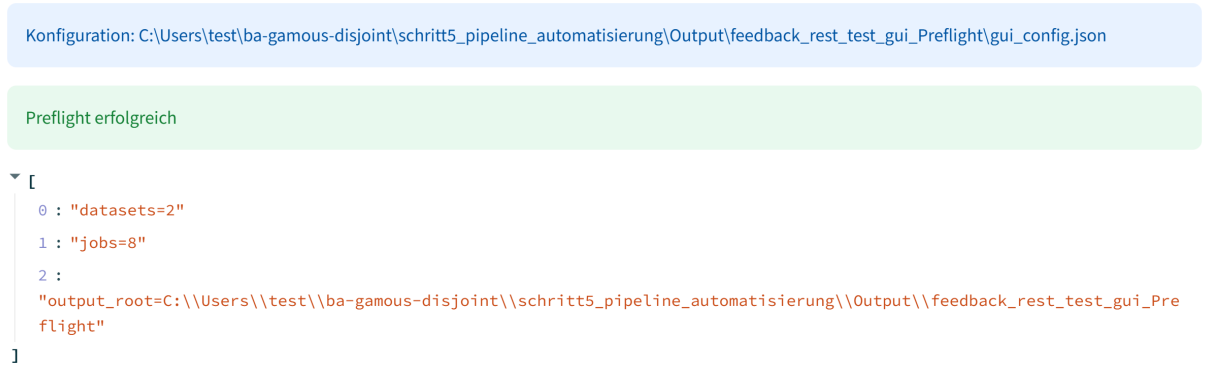


Abbildung 7: Erfolgreiche Preflight-Prüfung der GUI mit der Anzahl der geprüften Datasets und Experimentjobs

Dry Run und Preflight führen keine Routingexperimente aus und erzeugen daher keine Rohdaten oder Ergebnisplots. Beide Modi verfolgen unterschiedliche Zwecke und können nicht miteinander kombiniert werden: Der Dry Run zeigt, was ausgeführt würde, während Preflight prüft, ob die Konfiguration mit den vorhandenen Eingaben grundsätzlich ausgeführt werden kann.

Die grafische Bedienoberfläche ist in `gui.py` implementiert und verwendet die Python-Bibliothek Streamlit. Sie kann mit folgendem Befehl gestartet werden:

```
python -m streamlit run schritt5_pipeline_automatisierung/gui.py
```

Listing 25: Start der grafischen Bedienoberfläche

In der GUI können Algorithmen, Fehlertypen, Metriken und mehrere Graphkonfigurationen ausgewählt werden. Für Erdős-Rényi-Graphen lassen sich n, p und die Bereiche der Failure-Parameter einstellen. Für Real-World-Graphen stehen die eingebundenen GraphML-Topologien zur Verfügung. Random-Regular-Graphen werden vom Pipelinekern ebenfalls unterstützt, müssen aktuell jedoch über eine JSON-Konfiguration oder die CLI mit `-graph-type random_regular` und `-degree` ausgewählt werden. Die GUI verwendet den festen Seed 123 und aktiviert die Resume-Funktion. Über die CLI können der Seed und das Resume-Verhalten dagegen überschrieben werden.

Die GUI ersetzt die Pipeline nicht. Sie erzeugt aus den Eingaben zunächst eine Datei namens `gui_config.json` und ruft anschließend dieselbe Funktion `run_pipeline(...)` auf, die auch von der CLI verwendet wird. Dieser Zusammenhang ist in Listing 26 dargestellt.

```
config = current_config()
config_path = write_gui_config(config)

st.session_state.last_result = pipeline.run_pipeline(
    config_path,
    **mode_to_pipeline_flags(mode),
)
```

Listing 26: Erzeugung der GUI-Konfiguration und Aufruf der Pipeline

CLI und GUI verwenden somit denselben Experiment- und Auswertungscode. Bei gleicher effektiver Konfiguration, gleichem Seed und gleicher Softwareumgebung erzeugen beide Bedienwege dieselben Versuchsszenen und Messwerte. Unterschiede bestehen hauptsächlich in der Bedienung, in den Plot-Titeln und in den gewählten Ausgabeordnern.

Die Auswahl *Lokal ausführen* bedeutet, dass die Pipeline auf dem Rechner ausgeführt wird, auf dem der Streamlit-Prozess läuft. Wird Streamlit auf einem Server gestartet, erfolgt auch diese lokale Ausführung auf dem Server. Die Auswahl *Server vorbereiten* startet dagegen keinen entfernten Prozess. Sie speichert die Konfiguration und erzeugt ein Shell-Skript mit dem passenden CLI-Befehl für einen späteren Serverlauf.

4.7.3 Automatische Ausführung

Nach dem Laden der Konfiguration erstellt die Pipeline zunächst einen Experimentplan. Dazu wird jedes ausgewählte Dataset mit jeder ausgewählten Failure-Kombination verbunden. Sei n_D die Anzahl der Datasets und n_F die Anzahl der ausgewählten Failure-Kombinationen. Die Anzahl der geplanten Jobs ergibt sich aus

$$n_J = n_D \cdot n_F.$$

Bei sechs Datasets und vier Failure-Kombinationen entstehen beispielsweise

$$n_J = 6 \cdot 4 = 24$$

Jobs. Innerhalb jedes Jobs werden alle ausgewählten Algorithmen und alle für das Dataset festgelegten Failure-Parameter ausgeführt. Die Algorithmen bilden daher keine zusätzliche Dimension des Jobplans.

Der Aufbau des Plans ist in Listing 27 vereinfacht dargestellt.

```
for dataset in datasets:
    parameters = dataset_parameters(config, dataset)
```

```
    for failure_name, (
        failure_model,
        failure_type,
    ) in failures:
        plan.append({
            "dataset": dataset,
            "failure": failure_name,
            "failure_model": failure_model,
            "failure_type": failure_type,
            "algorithms": algorithms,
            "fail_factors":
                parameters["fail_factors"],
            "failure_starts":
                parameters["failure_starts"],
            "failure_drop":
                parameters["failure_drop"],
        })
```

Listing 27: Erzeugung des Experimentplans

Für jeden Eintrag ruft `run_experiments(...)` die in `experiment.py` implementierte Funktion `run_experiment_series(...)` auf. Diese erzeugt die einzelnen Versuchsszenen, führt alle ausgewählten Routingalgorithmen aus und schreibt die Rohdaten in CSV-Dateien.

Ist `resume` aktiviert, werden bereits vollständig gespeicherte Wiederholungen erkannt. Ein unterbrochener Lauf kann dadurch fortgesetzt werden, ohne vollständige Wiederholungen doppelt in die CSV-Dateien zu schreiben.

Die wesentlichen Phasen der Pipeline sind in Listing 28 dargestellt.

```
write_used_config(config, output_root)

if not only_plots:
    generated_csvs = run_experiments(
        config, plan, resume_override
    )

if not only_experiments:
    summary_path, plot_paths = (
        write_summary_and_plots(
            config, datasets,
            algorithms, failures
        )
    )

manifest_path = write_manifest(
    config,
    config_path,
    plan,
```

```
        generated_csvs,
        summary_path,
        plot_paths,
    )
```

Listing 28: Automatischer Ablauf der Pipeline

Im normalen Modus werden die Experimente und die Auswertung direkt hintereinander ausgeführt. Mit `--only-experiments` kann die Ausführung nach der Erzeugung der Rohdaten beendet werden. Der Modus `--only-plots` überspringt dagegen die Experimente und erstellt die Auswertung aus bereits vorhandenen CSV-Dateien.

Am Ende wird eine `manifest.json` erzeugt. Sie enthält unter anderem den Seed, die Python-Version, die ausgewählten Algorithmen und Datasets, die Failure-Parameter sowie die Pfade aller erzeugten CSV- und Plot-Dateien. Das Manifest dokumentiert damit, welche Ergebnisse zu einem Pipeline-Lauf gehören.

Relative Ausgabepfade werden immer vom Schritt-5-Ordner aus aufgelöst. Der Standardordner eines Laufs ist

```
schritt5_pipeline_automatisierung/Output/<run_name>/
```

Dadurch bleiben die Ergebnisse innerhalb des Schritt-5-Ordners, unabhängig davon, aus welchem Arbeitsverzeichnis die Pipeline gestartet wird.

4.7.4 Automatische Plot-Erstellung

Nach Abschluss der Experimente ruft die Pipeline `write_summary_and_plots(...)` auf. Die Funktion liest die vorhandenen Rohdaten ein und übergibt sie zunächst an `aggregate_results(...)`. Anschließend prüft `validate_summary(...)` die Vollständigkeit der Ergebnisse. Für jede Plotgruppe werden danach die ausgewählten Metriken und Failure-Kombinationen an `plot_all_comparisons(...)` übergeben.

Die Berechnung der Metriken wird in Abschnitt 4.6.2 beschrieben. Der Aufbau, die Aufteilung und die Speicherung der Vergleichsplots werden in Abschnitt 4.6.3 erläutert. An dieser Stelle steht daher die automatische Einbindung dieser Komponenten in den Pipeline-Ablauf im Vordergrund.

Jede Abbildung wird sowohl im PNG- als auch im SVG-Format gespeichert. PNG wird insbesondere für die Anzeige in der GUI verwendet. SVG eignet sich für die verlustfreie Einbindung in die schriftliche Ausarbeitung. Der entsprechende Aufruf ist in Listing 29 dargestellt.

```
plot_groups = (
    config.get("plot_groups")
```

```
        or default_plot_groups(datasets)
    )

    for group in plot_groups:
        plot_paths.extend(
            result_analysis.plot_all_comparisons(
                summary,
                datasets=group["datasets"],
                algorithms=algorithms,
                output_dir=(
                    output_root
                    / "plots"
                    / group["name"]
                ),
                metrics=metric_specs(
                    config["metrics"]
                ),
                show=False,
            )
        )
```

Listing 29: Automatische Erzeugung der Vergleichsplots

Die Pipeline speichert die von der Auswertung zurückgegebenen Pfade der Summary-CSV und der Plotdateien abschließend im Manifest. Der vollständige Aufbau des Ausgabeordners wurde bereits in Abschnitt 4.1 beschrieben.

Die Pipeline verbindet damit die Konfiguration, Durchführung und Auswertung zu einem durchgängigen Ablauf. Ein einzelner Aufruf genügt, um die Versuchsdaten zu erzeugen, die Metriken zu aggregieren und die zugehörigen Abbildungen zu erstellen.

5 Evaluation

Dieses Kapitel untersucht die sechs Routingverfahren unter verschiedenen Kanten- und Knotenausfällen. Ziel ist es, die EDP- und NDP-basierten Verfahren hinsichtlich ihrer Resilienz, ihrer durchschnittlichen Hopanzahl und ihres durchschnittlichen Stretchs zu vergleichen. Dabei wird außerdem betrachtet, welchen Einfluss die Erweiterung der disjunkten Pfade zu einer One-Tree- oder Multiple-Trees-Struktur auf das Routing besitzt.

Zunächst beschreibt Abschnitt 5.1 den Versuchsaufbau. Dazu gehören die verwendeten Graphen, Ausfallmodelle, Parameter, Wiederholungszahlen und die technische Ausführungsumgebung. Abschnitt 5.2 erläutert anschließend, wie die in Abschnitt 4.6.2 definierten Bewertungsmetriken für den Vergleich der Algorithmen verwendet werden.

Die Ergebnisse für die synthetischen Random-Graphen werden in Abschnitt 5.3 dargestellt. Danach behandelt Abschnitt 5.4 die Ergebnisse auf den ausgewählten Real-World-Topologien. Abschnitt 5.5 führt die Beobachtungen beider Graphkategorien zusammen und vergleicht die sechs Routingverfahren direkt miteinander. Abschließend werden die Ergebnisse in Abschnitt 5.6 zusammengeführt und im Hinblick auf die Forschungsfragen interpretiert. Dabei werden insbesondere auffällige Unterschiede zwischen den EDP- und NDP-basierten Verfahren sowie zwischen Paths, One Tree und Multiple Trees diskutiert.

Im Hauptteil werden diejenigen Abbildungen dargestellt, die für die Beantwortung der Forschungsfragen besonders relevant sind. Der Schwerpunkt liegt auf der Resilienz sowie bei Knotenausfällen zusätzlich auf der durchschnittlichen Hopanzahl der NDP-basierten Verfahren. Sämtliche weiteren Ergebnisplots, insbesondere die vollständigen Darstellungen von Average Hops und Average Stretch, befinden sich in Anhang

5.1 Versuchsaufbau

Der Versuchsaufbau ist so gewählt, dass die sechs Routingverfahren unter denselben Bedingungen miteinander verglichen werden können. Untersucht werden EDP Paths, EDP One Tree, EDP Multiple Trees, NDP Paths, NDP One Tree und NDP Multiple Trees. Für jedes Verfahren werden zufällige und geclusterte Kanten- sowie Knotenausfälle betrachtet. Daraus ergeben sich die vier Failure-Kombinationen Random Link, Random Node, Clustered Link und Clustered Node.

Innerhalb einer Versuchsszene erhalten alle sechs Algorithmen denselben Graphen, dieselben Start- und Zielknoten und dieselbe Ausfallmenge. Dadurch können Unterschiede in den Ergebnissen auf das jeweilige Routingverfahren zurückgeführt werden. Dadurch können die Verfahren innerhalb derselben Szene direkt verglichen werden. Die drei EDP-Varianten verwenden dieselben berechneten EDPs, die drei NDP-Varianten

dieselben berechneten NDPs. Auf dieser Grundlage werden die jeweiligen Pfad- und Baumstrukturen aufgebaut.

Zunächst stellt Abschnitt 5.1.1 die verwendeten synthetischen und realen Graphen sowie die zugehörigen Failure-Parameter, Wiederholungszahlen und Randomisierungseinstellungen vor. Anschließend beschreibt Abschnitt 5.1.2 den daraus entstehenden Umfang der Evaluation. Die für die Durchführung verwendete Hardware- und Softwareumgebung wird abschließend in Abschnitt 5.1.3 dokumentiert.

5.1.1 Verwendete Graphen

Die Evaluation verwendet drei synthetische Erdős-Rényi-Graphsets und drei Real-World-Topologien.

Randomisierte Graphen Die synthetischen Graphen werden durch die Knotenzahl n und die Kantenwahrscheinlichkeit p festgelegt. Für jede Wiederholung wird eine neue Graphinstanz mit diesen Parametern erzeugt. Die verwendeten Graphsets sind in Tabelle 1 dargestellt.

Tabelle 1: Verwendete Erdős-Rényi-Graphsets und Random-Failure-Parameter

n	p	`failFactor` von	`failFactor` bis	Schrittgröße
25	0,15	0,8	8,8	0,8
50	0,15	0,8	16,8	0,8
100	0,02	0,8	49,6	0,8

Real-World-Graphen Als Real-World-Graphen werden die Topologien ATT-MPLS, Oteglobe und Cogentco aus der Internet Topology Zoo verwendet [7]. Die GraphML-Dateien werden eingelesen und in einfache ungerichtete Graphen umgewandelt. Mehrfach vorhandene Verbindungen zwischen demselben Knotenpaar werden dabei zu einer Kante zusammengefasst. Die Tabelle nennt die Größen der tatsächlich verwendeten Graphen. Im Gegensatz zu den synthetischen Graphen bleibt die jeweilige Topologie in allen Wiederholungen gleich. Die Start- und Zielknoten sowie die Ausfälle werden jedoch in jeder Wiederholung erneut bestimmt.

Tabelle 2 zeigt die Größen der drei Topologien. Sie wurden so ausgewählt, dass ein kleiner, ein mittlerer und ein größerer Real-World-Graph betrachtet werden.

Parameter der randomisierten Ausfälle Für die zufälligen Kanten- und Knotenausfälle der Real-World-Graphen werden `failFactor`-Werte von 1 bis 10 mit einer Schrittgröße

Tabelle 2: Verwendete Real-World-Topologien

Topologie	Größenklasse	Knoten $\lvert V\rvert$	Kanten $\lvert E\rvert$
ATT-MPLS	klein	25	56
Oteglobe	mittel	93	103
Cogentco	groß	197	243

von 1 verwendet. Wie in den Abschnitten 4.3.1 und 4.3.3 beschrieben, gibt `failFactor` die Anzahl der Ausfälle nicht unmittelbar an. Der Wert skaliert die aus der Anzahl der EDPs beziehungsweise NDPs abgeleitete Ausfallanzahl.

Parameter der geclusterten Ausfälle Für Clustered Link und Clustered Node werden bei allen sechs Graphsets dieselben Parameter verwendet. `failureStart` nimmt die Werte von $0{,}3$ bis $0{,}9$ in Schritten von $0{,}1$ an. Nach jeder Cluster-Runde wird der Auswahlanteil um

$$\texttt{failureDrop} = 0{,}3$$

reduziert. Die genaue Bedeutung dieser Parameter und die schichtweise Ausbreitung der Ausfälle wurden in den Abschnitten 4.3.2 und 4.3.4 beschrieben.

Wiederholungen und Randomisierung Jede Kombination aus Dataset, Failure-Kombination und zugehörigem Failure-Parameter wird 1000-mal wiederholt. Als globaler Seed wird

$$\texttt{seed} = 123$$

verwendet. Bei synthetischen Datensätzen steuert dieser Seed die Folge der erzeugten Graphen, die Auswahl der Start- und Zielknoten sowie die Ausfälle. Bei den Real-World-Datensätzen bleibt die eingelesene Topologie fest; der Seed bestimmt dort die reproduzierbare Folge der Start-Ziel-Paare und Ausfallmengen. Die Seed-basierte Steuerung wurde in Abschnitt 4.4 erläutert.

Die Start- und Zielknoten werden aus derselben Zusammenhangskomponente ausgewählt und dürfen nicht unmittelbar benachbart sein. Ein gleicher Seed bedeutet nicht, dass in allen Wiederholungen dieselbe Szene verwendet wird. Er erzeugt stattdessen eine reproduzierbare Folge unterschiedlicher Versuchsszenen.

5.1.2 Umfang der Evaluation

Die sechs Datasets werden mit den vier Failure-Kombinationen verbunden, wodurch zunächst 24 Pipeline-Jobs entstehen. Unter Berücksichtigung aller datasetabhängigen Failure-Parameter umfasst der Versuchsaufbau insgesamt 332 Parameterkonfigurationen. Jede Konfiguration wird 1000-mal wiederholt und mit allen sechs Algorithmen ausgewertet. Insgesamt entstehen damit

$$332 \cdot 1000 \cdot 6 = 1\,992\,000$$

einzelne Algorithmusergebnisse. Für jedes Ergebnis werden unter anderem der Routingerfolg, die Hopanzahl und der Stretch in den Rohdaten gespeichert.

5.1.3 Technische Ausführungsumgebung

Die finalen Experimente wurden auf einem x86-64-Server mit Debian GNU/Linux und dem Linux-Kernel 6.1.0-51-amd64 ausgeführt. Der Server besitzt zwei Intel-Xeon-Silver-4210R-Prozessoren mit jeweils zehn physischen Kernen und insgesamt 40 logischen Prozessoren sowie 125 GiB Arbeitsspeicher. Als Softwareumgebung wurden Python 3.11.2, NetworkX 3.6.1, pandas 3.0.3 und Matplotlib 3.10.9 verwendet. Die für die Experimente eingesetzte Codeversion ist durch den Git-Commit `2409aac4ea1a` dokumentiert.

5.2 Auswertungsmethodik

Die bei den Experimenten erzeugten Rohdaten werden zunächst nach Dataset, Failure-Kombination, Failure-Parameter und Routingalgorithmus gruppiert. Für jede dieser Gruppen werden die Ergebnisse der 1000 Wiederholungen zusammengefasst. Die Auswertung verwendet die bereits in Abschnitt 4.6.2 definierten Metriken Resilienz, durchschnittliche Hopanzahl und durchschnittlicher Stretch.

Abschnitt 5.2.1 beschreibt zunächst die Aggregation und Interpretation dieser Metriken. Anschließend erläutert Abschnitt 5.2.2 ihre Darstellung in den Vergleichsplots. Das Vorgehen beim Vergleich der sechs Routingalgorithmen wird abschließend in Abschnitt 5.2.3 festgelegt.

5.2.1 Aggregation und Interpretation der Metriken

Die Resilienz gibt den Anteil der Versuchsszenen an, in denen ein Routingalgorithmus den Zielknoten erreicht. Sie wird über alle 1000 Wiederholungen einer Parameterkonfiguration berechnet. Dadurch kann unmittelbar verglichen werden, wie häufig die einzelnen Algorithmen unter denselben Ausfallszenen erfolgreich routen.

Die durchschnittliche Hopanzahl und der durchschnittliche Stretch werden nur aus den erfolgreichen Routings eines Algorithmus berechnet. Für fehlgeschlagene Routings wird `hops = -1` gespeichert. Sie werden bei der Mittelwertbildung für Hopanzahl und Stretch nicht berücksichtigt, da diese beiden Metriken ausschließlich erfolgreiche Routings beschreiben. Die durchschnittliche Hopanzahl beschreibt die Anzahl der tatsächlich durchlaufenen Kanten einschließlich möglicher Rückwege. Der additive Stretch gibt an, wie viele zusätzliche Hops gegenüber dem kürzesten noch verfügbaren Pfad benötigt wurden.

Die drei Metriken werden gemeinsam betrachtet. Eine hohe Resilienz zeigt, dass ein Algorithmus viele Szenen erfolgreich bewältigt. Eine geringe durchschnittliche Hopanzahl beschreibt kurze erfolgreiche Routen. Ein geringer additiver Stretch bedeutet dagegen, dass diese Routen nur wenige zusätzliche Hops gegenüber dem jeweiligen kürzesten Restpfad benötigen. Eine Baumstruktur kann beispielsweise die Resilienz erhöhen, gleichzeitig aber durch Umwege und Backtracking mehr Hops verursachen.

Beim Vergleich der Hopanzahl und des Stretchs ist zu beachten, dass die Menge erfolgreicher Szenen zwischen den Algorithmen unterschiedlich sein kann. Ein Algorithmus mit niedriger Resilienz wird nur anhand seiner wenigen erfolgreichen Routings bewertet. Die beiden Metriken werden deshalb nicht isoliert, sondern immer zusammen mit der zugehörigen Resilienz interpretiert.

5.2.2 Darstellung der Ergebnisse

Für eine einheitliche Darstellung wird der jeweils variierte Parameter der zufälligen und geclusterten Ausfälle in allen Ergebnisplots unter dem gemeinsamen Begriff *Failure Rate* auf der x-Achse dargestellt. Diese Bezeichnung stellt jedoch keine für alle Ausfallmodelle einheitliche prozentuale Ausfallrate dar.

Bei zufälligen Link- und Knotenausfällen entspricht der dargestellte Wert dem Parameter `failFactor`. Dieser skaliert die Anzahl der Ausfälle bei Linkausfällen anhand der Anzahl der EDPs und bei Knotenausfällen anhand der Anzahl der NDPs. Bei geclusterten Link- und Knotenausfällen entspricht der dargestellte Wert dagegen dem Parameter `failureStart`. Dieser legt den anfänglichen Auswahlanteil in der ersten Schicht um den Zielknoten fest und wird in den folgenden Schichten jeweils um `failureDrop` reduziert.

Die Werte auf der x-Achse besitzen somit abhängig von der Failure-Kombination eine unterschiedliche Bedeutung. Sie werden deshalb vor allem innerhalb derselben Failure-Kombination und desselben Datasets miteinander verglichen.

Auf der y-Achse wird jeweils eine der drei Bewertungsmetriken dargestellt. Bei der Resilienz liegt der Wertebereich zwischen 0 und 1. Ein Wert von 1 bedeutet, dass alle betrachteten Routings erfolgreich waren. Bei der durchschnittlichen Hopanzahl zeigt die y-Achse die mittlere Anzahl der durchlaufenen Kanten erfolgreicher Routings. Beim

durchschnittlichen Stretch stellt sie die mittlere Anzahl zusätzlicher Hops gegenüber dem kürzesten noch verfügbaren Pfad dar. Da die drei Metriken unterschiedliche Wertebereiche und Bedeutungen besitzen, wird für jede Metrik eine eigene Abbildung erzeugt. Die Höhen der Kurven aus unterschiedlichen Metrikplots können daher nicht direkt miteinander verglichen werden.

Für jede Kombination aus Ausfallmodell, Ausfalltyp und Metrik wird eine eigene Abbildung erzeugt. Ein Teilplot zeigt jeweils ein Dataset. Die Kurven innerhalb des Teilplots stellen die ausgewählten Routingalgorithmen dar. Bis zu drei Datasets derselben Graphkategorie werden nebeneinander in einer Abbildung angeordnet. Random-Graphen und Real-World-Graphen werden dabei getrennt dargestellt.

Die EDP-basierten Verfahren werden durch durchgezogene und die NDP-basierten Verfahren durch gestrichelte Linien gekennzeichnet. Jedem Algorithmus ist zusätzlich eine feste Farbe zugeordnet. Dadurch bleibt seine Darstellung über alle Abbildungen hinweg gleich. Die Plots zeigen die aggregierten Mittelwerte der jeweiligen Parameterkonfiguration. Konfidenzintervalle werden nicht eingezeichnet, weshalb sich die Interpretation vor allem auf wiederkehrende Trends und deutliche Unterschiede zwischen den Kurven konzentriert. Die technische Umsetzung der Ploterstellung wurde in Abschnitt 4.6.3 beschrieben.

5.2.3 Vorgehen beim Algorithmusvergleich

Der Vergleich erfolgt zunächst innerhalb der beiden Pfadgrundlagen. Für die EDP-basierten Verfahren werden EDP Paths, EDP One Tree und EDP Multiple Trees gegenübergestellt. Entsprechend werden NDP Paths, NDP One Tree und NDP Multiple Trees miteinander verglichen. Dadurch lässt sich untersuchen, welchen Einfluss die Erweiterung der disjunkten Pfade zu einer oder mehreren Baumstrukturen besitzt.

Anschließend werden die jeweils entsprechenden EDP- und NDP-Varianten verglichen. EDP Paths wird NDP Paths, EDP One Tree wird NDP One Tree und EDP Multiple Trees wird NDP Multiple Trees gegenübergestellt. Auf diese Weise kann getrennt betrachtet werden, welchen Einfluss die verwendete Disjunktheitsbedingung und welchen Einfluss die Erweiterung zu Baumstrukturen auf die Ergebnisse haben.

Die Auswertung unterscheidet außerdem zwischen Kanten- und Knotenausfällen sowie zwischen zufälligen und geclusterten Ausfällen. Zunächst werden die Ergebnisse der synthetischen Random-Graphen in Abschnitt 5.3 untersucht. Danach folgt in Abschnitt 5.4 die Auswertung der Real-World-Topologien. Die zentralen Beobachtungen aus beiden Graphkategorien werden anschließend in Abschnitt 5.5 zusammengeführt.

5.3 Ergebnisse auf Random-Graphen

Dieser Abschnitt untersucht die Ergebnisse auf den drei synthetischen Erdős-Rényi-Graphsets. Die Auswertung erfolgt getrennt für zufällige und geclusterte Kanten- sowie Knotenausfälle. Zunächst wird jeweils betrachtet, wie sich eine steigende Failure Rate auf die Resilienz auswirkt. Anschließend werden auffällige Unterschiede bei der durchschnittlichen Hopanzahl und beim durchschnittlichen Stretch berücksichtigt.

Die Werte der Failure Rate sind dabei nur innerhalb derselben Failure-Kombination und desselben Graphsets unmittelbar vergleichbar. Dies ist insbesondere deshalb wichtig, weil die drei Graphsets unterschiedliche `failFactor`-Bereiche verwenden. Abschnitt 5.3.1 behandelt zunächst zufällige Linkausfälle. Danach folgen geclusterte Linkausfälle in Abschnitt 5.3.2, zufällige Knotenausfälle in Abschnitt 5.3.3 und geclusterte Knotenausfälle in Abschnitt 5.3.4.

5.3.1 Random Link Failure

Zunächst werden die Random-Graphen unter zufälligen Kantenausfällen untersucht. Die Auswahl der Ausfälle erfolgt nach dem in Abschnitt 4.3.1 beschriebenen Modell.

Abbildung 8 zeigt die Resilienz der sechs Routingverfahren bei zufälligen Linkausfällen. Mit steigendem `failFactor` nimmt sie insgesamt ab. Dabei treten vereinzelt kleine Zwischenanstiege auf. Die Stärke des Rückgangs unterscheidet sich zwischen den Graphsets.

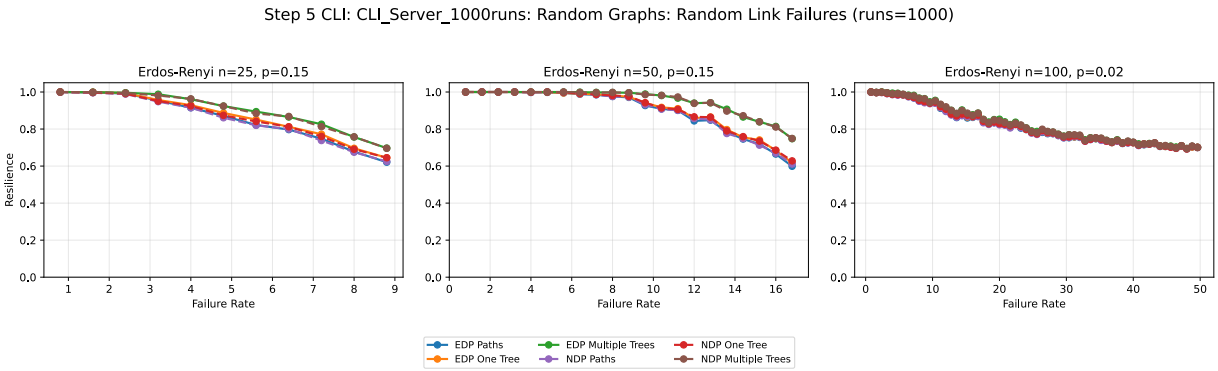


Abbildung 8: Resilienz bei zufälligen Linkausfällen auf den Random-Graphen

Beim Graphset mit $n = 25$ erreicht EDP Paths bei der höchsten betrachteten Failure Rate von $8{,}8$ eine Resilienz von $0{,}621$. EDP One Tree erreicht $0{,}645$, während EDP Multiple Trees mit $0{,}697$ den höchsten Wert der drei EDP-basierten Verfahren besitzt. Die entsprechenden NDP-Verfahren erreichen mit $0{,}625$, $0{,}645$ und $0{,}696$ nahezu dieselben Werte.

Beim Graphset mit $n = 50$ ist der Vorteil von Multiple Trees deutlicher. Bei der höchsten Failure Rate von $16{,}8$ steigt die Resilienz von $0{,}599$ bei EDP Paths auf $0{,}748$ bei EDP Multiple Trees. NDP Paths erreicht $0{,}611$, während NDP Multiple Trees $0{,}749$ erreicht. Die Erweiterung aller disjunkten Pfade zu mehreren Bäumen verbessert die Resilienz hier somit um ungefähr $0{,}14$ bis $0{,}15$, also um 14 bis 15 Prozentpunkte.

Beim dünneren Graphset mit $n = 100$ und $p = 0{,}02$ liegen die sechs Kurven dagegen eng beieinander. Bei der höchsten Failure Rate erreichen sie Werte zwischen $0{,}700$ und $0{,}701$. Der größte Vorteil von Multiple Trees gegenüber dem jeweiligen Pfadverfahren tritt bereits bei einer niedrigeren Failure Rate auf und beträgt weniger als drei Prozentpunkte. Dieses Verhalten steht im Zusammenhang mit der geringen lokalen Konnektivität des dünnen Graphsets. Da die Anzahl der zufälligen Linkausfälle aus der Anzahl der EDPs abgeleitet wird, kann ein hoher `failFactor` bei wenigen vorhandenen EDPs nicht dieselbe Wirkung wie bei einem stärker verbundenen Graphen entfalten.

Die höhere Resilienz der Multiple-Trees-Verfahren ist teilweise mit längeren Routen verbunden. Abbildung 9 zeigt die durchschnittliche Hopanzahl der erfolgreichen Routings.

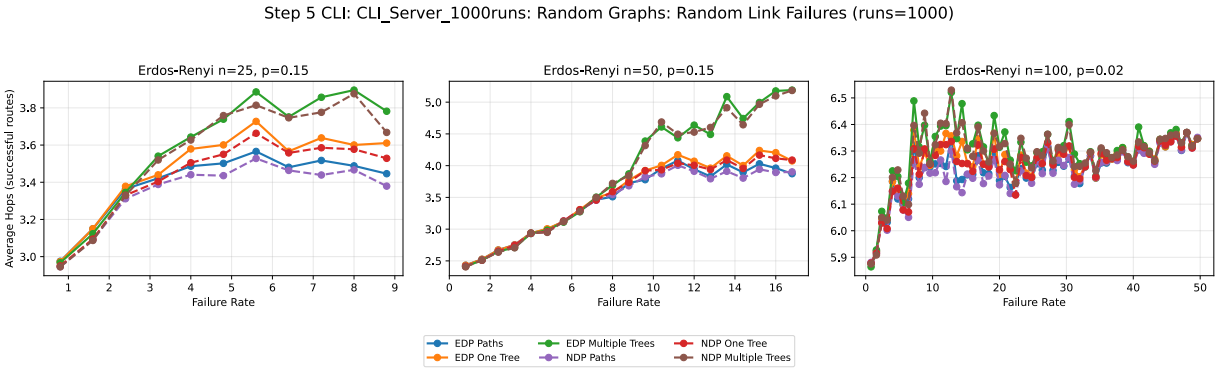


Abbildung 9: Durchschnittliche Hopanzahl bei zufälligen Linkausfällen

Beim Graphset mit $n = 50$ benötigt EDP Paths bei der höchsten Failure Rate durchschnittlich $3{,}873$ Hops, während EDP Multiple Trees $5{,}190$ Hops benötigt. Für NDP Paths und NDP Multiple Trees ergeben sich mit $3{,}902$ beziehungsweise $5{,}184$ vergleichbare Werte. Multiple Trees kann folglich zusätzliche Versuchsszenen erfolgreich routen, benötigt dafür aber teilweise längere Ausweichrouten. One Tree liegt sowohl bei der Resilienz als auch bei der Hopanzahl meist zwischen dem reinen Pfadverfahren und Multiple Trees.

Die EDP- und NDP-basierten Varianten zeigen bei zufälligen Linkausfällen insgesamt nur geringe Unterschiede. Der deutlichere Einfluss geht von der Erweiterung zu mehreren Bäumen aus.

5.3.2 Clustered Link Failure

Für die Untersuchung geclusterter Kantenausfälle auf den Random-Graphen wird das in Abschnitt 4.3.2 beschriebene Modell verwendet. Die Versuchsparameter entsprechen den Festlegungen in Abschnitt 5.1.

Abbildung 10 zeigt, dass die Resilienz bei allen drei Graphsets mit zunehmendem `failureStart` abnimmt. Bei niedrigen Werten zwischen $0{,}3$ und $0{,}4$ erreichen die sechs Verfahren noch eine Resilienz nahe 1. Ab mittleren Failure Rates werden deutlichere Unterschiede zwischen den Pfad- und Baumverfahren sichtbar.

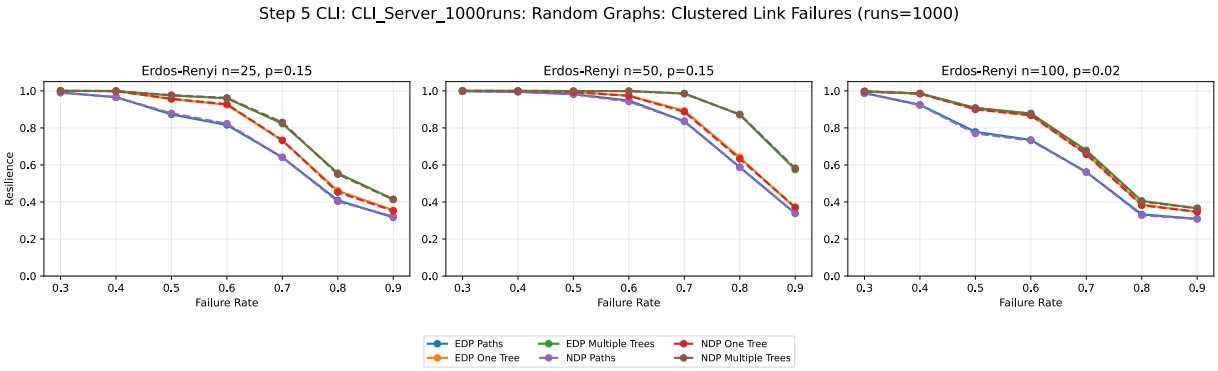


Abbildung 10: Resilienz bei geclusterten Linkausfällen auf den Random-Graphen

Beim Graphset mit $n = 25$ erreicht EDP Paths bei einer Failure Rate von $0{,}9$ eine Resilienz von $0{,}317$. EDP One Tree erreicht $0{,}355$ und EDP Multiple Trees $0{,}415$. Die entsprechenden NDP-Verfahren liegen mit $0{,}322$, $0{,}352$ und $0{,}413$ auf einem ähnlichen Niveau. Der größte Vorteil von Multiple Trees gegenüber dem jeweiligen Pfadverfahren tritt bei einer Failure Rate von $0{,}7$ auf und beträgt ungefähr 18 bis 19 Prozentpunkte.

Besonders deutlich ist der Unterschied beim Graphset mit $n = 50$. Bei einer Failure Rate von $0{,}9$ erreicht EDP Paths eine Resilienz von $0{,}338$, während EDP Multiple Trees $0{,}575$ erreicht. Bei den NDP-basierten Verfahren steigt der Wert von $0{,}340$ bei NDP Paths auf $0{,}582$ bei NDP Multiple Trees. Bei einer Failure Rate von $0{,}8$ beträgt der Vorteil der Multiple-Trees-Verfahren gegenüber den zugehörigen Pfadverfahren sogar ungefähr 28 bis 29 Prozentpunkte.

Auch beim dünneren Graphset mit $n = 100$ verbessert die Erweiterung zu Baumstrukturen die Resilienz. Bei einer Failure Rate von $0{,}9$ erreicht EDP Paths $0{,}309$, EDP One Tree $0{,}346$ und EDP Multiple Trees $0{,}366$. Die entsprechenden NDP-Werte betragen $0{,}308$, $0{,}347$ und $0{,}366$. Der Vorteil von Multiple Trees ist damit vorhanden, fällt aber geringer als beim Graphset mit $n = 50$ aus.

Die höhere Resilienz zeigt, dass zusätzliche Baumäste bei den um den Zielknoten erzeugten Kantenausfällen alternative Weiterleitungen ermöglichen können. Die Multiple-Trees-Verfahren besitzen dabei den größten Vorteil, da nicht nur ein einzelner, sondern jeder

disjunkte Pfad zu einer Baumstruktur erweitert wird. One Tree verbessert die Resilienz ebenfalls, bleibt jedoch meist zwischen dem Pfadverfahren und Multiple Trees.

Dieser Resilienzgewinn ist mit einer höheren durchschnittlichen Hopanzahl verbunden. Abbildung 11 zeigt, dass die Unterschiede vor allem bei mittleren und hohen Failure Rates zunehmen.

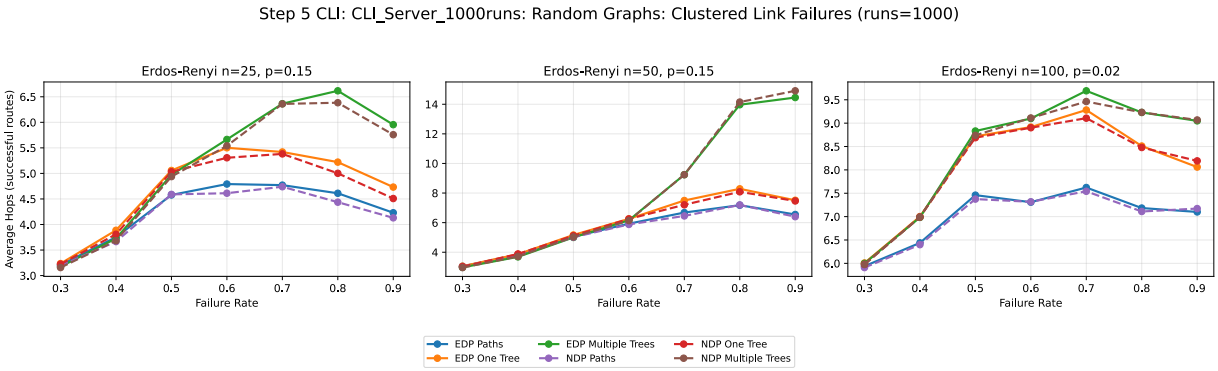


Abbildung 11: Durchschnittliche Hopanzahl bei geclusterten Linkausfällen

Beim Graphset mit $n = 50$ benötigt EDP Paths bei einer Failure Rate von $0{,}9$ durchschnittlich $6{,}550$ Hops. EDP One Tree benötigt $7{,}511$ Hops und EDP Multiple Trees $14{,}454$ Hops. Für NDP Paths, NDP One Tree und NDP Multiple Trees ergeben sich mit $6{,}403$, $7{,}465$ und $14{,}909$ vergleichbare Werte. Multiple Trees kann somit deutlich mehr Versuchsszenen erfolgreich routen, muss dabei aber häufiger längere Äste und Rückwege verwenden.

Die höheren Hop-Mittelwerte sind mit zusätzlichen Ausweichwegen vereinbar.

Der Average Stretch bestätigt diesen Zielkonflikt. Beim Graphset mit $n = 50$ steigt er bei einer Failure Rate von $0{,}9$ beispielsweise von $3{,}846$ bei EDP Paths auf $11{,}290$ bei EDP Multiple Trees.

Zwischen den jeweils entsprechenden EDP- und NDP-Varianten bestehen auch bei geclusterten Linkausfällen nur geringe Unterschiede. Der deutlichste Einfluss geht erneut von der Erweiterung der Pfade zu mehreren Baumstrukturen aus.

5.3.3 Random Node Failure

Im Folgenden werden die sechs Routingverfahren bei zufälligen Knotenausfällen auf den Random-Graphen verglichen. Die Auswahl und Modellierung der Knotenausfälle wurde in Abschnitt 4.3.3 erläutert.

Abbildung 12 zeigt die Resilienz der sechs Routingverfahren. Wie bei den zufälligen Linkausfällen nimmt sie bei allen Graphsets mit zunehmender Failure Rate insgesamt ab. Die Unterschiede zwischen den Verfahren fallen insgesamt jedoch geringer aus als bei den geclusterten Linkausfällen.

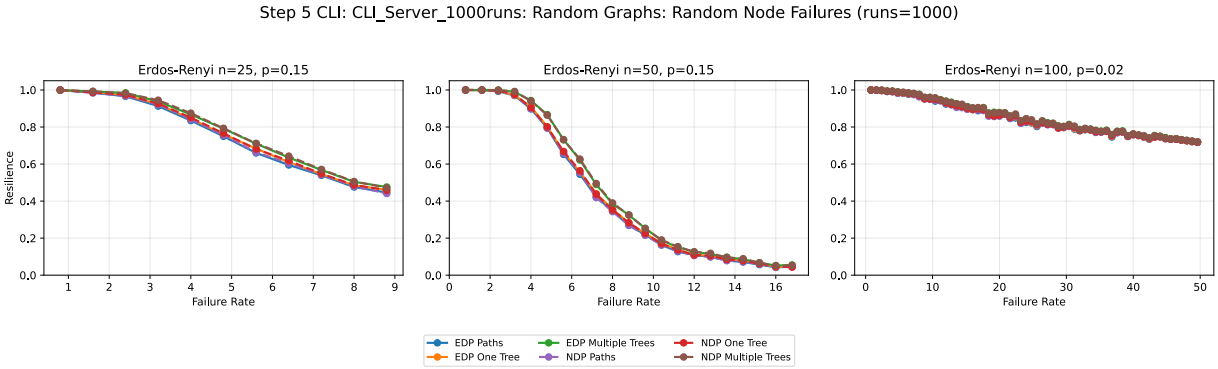


Abbildung 12: Resilienz bei zufälligen Knotenausfällen auf den Random-Graphen

Beim Graphset mit $n = 25$ erreicht EDP Paths bei der höchsten Failure Rate von $8{,}8$ eine Resilienz von $0{,}446$. EDP One Tree erreicht $0{,}461$ und EDP Multiple Trees $0{,}476$. Die entsprechenden NDP-Verfahren erreichen $0{,}441$, $0{,}459$ und $0{,}472$. Multiple Trees besitzt damit zwar die höchste Resilienz, der Abstand zum zugehörigen Pfadverfahren beträgt an diesem Punkt jedoch nur ungefähr drei Prozentpunkte. Der größte Vorteil tritt bei einer mittleren Failure Rate auf und beträgt ungefähr fünf Prozentpunkte.

Beim Graphset mit $n = 50$ fällt die Resilienz bei hohen Failure Rates besonders stark ab. Bei einer Failure Rate von $16{,}8$ erreicht EDP Paths nur noch $0{,}046$, EDP One Tree ebenfalls $0{,}046$ und EDP Multiple Trees $0{,}055$. Die NDP-basierten Varianten erreichen $0{,}044$, $0{,}044$ und $0{,}053$. Unter dieser hohen Ausfallbelastung sind damit nur noch ungefähr vier bis sechs Prozent der Versuchsszenen erfolgreich. Bei mittleren Failure Rates kann Multiple Trees die Resilienz um bis zu ungefähr acht Prozentpunkte erhöhen. Dieser Vorteil nimmt jedoch wieder ab, wenn durch die Knotenausfälle nur noch wenige nutzbare Verbindungen zum Ziel vorhanden sind.

Beim dünneren Graphset mit $n = 100$ und $p = 0{,}02$ liegen die sechs Kurven erneut eng beieinander. Bei der höchsten Failure Rate erreichen sie Werte zwischen $0{,}717$ und $0{,}720$. Der größte Vorteil von Multiple Trees gegenüber dem jeweiligen Pfadverfahren beträgt über den gesamten Parameterbereich ungefähr zwei Prozentpunkte. Wie bei Random Link Failure steht dieses Verhalten im Zusammenhang mit der geringen lokalen Konnektivität. Wenn zwischen Start und Ziel nur ein NDP vorhanden ist, ergibt die verwendete Berechnung unabhängig von einem hohen `failFactor` keinen zusätzlichen Knotenausfall.

Entgegen einer möglichen Erwartung zeigen die NDP-basierten Verfahren bei den zufälligen Knotenausfällen keinen durchgängigen Vorteil gegenüber den entsprechenden EDP-Verfahren. Die Abweichungen liegen bei den untersuchten Graphsets überwiegend im Bereich weniger Prozentpunkte und wechseln teilweise ihre Richtung. Die stärkere Knotendisjunktheit verhindert zwar, dass mehrere NDPs denselben inneren Knoten verwenden, kann jedoch gleichzeitig die Anzahl der verfügbaren Ausgangspfade begren-

zen. In den untersuchten Szenen ergibt sich daraus kein systematischer Resilienzvorteil gegenüber EDP.

Neben der Resilienz ist für die Bewertung der NDP-basierten Verfahren entscheidend, wie viele Routingschritte ihre erfolgreichen Routen benötigen. Abbildung 13 zeigt deshalb die durchschnittliche Hopanzahl bei zufälligen Knotenausfällen.

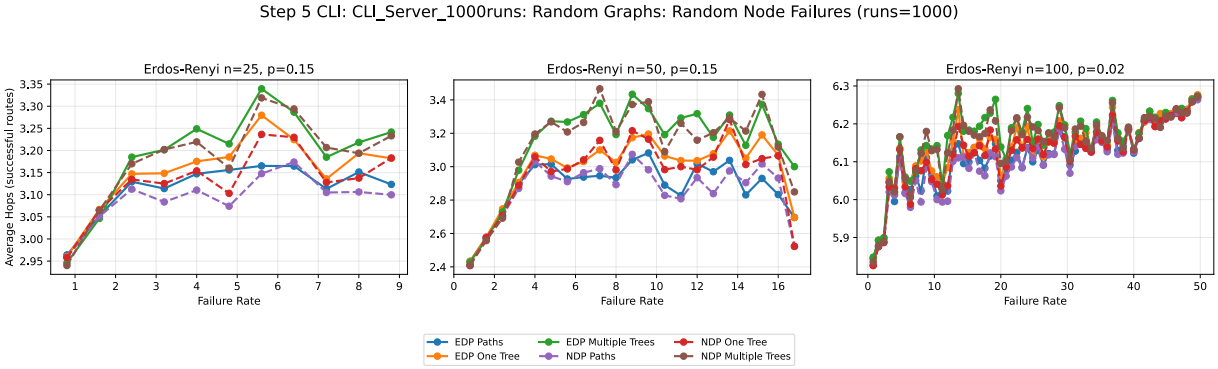


Abbildung 13: Durchschnittliche Hopanzahl bei zufälligen Knotenausfällen

Beim Graphset mit $n = 25$ liegen die durchschnittlichen Hopanzahlen der NDP-basierten Verfahren bei der höchsten Failure Rate zwischen $3{,}100$ und $3{,}233$. NDP Paths benötigt $3{,}100$ Hops, NDP One Tree $3{,}183$ Hops und NDP Multiple Trees $3{,}233$ Hops. Die zugehörigen EDP-Verfahren erreichen mit $3{,}123$, $3{,}182$ und $3{,}242$ vergleichbare Werte. Der zusätzliche Resilienzgewinn von Multiple Trees ist somit mit einer leicht höheren durchschnittlichen Hopanzahl verbunden.

Beim Graphset mit $n = 50$ schwanken die Hop-Mittelwerte bei hohen Failure Rates stärker. Bei der höchsten Failure Rate benötigt NDP Paths durchschnittlich $2{,}523$ Hops, NDP One Tree ebenfalls $2{,}523$ Hops und NDP Multiple Trees $2{,}849$ Hops. Diese Werte beruhen jedoch nur auf 44, 44 beziehungsweise 53 erfolgreichen Routings. Sie dürfen daher nicht unabhängig von der sehr niedrigen Resilienz interpretiert werden. Die scheinbar niedrige Hopanzahl bedeutet hier nicht, dass das Routing insgesamt leistungsfähiger ist, sondern beschreibt nur die wenigen noch erfolgreichen Versuchsszenen.

Beim Graphset mit $n = 100$ liegen die Kurven erneut eng beieinander. Bei der höchsten Failure Rate ergeben sich für NDP Paths, NDP One Tree und NDP Multiple Trees durchschnittlich $6{,}264$, $6{,}272$ und $6{,}271$ Hops. Die entsprechenden EDP-Werte liegen zwischen $6{,}269$ und $6{,}277$. Die Erweiterung der NDPs zu Baumstrukturen verursacht in diesem Graphset somit keine deutliche zusätzliche Hopbelastung, erhöht die Resilienz aber ebenfalls nur geringfügig.

Insgesamt zeigt die Hop-Auswertung, dass NDP Paths häufig die kürzesten erfolgreichen Routen der drei NDP-Varianten besitzt. NDP One Tree bleibt meist nahe am Pfadverfahren, während NDP Multiple Trees bei einzelnen Failure Rates mehr Hops benötigt. Dieser zusätzliche Aufwand entsteht durch die Nutzung alternativer Baumä-

ste und möglicher Rückwege. Gleichzeitig ermöglichen diese Strukturen zusätzliche erfolgreiche Routings.

Der Average Stretch zeigt ein ähnliches Bild. Bei der höchsten Failure Rate bleibt er in allen drei Graphsets vergleichsweise niedrig.

5.3.4 Clustered Node Failure

Abschließend wird das Verhalten auf den Random-Graphen bei geclusterten Knotenausfällen untersucht. Hierfür wird das in Abschnitt 4.3.4 beschriebene Modell mit den in Abschnitt 5.1 festgelegten Parametern verwendet.

Abbildung 14 zeigt, dass die Resilienz bei allen drei Graphsets mit zunehmender Failure Rate deutlich abnimmt. Bei niedrigen Werten liegen die sechs Verfahren noch eng beieinander. Ab mittleren Failure Rates werden die Vorteile der Baumstrukturen sichtbar.

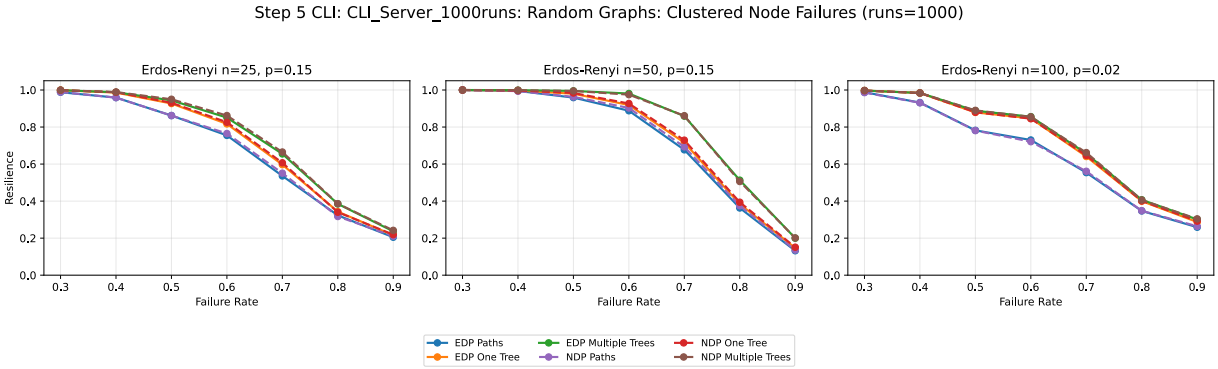


Abbildung 14: Resilienz bei geclusterten Knotenausfällen auf den Random-Graphen

Beim Graphset mit $n = 25$ erreicht NDP Paths bei einer Failure Rate von $0{,}7$ eine Resilienz von $0{,}551$. NDP One Tree erhöht diesen Wert auf $0{,}607$, während NDP Multiple Trees $0{,}665$ erreicht. Die Erweiterung aller NDPs zu mehreren Bäumen verbessert die Resilienz gegenüber NDP Paths damit um $0{,}114$, also um 11,4 Prozentpunkte. Die EDP-basierten Verfahren zeigen mit $0{,}536$, $0{,}597$ und $0{,}655$ einen ähnlichen Verlauf.

Beim Graphset mit $n = 50$ ist der Vorteil von NDP Multiple Trees besonders deutlich. Bei einer Failure Rate von $0{,}7$ erreicht NDP Paths eine Resilienz von $0{,}693$, NDP One Tree $0{,}729$ und NDP Multiple Trees $0{,}861$. Der Vorteil von NDP Multiple Trees gegenüber NDP Paths beträgt damit 16,8 Prozentpunkte. Bei 1000 Wiederholungen entspricht dies 168 zusätzlichen erfolgreichen Routings. Die entsprechenden EDP-Verfahren erreichen $0{,}677$, $0{,}716$ und $0{,}859$.

Auch beim dünneren Graphset mit $n = 100$ verbessert die Erweiterung der NDPs die Resilienz. Bei einer Failure Rate von $0{,}7$ erreicht NDP Paths $0{,}561$, NDP One Tree $0{,}647$ und NDP Multiple Trees $0{,}662$. NDP One Tree erzielt hier bereits einen großen Teil des

durch die Baumerweiterung möglichen Resilienzgewinns. Der zusätzliche Vorteil von Multiple Trees gegenüber One Tree fällt mit 1,5 Prozentpunkten geringer aus.

Bei der höchsten Failure Rate von $0{,}9$ sinkt die Resilienz in allen Graphsets weiter. NDP Multiple Trees erreicht für $n = 25$, $n = 50$ und $n = 100$ die Werte $0{,}242$, $0{,}201$ und $0{,}304$. NDP Paths erreicht dagegen $0{,}211$, $0{,}136$ und $0{,}264$. Multiple Trees bleibt somit auch bei starker Ausfallbelastung die resilienteste NDP-basierte Variante.

Abbildung 15 zeigt die durchschnittliche Hopanzahl der erfolgreichen Routings. Die zusätzlichen Baumstrukturen erhöhen nicht nur die Resilienz, sondern teilweise auch die Länge der ausgeführten Routen.

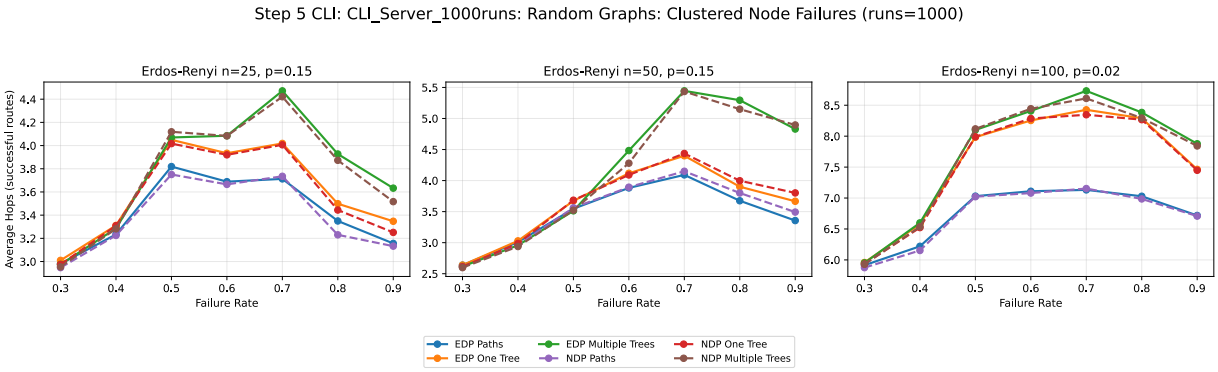


Abbildung 15: Durchschnittliche Hopanzahl bei geclusterten Knotenausfällen

Bei einer Failure Rate von $0{,}7$ benötigt NDP Paths beim Graphset mit $n = 25$ durchschnittlich $3{,}735$ Hops. NDP One Tree benötigt $4{,}007$ und NDP Multiple Trees $4{,}421$ Hops. Beim Graphset mit $n = 50$ steigen die Werte von $4{,}150$ über $4{,}436$ auf $5{,}434$ Hops. Für $n = 100$ ergeben sich $7{,}153$, $8{,}346$ und $8{,}610$ Hops.

NDP Multiple Trees benötigt somit in allen drei Graphsets die meisten Hops, erreicht gleichzeitig aber die höchste Resilienz. NDP One Tree bildet meist einen Zwischenwert: Die Resilienz liegt über NDP Paths, während die Hopanzahl geringer als bei NDP Multiple Trees bleibt. One Tree stellt damit in einigen Szenen einen Kompromiss zwischen Routingerfolg und Routenlänge dar.

Nach einer Failure Rate von ungefähr $0{,}7$ nehmen einzelne Hop-Mittelwerte wieder ab.

Der Average Stretch bestätigt den zusätzlichen Routingaufwand der Baumverfahren. Bei einer Failure Rate von $0{,}7$ steigt er beispielsweise beim Graphset mit $n = 50$ von $1{,}352$ bei NDP Paths auf $1{,}588$ bei NDP One Tree und $2{,}439$ bei NDP Multiple Trees.

Insgesamt erzielt NDP Multiple Trees bei geclusterten Knotenausfällen die höchste Resilienz der drei NDP-basierten Verfahren. Dieser Vorteil ist besonders beim mittleren Graphset sichtbar, geht jedoch mit einer höheren Hopanzahl und einem größeren Stretch einher. Die Unterschiede zwischen den entsprechenden EDP- und NDP-Varianten bleiben dagegen überwiegend klein.

5.4 Ergebnisse auf Real-World-Graphen

Dieser Abschnitt untersucht die sechs Routingverfahren auf den Real-World-Topologien ATT-MPLS, Oteglobe und Cogentco. Im Gegensatz zu den synthetischen Graphen bleibt die jeweilige Topologie in allen Wiederholungen unverändert. Start- und Zielknoten sowie die Ausfallmengen werden weiterhin für jede Versuchsszene neu und reproduzierbar bestimmt.

Für die zufälligen Ausfälle verwenden alle drei Topologien `failFactor`-Werte von 1 bis 10. Da die daraus berechnete Ausfallanzahl von der Anzahl der disjunkten Pfade abhängt, entspricht derselbe Wert nicht zwingend derselben absoluten Anzahl ausgefallener Kanten oder Knoten. Die Ergebnisse werden deshalb zunächst innerhalb jeder Topologie verglichen.

Abschnitt 5.4.1 behandelt zufällige Linkausfälle. Danach folgen geclusterte Linkausfälle in Abschnitt 5.4.2, zufällige Knotenausfälle in Abschnitt 5.4.3 und geclusterte Knotenausfälle in Abschnitt 5.4.4.

5.4.1 Random Link Failure

Die Untersuchung der Real-World-Topologien beginnt mit zufälligen Kantenausfällen nach dem Modell aus Abschnitt 4.3.1.

Abbildung 16 zeigt die Resilienz bei zufälligen Linkausfällen. Mit steigendem `failFactor` nimmt die Resilienz bei allen drei Topologien insgesamt ab. Der Rückgang fällt bei ATT-MPLS deutlich stärker als bei Oteglobe und Cogentco aus.

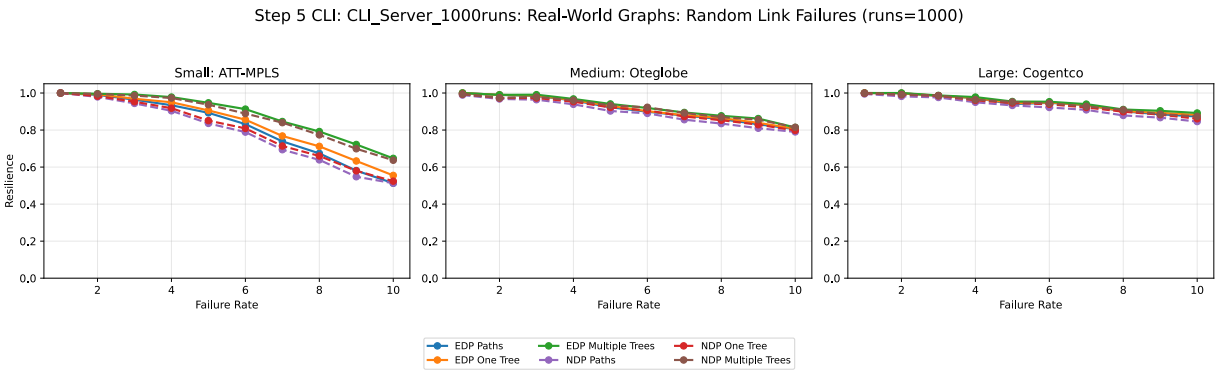


Abbildung 16: Resilienz bei zufälligen Linkausfällen auf den Real-World-Graphen

Bei ATT-MPLS erreicht EDP Paths bei der höchsten Failure Rate von 10 eine Resilienz von 0,513. EDP One Tree erhöht diesen Wert auf 0,555 und EDP Multiple Trees auf 0,647. NDP Paths erreicht ebenfalls 0,513, NDP One Tree 0,524 und NDP Multiple Trees 0,637. Multiple Trees verbessert die Resilienz gegenüber dem jeweiligen Pfadverfahren damit um ungefähr 12 bis 13 Prozentpunkte.

Bei Oteglobe liegen die sechs Verfahren enger beieinander. Bei einer Failure Rate von 10 erreichen EDP Paths, EDP One Tree und EDP Multiple Trees die Werte 0,805, 0,807 und 0,814. Für die NDP-basierten Varianten ergeben sich 0,790, 0,798 und 0,815. Der Vorteil der Baumstrukturen ist hier kleiner als bei ATT-MPLS. NDP Multiple Trees erreicht dennoch die höchste Resilienz der sechs Verfahren.

Bei der höchsten betrachteten Failure Rate von 10 erreichen die Verfahren auf Cogentco jeweils höhere Resilienzwerte als ihre entsprechenden Varianten auf ATT-MPLS und Oteglobe. Dabei erreicht EDP Paths 0,873, EDP One Tree 0,882 und EDP Multiple Trees 0,892. NDP Paths erreicht 0,846, NDP One Tree 0,861 und NDP Multiple Trees 0,874. Die NDP-basierten Verfahren liegen in dieser Topologie somit leicht unter den entsprechenden EDP-Verfahren.

Die Ergebnisse zeigen, dass Multiple Trees innerhalb beider Pfadgrundlagen meist die höchste Resilienz erreicht. Die Stärke des Vorteils hängt jedoch deutlich von der Topologie ab. Bei ATT-MPLS stellen die zusätzlichen Baumstrukturen mehr erfolgreiche Ausweichmöglichkeiten bereit. Bei Oteglobe und Cogentco sind die Unterschiede geringer, da bereits die Pfadverfahren einen großen Anteil der Szenen erfolgreich routen.

Abbildung 17 zeigt die durchschnittliche Hopanzahl der erfolgreichen Routings. Aufgrund der unterschiedlichen Größe der Topologien liegen die absoluten Werte bei Cogentco erwartungsgemäß höher als bei ATT-MPLS und Oteglobe.

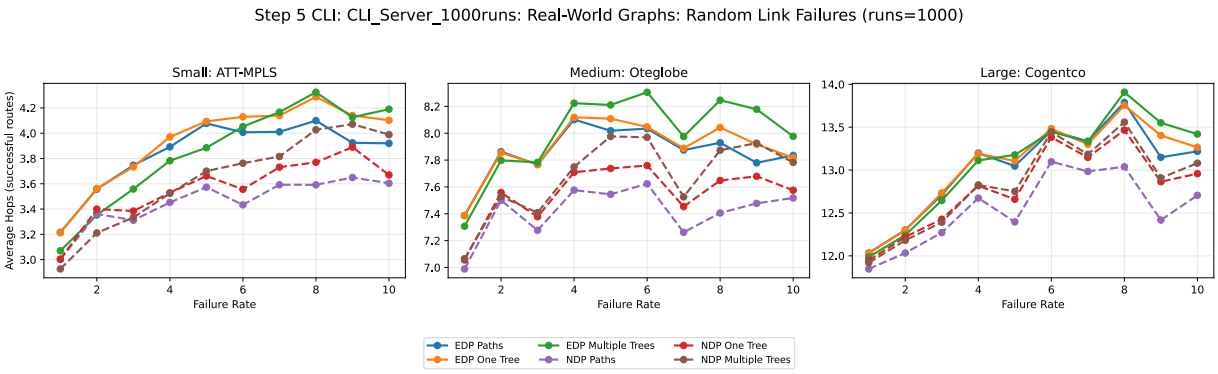


Abbildung 17: Durchschnittliche Hopanzahl bei zufälligen Linkausfällen

Bei ATT-MPLS benötigt NDP Paths bei der höchsten Failure Rate durchschnittlich 3,604 Hops. NDP One Tree benötigt 3,670 und NDP Multiple Trees 3,989 Hops. Die entsprechenden EDP-Werte betragen 3,920, 4,103 und 4,190. Die NDP-basierten Verfahren benötigen bei erfolgreichen Routings damit im Mittel weniger Hops.

Ein ähnliches Bild zeigt sich bei Oteglobe. Dort ergeben sich für NDP Paths, NDP One Tree und NDP Multiple Trees bei einer Failure Rate von 10 durchschnittlich 7,518, 7,575 und 7,783 Hops. Bei Cogentco liegen die entsprechenden Werte bei 12,706, 12,958 und 13,080 Hops. Innerhalb der NDP-basierten Verfahren besitzt NDP Paths damit meist die

niedrigste Hopanzahl, während NDP Multiple Trees für seine zusätzlichen erfolgreichen Routings mehr Routingaufwand benötigt.

Bei der betrachteten Failure Rate von 10 sind auf Cogentco die NDP-Verfahren weniger häufig erfolgreich als ihre EDP-Gegenstücke. Für den Vergleich ihrer Hop-Mittelwerte gilt daher die in Abschnitt 5.2.1 erläuterte Einschränkung.

Auch der Average Stretch fällt bei den NDP-basierten Verfahren überwiegend etwas geringer aus. Bei Cogentco beträgt er bei der höchsten Failure Rate beispielsweise 1,917 für NDP Paths und 2,224 für NDP Multiple Trees, gegenüber 2,234 für EDP Paths und 2,435 für EDP Multiple Trees.

5.4.2 Clustered Link Failure

Anschließend werden geclusterte Kantenausfälle auf den Real-World-Topologien betrachtet. Das zugrunde liegende Ausfallmodell ist in Abschnitt 4.3.2 beschrieben.

Abbildung 18 zeigt, dass die Resilienz bei allen drei Real-World-Topologien deutlich abnimmt.

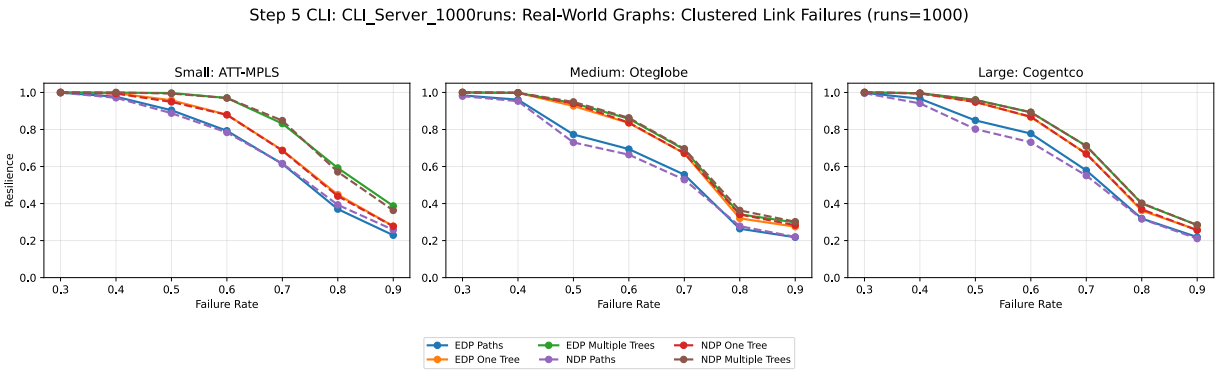


Abbildung 18: Resilienz bei geclusterten Linkausfällen auf den Real-World-Graphen

Bei einer Failure Rate von 0,3 erreichen nahezu alle Verfahren eine Resilienz nahe 1. Deutlichere Unterschiede treten ab mittleren Failure Rates auf. Bei ATT-MPLS erreicht EDP Paths bei einer Failure Rate von 0,7 eine Resilienz von 0,614, EDP One Tree 0,688 und EDP Multiple Trees 0,832. Die NDP-basierten Varianten erreichen 0,616, 0,686 und 0,848. NDP Multiple Trees verbessert die Resilienz gegenüber NDP Paths damit um 23,2 Prozentpunkte.

Bei Oteglobe erreicht NDP Paths bei derselben Failure Rate eine Resilienz von 0,529. NDP One Tree erhöht diesen Wert deutlich auf 0,672, NDP Multiple Trees erreicht 0,696. Der überwiegende Resilienzgewinn entsteht hier bereits durch die Erweiterung eines NDP zu einem Baum. Die zusätzlichen Bäume von NDP Multiple Trees verbessern das Ergebnis nur noch um 2,4 Prozentpunkte. Die EDP-basierten Varianten zeigen mit 0,556, 0,671 und 0,692 einen ähnlichen Verlauf.

Bei Cogentco erreicht NDP Paths bei einer Failure Rate von 0,7 eine Resilienz von 0,551. NDP One Tree erreicht 0,668 und NDP Multiple Trees 0,711. Die entsprechenden EDP-Werte betragen 0,579, 0,670 und 0,711. Auch in dieser Topologie erhöhen die Baumstrukturen die Resilienz deutlich, während die Unterschiede zwischen den entsprechenden EDP- und NDP-Varianten klein bleiben.

Bei der höchsten Failure Rate von 0,9 sinkt die Resilienz aller Verfahren weiter. NDP Multiple Trees erreicht für ATT-MPLS, Oteglobe und Cogentco die Werte 0,363, 0,302 und 0,284. NDP Paths erreicht dagegen 0,258, 0,220 und 0,211. Multiple Trees bleibt damit auch unter hoher Ausfallbelastung die resilienteste oder eine der resilientesten NDP-basierten Varianten.

Abbildung 19 zeigt die durchschnittliche Hopanzahl. Mit steigender Failure Rate nimmt der Routingaufwand zunächst deutlich zu. Die Baumverfahren benötigen mehr Hops, können damit aber zusätzliche Szenen erfolgreich routen.

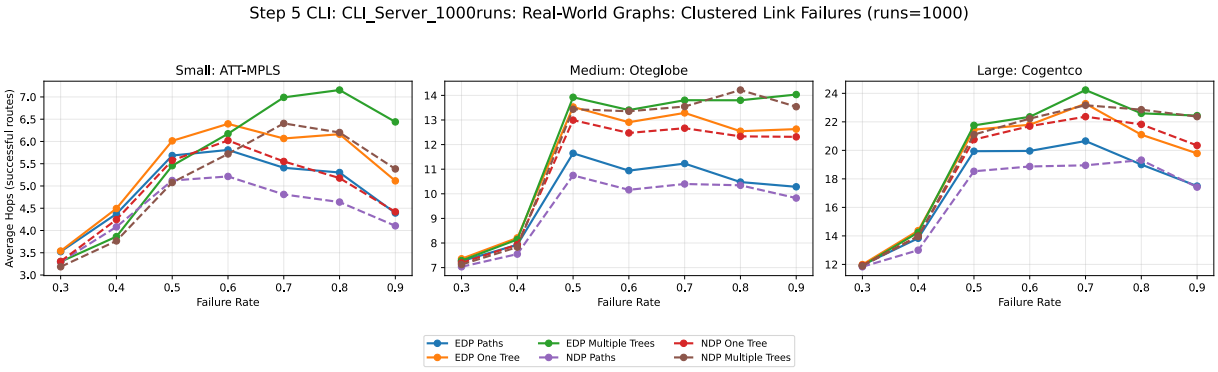


Abbildung 19: Durchschnittliche Hopanzahl bei geclusterten Linkausfällen

Bei ATT-MPLS benötigt NDP Paths bei einer Failure Rate von 0,7 durchschnittlich 4,810 Hops. NDP One Tree benötigt 5,551 und NDP Multiple Trees 6,406 Hops. Bei Oteglobe steigen die entsprechenden Werte von 10,399 über 12,667 auf 13,546 Hops. Für Cogentco ergeben sich 18,951, 22,362 und 23,165 Hops.

Die NDP-basierten Verfahren benötigen bei erfolgreichen Routings häufig weniger Hops als die entsprechenden EDP-Varianten. Bei Cogentco ergeben sich bei einer Failure Rate von 0,7 beispielsweise 20,655 Hops für EDP Paths und 18,951 Hops für NDP Paths. Bei Multiple Trees beträgt der Unterschied 24,232 gegenüber 23,165 Hops. Eine mögliche Ursache ist, dass die stärkere Knotendisjunktheit der NDPs zu anders aufgebauten und teilweise kürzeren erfolgreichen Routingstrukturen führt.

Auch hier müssen die Hop-Mittelwerte gemeinsam mit der Resilienz betrachtet werden.

Der Average Stretch bestätigt den erhöhten Routingaufwand der Baumverfahren. Bei Cogentco steigt er bei einer Failure Rate von 0,7 von 7,214 bei NDP Paths auf 9,825 bei NDP One Tree und 10,689 bei NDP Multiple Trees.

Insgesamt erhöhen One Tree und Multiple Trees die Resilienz bei geclusterten Linkausfällen deutlich. Multiple Trees erreicht meist den höchsten Wert, benötigt dafür jedoch mehr Hops und weist einen größeren Stretch auf. Zwischen den entsprechenden EDP- und NDP-Varianten besteht dagegen kein über alle Topologien einheitlicher Resilienzvorteil.

5.4.3 Random Node Failure

Für den Vergleich bei zufälligen Knotenausfällen wird das in Abschnitt 4.3.3 erläuterte Modell auf ATT-MPLS, Oteglobe und Cogentco angewendet.

Abbildung 20 zeigt die Resilienz der sechs Verfahren. Bei ATT-MPLS nimmt sie mit steigender Failure Rate deutlich ab. Oteglobe und Cogentco behalten dagegen über den untersuchten Parameterbereich vergleichsweise hohe Werte.

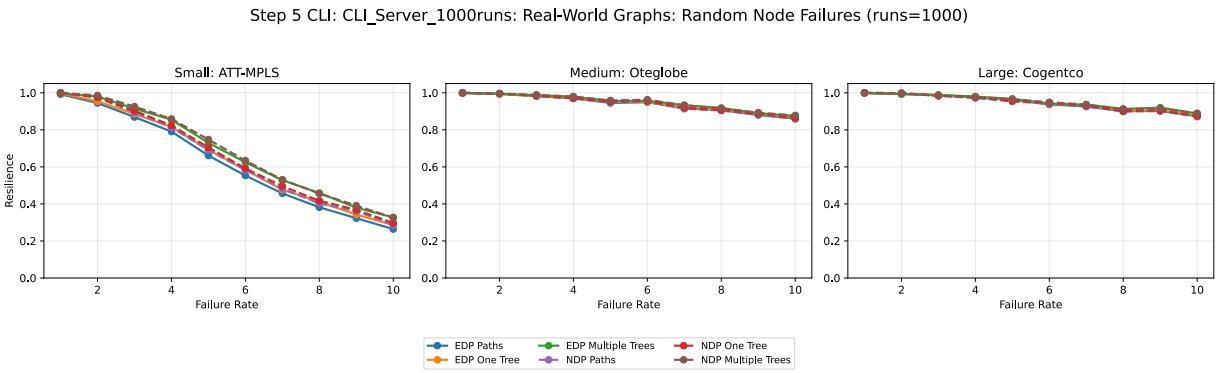


Abbildung 20: Resilienz bei zufälligen Knotenausfällen auf den Real-World-Graphen

Bei ATT-MPLS zeigen die NDP-basierten Verfahren teilweise einen leichten Vorteil gegenüber den entsprechenden EDP-Varianten. Bei einer Failure Rate von 5 erreicht EDP Paths eine Resilienz von $0{,}661$, während NDP Paths $0{,}690$ erreicht. EDP One Tree und NDP One Tree erreichen $0{,}693$ beziehungsweise $0{,}705$. Bei Multiple Trees steigen die Werte auf $0{,}730$ für EDP und $0{,}748$ für NDP.

Bei der höchsten Failure Rate von 10 erreicht NDP Paths bei ATT-MPLS eine Resilienz von $0{,}287$, NDP One Tree $0{,}297$ und NDP Multiple Trees $0{,}326$. Die entsprechenden EDP-Werte betragen $0{,}264$, $0{,}286$ und $0{,}327$. NDP Paths und NDP One Tree liegen damit leicht über ihren EDP-Gegenstücken. Bei Multiple Trees sind die Ergebnisse dagegen nahezu identisch.

Bei Oteglobe liegen alle sechs Verfahren eng beieinander. Bei einer Failure Rate von 10 erreichen NDP Paths, NDP One Tree und NDP Multiple Trees die Werte $0{,}859$, $0{,}864$ und $0{,}878$. Die EDP-basierten Varianten erreichen $0{,}861$, $0{,}865$ und $0{,}875$. NDP Multiple Trees besitzt damit die höchste Resilienz, der Abstand zu den übrigen Verfahren bleibt jedoch gering.

Ein ähnliches Bild zeigt sich bei Cogentco. Bei der höchsten Failure Rate erreichen NDP Paths 0,871, NDP One Tree 0,875 und NDP Multiple Trees 0,889. Die entsprechenden EDP-Werte betragen 0,874, 0,879 und 0,889. Die Wahl zwischen EDP und NDP hat in dieser Topologie somit nur einen kleinen Einfluss auf die Resilienz.

Die Ergebnisse zeigen keinen über alle Topologien einheitlichen Resilienzvorteil der NDP-basierten Verfahren. Bei ATT-MPLS erreichen NDP Paths und NDP One Tree teilweise höhere Werte als ihre EDP-Gegenstücke. Bei Oteglobe und Cogentco sind die Unterschiede dagegen sehr klein. Der Einfluss der Knotendisjunktheit hängt damit von der Struktur der betrachteten Topologie ab.

Abbildung 21 zeigt die durchschnittliche Hopanzahl. Die NDP-basierten Verfahren benötigen bei erfolgreichen Routings überwiegend weniger Hops als die entsprechenden EDP-Varianten.

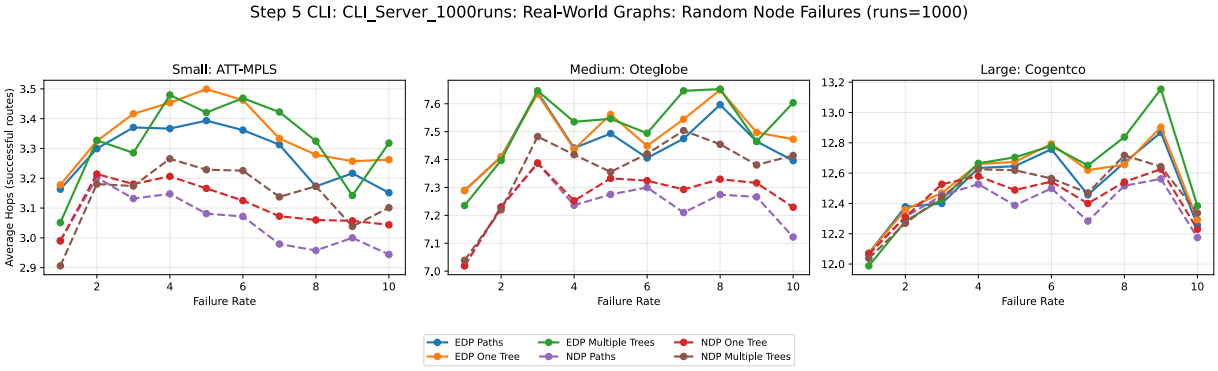


Abbildung 21: Durchschnittliche Hopanzahl bei zufälligen Knotenausfällen

Bei ATT-MPLS benötigt NDP Paths bei einer Failure Rate von 10 durchschnittlich 2,944 Hops. NDP One Tree benötigt 3,044 und NDP Multiple Trees 3,101 Hops. Die entsprechenden EDP-Werte betragen 3,152, 3,262 und 3,318. Die NDP-basierten Verfahren benötigen damit ungefähr 0,2 Hops weniger.

Bei Oteglobe ergeben sich für NDP Paths, NDP One Tree und NDP Multiple Trees bei der höchsten Failure Rate durchschnittlich 7,122, 7,229 und 7,415 Hops. Die EDP-basierten Varianten benötigen 7,396, 7,473 und 7,603 Hops. Auch hier sind die erfolgreichen NDP-Routen im Mittel kürzer.

Bei Cogentco fallen die Unterschiede geringer aus. NDP Paths benötigt durchschnittlich 12,175 Hops, NDP One Tree 12,230 und NDP Multiple Trees 12,336 Hops. Für die EDP-Verfahren ergeben sich 12,256, 12,292 und 12,384 Hops.

Innerhalb der NDP-basierten Verfahren besitzt NDP Paths meist die geringste Hopanzahl. NDP One Tree verursacht nur einen kleinen zusätzlichen Aufwand, während NDP Multiple Trees etwas mehr Hops benötigt und gleichzeitig die höchste oder eine der höchsten Resilienzen erreicht. Die Unterschiede sind jedoch deutlich kleiner als bei den geclusterten Ausfällen.

Der Average Stretch fällt bei den NDP-basierten Verfahren ebenfalls geringer aus. Bei Oteglobe beträgt er bei einer Failure Rate von 10 beispielsweise $0{,}248$ für NDP Paths, $0{,}317$ für NDP One Tree und $0{,}482$ für NDP Multiple Trees. Die entsprechenden EDP-Werte betragen $0{,}546$, $0{,}599$ und $0{,}696$.

Insgesamt zeigen die NDP-basierten Verfahren bei zufälligen Knotenausfällen auf den Real-World-Graphen vor allem Vorteile bei Hopanzahl und Stretch. Ein deutlicher Resilienzvorteil tritt dagegen nur bei einzelnen Topologien und Failure Rates auf. NDP Multiple Trees erreicht meist die höchste Resilienz der NDP-Varianten, benötigt dafür aber mehr Routingschritte als NDP Paths und NDP One Tree.

5.4.4 Clustered Node Failure

Die Auswertung der Real-World-Topologien schließt mit geclusterten Knotenausfällen nach Abschnitt 4.3.4 ab. Die verwendeten Parameter entsprechen dem Versuchsaufbau in Abschnitt 5.1.

Abbildung 22 zeigt die Resilienz der sechs Algorithmen. Bei einer Failure Rate von $0{,}3$ erreichen nahezu alle Verfahren auf den drei Topologien eine Resilienz nahe eins. Mit steigender Failure Rate nimmt die Resilienz jedoch deutlich ab. Der Rückgang ist bei ATT-MPLS besonders stark. Bei einer Failure Rate von $0{,}9$ erreichen dort nur noch zwischen $6{,}9\,\%$ und $9{,}8\,\%$ der Versuchsszenen erfolgreich das Ziel. Bei Oteglobe liegen die Werte zwischen $19{,}6\,\%$ und $26{,}0\,\%$, während Cogentco zwischen $23{,}2\,\%$ und $28{,}3\,\%$ erreicht.

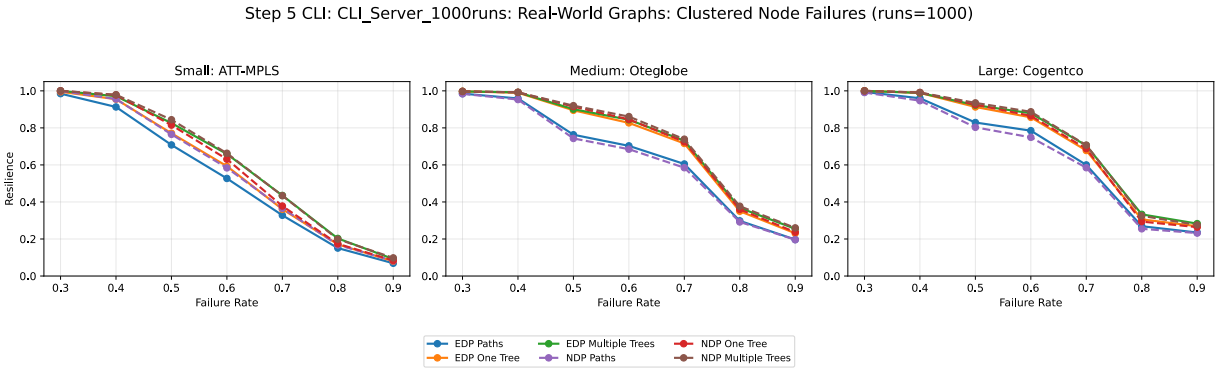


Abbildung 22: Resilienz bei geclusterten Knotenausfällen auf den Real-World-Graphen

Die Baumvarianten verbessern die Resilienz gegenüber den zugehörigen reinen Pfadverfahren. Dies wird beispielsweise bei einer Failure Rate von $0{,}7$ deutlich. Auf Oteglobe steigt die Resilienz von NDP Paths mit $0{,}585$ auf $0{,}725$ bei NDP One Tree und auf $0{,}739$ bei NDP Multiple Trees. Auf Cogentco erhöht sie sich von $0{,}585$ auf $0{,}688$ beziehungsweise $0{,}707$. Bei ATT-MPLS ist die Verbesserung geringer, aber ebenfalls erkennbar: NDP Paths erreicht $0{,}366$, NDP One Tree $0{,}378$ und NDP Multiple Trees $0{,}434$.

Zwischen den jeweils entsprechenden EDP- und NDP-Verfahren bestehen dagegen nur geringe Unterschiede. Bei einer Failure Rate von $0{,}7$ erreicht NDP Multiple Trees auf Oteglobe eine Resilienz von $0{,}739$, während EDP Multiple Trees $0{,}731$ erreicht. Auf Cogentco liegen die Werte bei $0{,}707$ und $0{,}705$. Bei ATT-MPLS liegt EDP Multiple Trees mit $0{,}435$ geringfügig vor NDP Multiple Trees mit $0{,}434$. Die NDP-basierten Verfahren sind bei geclusterten Knotenausfällen somit konkurrenzfähig, besitzen aber nicht auf jeder Topologie einen allgemeinen Resilienzvorteil.

Die durchschnittliche Hopanzahl der erfolgreichen Routings ist in Abbildung 23 dargestellt. Die reinen NDP-Pfade benötigen überwiegend weniger Hops als die NDP-Baumvarianten. Bei einer Failure Rate von $0{,}7$ beträgt die durchschnittliche Hopanzahl auf Oteglobe $9{,}53$ für NDP Paths, $11{,}60$ für NDP One Tree und $12{,}06$ für NDP Multiple Trees. Auf Cogentco ergeben sich entsprechend $18{,}66$, $21{,}28$ und $21{,}79$ Hops.

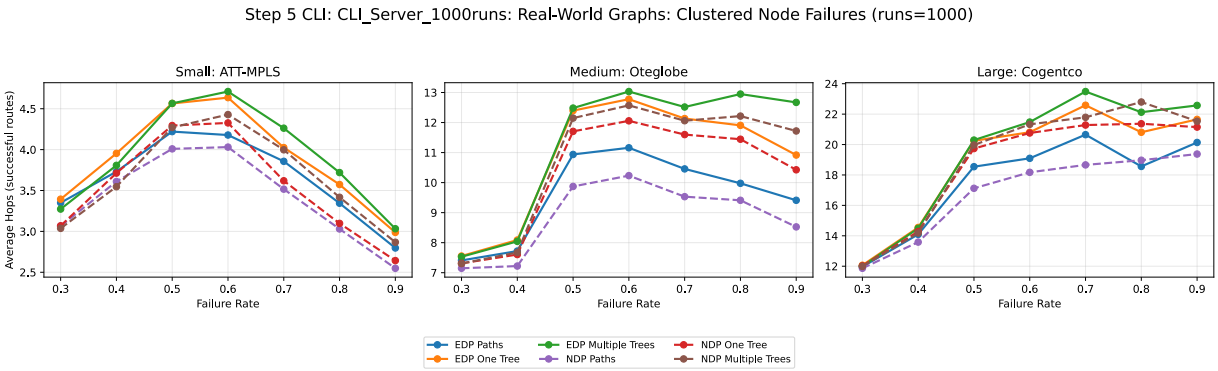


Abbildung 23: Durchschnittliche Hopanzahl bei geclusterten Knotenausfällen auf den Real-World-Graphen

Das Bild zeigt erneut den Zielkonflikt zwischen Resilienz und durchschnittlicher Hopanzahl.

Bei der Interpretation hoher Failure Rates ist zu beachten, dass Average Hops nur über erfolgreiche Routings berechnet wird. Auf ATT-MPLS sind bei einer Failure Rate von $0{,}9$ beispielsweise nur 82 Routings von NDP Paths, 84 von NDP One Tree und 98 von NDP Multiple Trees erfolgreich. Die dort vergleichsweise niedrigen Hopwerte beziehen sich daher auf eine kleine Teilmenge der insgesamt 1000 Versuchsszenen.

Die Ergebnisse für Average Stretch bestätigen den Zielkonflikt zwischen Resilienz und Routeneffizienz. Die Baumvarianten können zusätzliche Szenen erfolgreich routen, benötigen dafür jedoch teilweise mehr Umwege.

Insgesamt zeigen die Real-World-Graphen, dass die NDP-Erweiterungen bei geclusterten Knotenausfällen eine mit den EDP-Verfahren vergleichbare Resilienz erreichen. Der Vorteil von One Tree und Multiple Trees liegt vor allem in der höheren Erfolgswahrscheinlichkeit gegenüber dem jeweiligen reinen Pfadverfahren.

5.5 Vergleich der Algorithmen

Die Ergebnisse aus den Abschnitten 5.3 und 5.4 ermöglichen einen übergreifenden Vergleich der sechs Routingverfahren. Dabei werden zwei Unterschiede betrachtet: die Verwendung von EDPs oder NDPs als Ausgangspfade und deren Erweiterung durch One Tree oder Multiple Trees. Im Mittelpunkt stehen die Auswirkungen auf die Resilienz bei Knotenausfällen sowie auf die Länge der erfolgreichen Routen.

5.5.1 Vergleich der EDP- und NDP-Verfahren

Der Wechsel von EDPs zu NDPs führt in den untersuchten Konfigurationen nicht durchgehend zu einer höheren Resilienz. Auf den synthetischen Random-Graphen liegen die entsprechenden Varianten häufig nahe beieinander. Auch bei den Real-World-Graphen hängt die beobachtete Reihenfolge von der Topologie und dem Ausfallmodell ab. Bei zufälligen Linkausfällen erreichen die EDP-Verfahren dort überwiegend höhere Resilienzwerte als ihre NDP-basierten Gegenstücke.

Tabelle 3 stellt die Resilienz der EDP- und NDP-Verfahren für alle sechs Datasets bei geclusterten Knotenausfällen gegenüber. Für diesen Vergleich werden einheitlich `failureStart` $= 0{,}7$ und `failureDrop` $= 0{,}3$ verwendet. Die Tabelle zeigt damit eine ausgewählte Parameterkonfiguration; die vollständigen Verläufe werden in den Abschnitten 5.3 und 5.4 betrachtet.

Tabelle 3: Resilienz bei geclusterten Knotenausfällen mit `failureStart` $= 0{,}7$, `failureDrop` $= 0{,}3$ und 1000 Wiederholungen. Der höhere Wert jedes EDP-NDP-Paars ist fett hervorgehoben.

	Paths		One Tree		Multiple Trees	
Dataset	EDP	NDP	EDP	NDP	EDP	NDP
Random-Graphen						
$n = 25,\ p = 0{,}15$	0,536	**0,551**	0,597	**0,607**	0,655	**0,665**
$n = 50,\ p = 0{,}15$	0,677	**0,693**	0,716	**0,729**	0,859	**0,861**
$n = 100,\ p = 0{,}02$	0,554	**0,561**	0,641	**0,647**	0,657	**0,662**
Real-World-Graphen						
ATT-MPLS	0,328	**0,366**	0,361	**0,378**	**0,435**	0,434
Oteglobe	**0,605**	0,585	0,716	**0,725**	0,731	**0,739**
Cogentco	**0,600**	0,585	0,678	**0,688**	0,705	**0,707**

Bei dieser Parameterwahl erreichen die NDP-Verfahren auf den drei Random-Graph-Datasets jeweils eine etwas höhere Resilienz als die entsprechenden EDP-Varianten. Auf den Real-World-Graphen ist das Bild uneinheitlich. Während NDP Paths auf ATT-MPLS

vor EDP Paths liegt, ist die Reihenfolge auf Oteglobe und Cogentco umgekehrt. Bei den Baumvarianten fallen die Unterschiede überwiegend gering aus.

Ergänzend zeigt Tabelle 4 die durchschnittliche Hopanzahl für dieselben Konfigurationen wie Tabelle 3. Dadurch können Resilienz und Routenlänge gemeinsam betrachtet werden. Die Hopwerte werden jeweils ausschließlich über die erfolgreichen Routings eines Verfahrens gemittelt.

Tabelle 4: Durchschnittliche Hopanzahl erfolgreicher Routings bei geclusterten Knotenausfällen mit `failureStart` $= 0{,}7$ und `failureDrop` $= 0{,}3$. Der niedrigere Wert jedes EDP-NDP-Paars ist fett hervorgehoben.

	Paths		One Tree		Multiple Trees	
Dataset	EDP	NDP	EDP	NDP	EDP	NDP
Random-Graphen						
$n = 25,\ p = 0{,}15$	**3,71**	3,74	4,02	**4,01**	4,47	**4,42**
$n = 50,\ p = 0{,}15$	**4,09**	4,15	**4,40**	4,44	5,44	**5,43**
$n = 100,\ p = 0{,}02$	**7,13**	7,15	8,43	**8,35**	8,73	**8,61**
Real-World-Graphen						
ATT-MPLS	3,86	**3,52**	4,02	**3,62**	4,26	**4,00**
Oteglobe	10,46	**9,53**	12,13	**11,60**	12,52	**12,06**
Cogentco	20,65	**18,66**	22,58	**21,28**	23,49	**21,79**

Auf den drei Real-World-Graphen erreichen bei dieser Parameterwahl alle NDP-Varianten niedrigere durchschnittliche Hopwerte als ihre jeweiligen EDP-Gegenstücke. Auf Cogentco benötigt NDP Multiple Trees beispielsweise durchschnittlich $21{,}79$ statt $23{,}49$ Hops bei EDP Multiple Trees. Gleichzeitig liegt die Resilienz mit $0{,}707$ beziehungsweise $0{,}705$ nahe beieinander. Auf den Random-Graphen sind die Hopunterschiede kleiner und fallen nicht durchgehend zugunsten der NDP-Verfahren aus.

Für die Interpretation gelten die in Abschnitt 5.2.1 beschriebenen Einschränkungen der Hop-Mittelwerte.

Die stärkere Disjunktheit der Ausgangspfade allein reicht somit nicht aus, um eine höhere Erfolgswahrscheinlichkeit zu gewährleisten. Außerdem bleiben die erweiterten Bäume bei NDP Multiple Trees nicht zwingend intern knotendisjunkt, wie in Abschnitt 4.2 erläutert.

5.5.2 Einfluss der Baumerweiterungen

Deutlicher als die Unterschiede zwischen EDPs und NDPs sind in vielen Konfigurationen die Verbesserungen durch die Baumerweiterungen. In sämtlichen ausgewerteten

Konfigurationen erreicht NDP Multiple Trees mindestens die Resilienz von NDP One Tree. NDP One Tree erreicht wiederum mindestens die Resilienz von NDP Paths. Diese Reihenfolge beschreibt die vorliegenden Messwerte und stellt keine allgemeine Garantie für beliebige Graphen dar.

Besonders deutlich wird der Nutzen bei geclusterten Knotenausfällen. Für die Random-Graphen mit $n = 50$, $p = 0{,}15$ und `failureStart` $= 0{,}7$ steigt die Resilienz von $0{,}693$ bei NDP Paths auf $0{,}729$ bei NDP One Tree und $0{,}861$ bei NDP Multiple Trees. Die Erweiterung aller Ausgangspfade verbessert die Resilienz gegenüber NDP Paths damit um $16{,}8$ Prozentpunkte. Der Abstand zwischen NDP Multiple Trees und EDP Multiple Trees beträgt bei derselben Konfiguration dagegen lediglich $0{,}2$ Prozentpunkte. In diesem Beispiel ist die Baumerweiterung für den Resilienzgewinn wesentlich bedeutender als die Wahl zwischen EDPs und NDPs.

5.5.3 Resilienz und Routeneffizienz

Die höhere Resilienz der Baumvarianten geht häufig mit einer größeren durchschnittlichen Hopanzahl und einem höheren durchschnittlichen Stretch einher. Zusätzliche Äste ermöglichen alternative Weiterleitungen, können jedoch auch Umwege und Rückwege verursachen. Bei der zuvor betrachteten Random-Graph-Konfiguration benötigt NDP Paths durchschnittlich $4{,}15$ Hops, NDP One Tree $4{,}44$ und NDP Multiple Trees $5{,}43$. Der durchschnittliche additive Stretch steigt entsprechend von $1{,}35$ über $1{,}59$ auf $2{,}44$.

Diese Mittelwerte beziehen sich jeweils auf die erfolgreichen Routings des einzelnen Algorithmus. Die verglichenen Teilmengen sind daher nicht identisch: NDP Multiple Trees routet im genannten Beispiel 861 Szenen erfolgreich, NDP Paths dagegen 693. Ein niedrigerer Hopwert bedeutet deshalb nicht automatisch, dass ein Verfahren auf denselben Szenen kürzere Routen findet. Resilienz, Average Hops und Average Stretch werden deshalb gemeinsam betrachtet. NDP Multiple Trees bietet in den untersuchten Konfigurationen die höchste oder eine gleich hohe Resilienz innerhalb der NDP-Verfahren; eine gleichzeitige Überlegenheit bei allen drei Metriken lässt sich daraus jedoch nicht ableiten.

5.6 Diskussion der Ergebnisse

Die Ergebnisse aus den Abschnitten 5.3 bis 5.5 zeigen, dass die Baumerweiterungen auch bei knotendisjunkten Ausgangspfaden Vorteile bieten. NDP One Tree und NDP Multiple Trees können zusätzliche Versuchsszenen erfolgreich routen, in denen NDP Paths scheitert. Die erweiterten Strukturen stellen weitere Wege zum Ziel bereit. Der Wechsel von EDPs zu NDPs allein führt dagegen nicht auf allen Graphen und bei allen Ausfallmodellen zu einer höheren Resilienz.

Eine mögliche Erklärung liegt im Aufbau der Ausgangspfade und der Bäume. Durch die stärkere Disjunktheitsbedingung können weniger NDPs als EDPs zur Verfügung stehen. Gleichzeitig hängt die Erweiterung davon ab, welche zusätzlichen Kanten im Graphen vorhanden sind und zu einem Ausgang führen. Bei NDP Multiple Trees können die erweiterten Bäume außerdem gemeinsame innere Knoten enthalten. Ein Knotenausfall kann dadurch mehrere Bäume beeinträchtigen, obwohl ihre ursprünglichen NDPs diesen Knoten nicht gemeinsam verwenden. Die stärkere Trennung der Ausgangspfade bedeutet somit nicht automatisch eine entsprechend stärkere Trennung der vollständigen Routingstrukturen.

Auch die Lage der Ausfälle spielt eine Rolle. Bei den geclusterten Fehlermodellen werden die Ausfälle ausgehend von der Umgebung des Zielknotens ausgewählt. Dadurch können mehrere Zugänge zum Ziel betroffen sein. Zusätzliche Baumäste helfen nur, wenn über sie noch eine verfügbare Verbindung zum Ziel erreicht werden kann. Wie viele solcher Möglichkeiten bestehen, hängt von der jeweiligen Topologie ab. Dies gilt sowohl für die Real-World-Graphen als auch für die synthetischen Random-Graphen. Bei Erdős-Rényi-Graphen beeinflussen die Parameter n und p, welche Verbindungen entstehen. Auch bei gleicher Parameterwahl können sich die erzeugten Graphinstanzen darin unterscheiden, wie viele alternative Wege zum Ziel vorhanden sind. Die Unterschiede zwischen den untersuchten Datasets lassen sich deshalb nicht allein durch ihre Knotenanzahl erklären.

Bei zufälligen Ausfällen muss zwischen dem eingestellten `failFactor` und der tatsächlichen Ausfallanzahl unterschieden werden. Die Berechnung verwendet bei Kantenausfällen die Anzahl der EDPs und bei Knotenausfällen die Anzahl der NDPs. Derselbe Faktor kann deshalb in verschiedenen Versuchsszenen unterschiedliche Ausfallmengen erzeugen. Wird beispielsweise nur ein NDP berechnet, ergibt die implementierte Formel für Random Node Failure null Knotenausfälle. Ein erfolgreiches Routing in einer solchen Szene zeigt daher nicht, dass das Verfahren Ausfälle überwunden hat. Innerhalb derselben Versuchsszene erhalten jedoch alle sechs Verfahren dieselbe Ausfallmenge, sodass ihr direkter Vergleich unter gleichen Bedingungen erfolgt.

Ein weiterer Unterschied besteht zwischen dem Scheitern eines Routingverfahrens und dem vollständigen Verlust der Verbindung. Nach den Ausfällen kann es vorkommen, dass im verbleibenden Graphen überhaupt kein Pfad zwischen Start und Ziel mehr existiert. Dann kann keines der Verfahren erfolgreich sein. Solche Szenen gehen ebenfalls als Fehlschläge in die Resilienz ein. Ein niedriger Resilienzwert bedeutet deshalb nicht zwangsläufig, dass der Algorithmus vorhandene Alternativwege nicht gefunden hat.

Die Rohdaten zeigen außerdem, dass ein Routing auch dann scheitern kann, wenn Start und Ziel nach den Ausfällen weiterhin verbunden sind. Bei Oteglobe bleiben beispielsweise unter geclusterten Knotenausfällen mit `failureStart` $= 0{,}7$ und `failureDrop` $= 0{,}3$ in 807 der 1000 Versuchsszenen Restpfade erhalten. Innerhalb dieser 807 Szenen scheitert NDP Paths in 222 Fällen, NDP One Tree in 82 Fällen und NDP Multiple Trees in 68 Fällen. Bezogen ausschließlich auf die weiterhin verbundenen Szenen ergeben sich

damit Erfolgsquoten von ungefähr $72{,}5\,\%$, $89{,}8\,\%$ und $91{,}6\,\%$. Die Baumerweiterungen können somit in mehr dieser Szenen eine verfügbare Verbindung erfolgreich nutzen. Ein vorhandener Pfad im Restgraphen garantiert jedoch nicht, dass dieser vollständig innerhalb einer der vorbereiteten Routingstrukturen verläuft. Es kann weiterhin eine Verbindung zwischen Start und Ziel bestehen, die das Verfahren mit seinen vorbereiteten Pfaden und Bäumen nicht nutzen kann. Dadurch können auch die Baumvarianten trotz bestehender Verbindung scheitern.

Die höhere Resilienz der Baumvarianten geht häufig mit einer größeren durchschnittlichen Hopanzahl und einem höheren durchschnittlichen Stretch einher. Das Routing kann zusätzliche Äste ausprobieren und muss bei erfolglosen Versuchen zurückgehen. Dadurch entstehen weitere Hops. Die zusätzlichen Wege können aber auch ermöglichen, dass eine zuvor nicht erfolgreich geroutete Szene das Ziel erreicht. Der höhere Routingaufwand steht damit dem Gewinn an erfolgreichen Routings gegenüber.

6 Experimentierumgebung, Reproduzierbarkeit und Erweiterbarkeit

Dieses Kapitel untersucht, ob andere Personen die entwickelte Experimentierumgebung verwenden und erweitern können. Dafür erprobten drei Studierende das Framework und beantworteten zwei Fragebögen. Abschnitt 6.1 beschreibt zunächst die bereitgestellten Dateien. Anschließend erläutert Abschnitt 6.2 den Ablauf der Evaluation. Die Ergebnisse zur Bedienbarkeit und Reproduzierbarkeit werden in Abschnitt 6.3 vorgestellt. Darauf folgen die Ergebnisse zur Erweiterbarkeit in Abschnitt 6.4. Abschnitt 6.5 behandelt die Überarbeitung anhand der Rückmeldungen. Abschließend fasst Abschnitt 6.6 die Bewertung der Umgebung zusammen.

6.1 Bereitstellung der Experimentierumgebung

Die drei Studierenden erhielten die Experimentierumgebung als ZIP-Datei. Diese enthielt die Projektordner der Schritte 5 bis 7. Damit standen sowohl der ausführbare Code als auch die Dokumentation, die Erweiterungsanleitung und die beiden Fragebögen zur Verfügung.

Der Schritt-5-Ordner enthielt die Pipeline, die grafische Bedienoberfläche und die Module für Routing, Ausfallerzeugung und Ergebnisauswertung. Die GraphML-Dateien der Topologien ATT-MPLS, Oteglobe und Cogentco waren ebenfalls enthalten. Die benötigten Projektmodule und Real-World-Graphen mussten deshalb nicht aus anderen Projektordnern übernommen werden. Der technische Aufbau wurde bereits in Kapitel 4 beschrieben.

Als Einstieg diente die Datei `README.txt` im Dokumentationsordner. Sie erläutert die Einrichtung einer virtuellen Python-Umgebung unter Windows und Linux sowie den Start über die CLI oder die Streamlit-GUI. Außerdem beschreibt sie die Konfiguration der Experimente und die Ablage der Ergebnisse. Ergänzend enthielt das Paket eine Modulübersicht, Protokolle zu den einzelnen Modulen und Testdateien für Pipeline und GUI.

Die zum Zeitpunkt der Artefaktevaluation bereitgestellte `pilot.json` verwendete zwei Wiederholungen auf einem kleinen synthetischen Graphen. Die Konfiguration `full.json` sah dagegen 1000 Wiederholungen für drei Random-Graphsets und die drei Real-World-Topologien vor. Beide Dateien verwendeten den Seed 123 und konnten als Ausgangspunkt für eigene Konfigurationen genutzt werden.

Der Schritt-6-Ordner enthielt den Fragebogen zur Installation, Bedienung und Reproduzierbarkeit. Im Schritt-7-Ordner lagen der Erweiterungsleitfaden `ERWEITERUNG.txt` und der zweite Fragebogen. Der Leitfaden benennt die relevanten Stellen im Code und enthält Beispiele für einen kürzesten-Pfad-Algorithmus, geclusterte Ausfälle um einen

zufälligen Knoten und einen vollständigen Graphen. Diese Unterlagen bildeten die Grundlage für die anschließende Erprobung der Erweiterbarkeit.

6.2 Aufbau und Durchführung der Artefaktevaluation

An der Evaluation nahmen drei Studierende der Informatik teil. Sie werden im Folgenden als T1, T2 und T3 bezeichnet. Als Ausgangspunkt diente das in Abschnitt 6.1 beschriebene ZIP-Paket. Die Erprobung erfolgte in zwei Phasen auf unterschiedlichen Testsystemen. Tabelle 5 fasst die Ausführungsumgebungen der zurückgegebenen Laufmanifeste zusammen.

Tabelle 5: Ausführungsumgebungen der dokumentierten Testläufe

Testperson	Betriebssystem	Python-Version
T1	Windows 11	3.14.4
T2	macOS 26.6.2	3.12.14
T3	Windows 11	3.14.4

T2 berichtete außerdem von einem ersten Startversuch mit Python 3.9.6 und ergänzenden Prüfläufen unter Linux mit Python 3.12.3. Die Testbedingungen waren somit nicht vollständig vereinheitlicht.

Vor der Erprobung erhielten alle drei Studierenden durch den Verfasser eine Einführung in das Thema und die Aufgabenstellung. Während beider Testphasen wurden sie bei Bedarf gelegentlich unterstützt. Die Evaluation erfolgte somit nicht vollständig ohne Hilfestellung. Die Rückmeldungen beschreiben die Erfahrungen mit der Nutzung und Erweiterung der Umgebung unter diesen Bedingungen.

In der ersten Phase standen Installation und Bedienung im Vordergrund. Die Studierenden sollten die Abhängigkeiten installieren, die Umgebung starten und kleine Experimente durchführen. Als Einstieg standen die Pilot-Konfiguration und die GUI zur Verfügung. T1 und T2 verwendeten beide Zugangswege. T3 konzentrierte sich in dieser Phase auf die CLI. Der erste Fragebogen erfasste anschließend die Erfahrungen mit Installation, Konfiguration, Ausführung und Ergebnisablage. Zusätzlich wurde nach der Wiederholbarkeit der Ergebnisse und der Verständlichkeit der Dokumentation gefragt.

In der zweiten Phase wurde die Erweiterbarkeit untersucht. Als Hilfsmittel dienten der Erweiterungsleitfaden, die Modulprotokolle und die markierten Erweiterungsstellen im Code. T1 und T2 ergänzten alle drei vorgeschlagenen Beispiele: einen kürzesten-Pfad-Algorithmus, ein geclustertes Fehlermodell mit zufälligem Mittelpunkt und einen vollständigen Graphen. T3 beschränkte sich auf den kürzesten-Pfad-Algorithmus und integrierte diesen auch in die GUI und die Auswertung.

Zur Prüfung der Erweiterungen wurden kleine Experimentläufe durchgeführt und zusätzliche Unit-Tests ergänzt. T3 verwendete in dieser Phase ebenfalls die GUI. Danach

beantworteten die Studierenden den zweiten Fragebogen. Dieser erfasste unter anderem die Orientierung im Code, die Verständlichkeit der Erweiterungsstellen und den benötigten Aufwand.

Beide Fragebögen enthielten offene Fragen sowie Bewertungen auf einer Skala von eins bis fünf. Für die Auswertung wurden die Antworten thematisch zusammengefasst und durch die zurückgegebenen Codeänderungen und Ergebnisdateien ergänzt.

6.3 Ergebnisse zur Bedienbarkeit und Reproduzierbarkeit

Alle drei Testpersonen konnten die Umgebung starten und kleine Experimente durchführen. Unterschiede zeigten sich vor allem bei der Installation. T1 berichtete von keinen Startproblemen. Bei T2 fehlte zunächst NetworkX. Zusätzlich musste die zuerst verwendete Python-Version 3.9.6 durch eine neuere Version ersetzt werden. T3 musste Matplotlib und Pandas nachinstallieren. Diese Rückmeldungen zeigen, dass die bereitgestellten Installationshinweise die benötigte Umgebung noch nicht vollständig beschrieben.

Die Ordnerstruktur und die grundlegende Konfiguration wurden überwiegend als verständlich bewertet. Auch die Ablage der Ergebnisdateien war für alle drei Personen nachvollziehbar. Unklarheiten bestanden bei einzelnen Parametern: T2 vermisste eine genauere Erklärung der Failure-Parameter. T3 musste zunächst nachsehen, wie die Anzahl der Wiederholungen geändert werden kann. Die GUI-Auswahl wurde von T1 und T2 positiv beurteilt. T3 verwendete in dieser ersten Phase ausschließlich die CLI und bewertete die GUI daher nicht.

Die README und die Modulprotokolle unterstützten nach den Rückmeldungen die Orientierung im Projekt. T1 schlug jedoch eine bessere Strukturierung der README vor. T2 und T3 wünschten sich insbesondere vollständigere Installationsangaben und zusätzliche Beispiele. T2 bemängelte außerdem fehlende Fortschrittsmeldungen während längerer Läufe.

Tabelle 6 zeigt die subjektiven Bewertungen aus dem ersten Fragebogen. Die Skala reicht von eins bis fünf, wobei höhere Werte eine positivere Bewertung ausdrücken. Die Bewertung der Reproduzierbarkeit bezeichnet hier deren Nachvollziehbarkeit aus Sicht der Testpersonen.

Tabelle 6: Subjektive Bewertungen aus dem ersten Fragebogen

Testperson	Einfachheit des Starts	Reproduzierbarkeit
T1	5	5
T2	2	3
T3	4	5

Zur Wiederholbarkeit machten alle drei Personen positive Angaben. T1 bestätigte diese im Fragebogen, ohne den Vergleich näher zu beschreiben. T2 berichtete von gleichen

Ergebnisdateien bei wiederholten Läufen mit gleichem Seed. T3 beschrieb ausdrücklich, dass die Summary- und Rohdaten-Dateien bei identischer Konfiguration und identischem Seed übereinstimmten. Diese Angaben betreffen Wiederholungen innerhalb der jeweiligen Testumgebung. Ein systematischer Ergebnisvergleich zwischen den unterschiedlichen Testsystemen ist damit noch nicht belegt.

T2 wies zudem darauf hin, dass feste Bibliotheksversionen fehlen. Die gespeicherte Konfiguration erleichtert zwar das Nachvollziehen eines Laufs, legt aber nicht die gesamte Softwareumgebung fest. Die Rückmeldungen zeigen somit eine grundsätzlich nutzbare Experimentierumgebung, deren Bereitstellung noch verbessert werden kann. Dabei ist die in Abschnitt 6.2 beschriebene Einführung und gelegentliche Unterstützung durch den Verfasser zu berücksichtigen.

6.4 Ergebnisse zur Erweiterbarkeit

Alle drei Testpersonen berichteten, dass sie die von ihnen bearbeiteten Erweiterungen einbinden und anschließend kleine Testläufe durchführen konnten. Die zurückgegebenen Projektstände enthalten die entsprechenden Codeänderungen und ergänzte Tests. Der Umfang unterschied sich jedoch: T1 und T2 bearbeiteten alle drei Beispiele, während T3 ausschließlich den kürzesten-Pfad-Algorithmus ergänzte.

Der Erweiterungsleitfaden und die markierten Stellen im Code wurden von allen drei Personen als hilfreich bewertet. Insbesondere der Hook `EXTRA_ALGORITHM_RUNNERS` erleichterte die Einbindung des neuen Algorithmus. T2 hob hervor, dass dafür nicht der gesamte Aufbau der bestehenden Routingverfahren verstanden werden musste. T1 wünschte sich lediglich eine übersichtlichere Strukturierung des Leitfadens.

Bei der praktischen Umsetzung wurden dennoch Schwierigkeiten sichtbar. T3 berichtete von einem `KeyError` bei der Ploterstellung, weil der Farbeintrag für den neuen Algorithmus noch nicht im geladenen Modul verfügbar war. Nach dem Speichern und Prüfen des Moduls konnte der Testlauf abgeschlossen werden. Das Beispiel zeigt, dass neben dem Algorithmus selbst auch seine Einbindung in die Ergebnisdarstellung geprüft werden muss.

T2 benannte weitere Stellen, an denen der Leitfaden genauer werden sollte. Dazu gehörten die Eingabe- und Rückgabeformate neuer Algorithmen sowie die Einbindung eines neuen Graphtyps in die GUI. Für den vollständigen Graphen reichte ein zusätzlicher Auswahleintrag nicht aus; auch die Verarbeitung der zugehörigen Parameter musste ergänzt werden. Außerdem wies T2 darauf hin, dass neu registrierte Algorithmen und Fehlermodelle automatisch in die bisherige Vorauswahl aufgenommen werden konnten. Damit könnten Erweiterungen unbeabsichtigt den Umfang bestehender Versuchskonfigurationen verändern. Nach eigenen Angaben benötigten die Studierenden je nach Erweiterungsumfang etwa zwei bis fünf Stunden.

Alle drei Studierenden bewerteten die Einfachheit der Erweiterung mit vier von fünf Punkten. Die unterschiedlichen Zeitangaben lassen sich jedoch nicht unmittelbar miteinander vergleichen, da sich Aufgabenumfang, Vorgehen und Testaufwand unterschieden. Zusätzlich ist die gelegentliche Unterstützung durch den Verfasser zu berücksichtigen.

Die Rückmeldungen sprechen dafür, dass einfache Erweiterungen mit den bereitgestellten Hilfsmitteln möglich sind. Sie zeigen aber auch, dass eine erfolgreiche Registrierung allein nicht genügt. GUI, Auswertung und bisherige Standardkonfigurationen müssen ebenfalls überprüft werden. Die daraus abgeleiteten Anpassungen werden im folgenden Abschnitt behandelt.

6.5 Überarbeitung anhand der Rückmeldungen

Die Rückmeldungen der drei Studierenden wurden genutzt, um die Dokumentation und die Bedienung der Experimentierumgebung zu verbessern. Im Mittelpunkt standen die Installation, verständliche Fehlermeldungen und die Einbindung weiterer Verfahren. Die bestehenden Routingalgorithmen und Ausfallmodelle wurden dabei nicht verändert.

Zunächst wurde die README gekürzt und auf die wichtigsten Schritte für Installation, Konfiguration und Ausführung ausgerichtet. Ausführliche Erklärungen stehen in den jeweiligen Modulprotokollen. Die Installationsbefehle unterscheiden zwischen Windows und Linux beziehungsweise macOS. Die Datei `requirements.txt` enthält die benötigten Bibliotheken mit festgelegten Versionen. Zusätzlich erläutert die Anleitung die Einrichtung einer virtuellen Python-Umgebung. Für den Pilotlauf werden die erwarteten CSV- und Plotdateien angegeben, damit neue Benutzer ihren ersten Versuch leichter überprüfen können.

Auch der Erweiterungsleitfaden wurde präzisiert. Er beschreibt nun genauer, welche Argumente eine neue Funktion erhält, welche Werte sie zurückgeben muss und an welchen Stellen sie in die Umgebung eingetragen wird. Über die Erweiterungsstellen ergänzte Algorithmen und Fehlermodelle müssen ausdrücklich ausgewählt werden und verändern nicht die bisherige Standardauswahl. Neue Grapharten benötigen eine passende Parameterverarbeitung und gegebenenfalls eine eigene Regel zur Auswahl der Start- und Zielknoten. Die studentischen Erweiterungsbeispiele wurden nicht automatisch in den Basiscode übernommen.

Weitere Anpassungen betreffen die Ausführung. Fehlende Bibliotheken, ungültige Parameter sowie fehlende Plotbeschriftungen oder Farben werden früher gemeldet. Unbekannte Grapharten werden in der GUI nicht mehr versehentlich als Real-World-Datasets behandelt. CLI und GUI zeigen außerdem den Fortschritt der Jobs und der abgeschlossenen Wiederholungen an. Die Standardausgaben wurden unter `Output/` vereinheitlicht. Das Manifest wurde um Paketversionen und Prüfsummen des Codes sowie der verwendeten GraphML-Dateien ergänzt.

Auch die Speicherung von CLI-Überschreibungen wurde vereinheitlicht. Die tatsächlich verwendeten Algorithmus- und Failure-Auswahlen sowie der wirksame Resume-Wert werden nun in `used_config.json` und im Manifest gespeichert. Dadurch stimmen die gespeicherten Metadaten mit dem ausgeführten Experimentplan überein.

Vor der Erstellung der Summary und der Plots leitet die Pipeline alle erwarteten Parametergruppen aus dem Experimentplan ab. Diese werden mit den tatsächlich vorhandenen Ergebnisgruppen verglichen. Dadurch wird nun auch erkannt, wenn beispielsweise die vollständige Rohdatendatei für einen bestimmten `failFactor` fehlt. Zusätzlich wird für jede vorhandene Gruppe geprüft, ob alle ausgewählten Algorithmen und die erwartete Anzahl an Ergebniszeilen enthalten sind.

Die Änderungen wurden anschließend durch automatisierte Tests geprüft. Diese kontrollieren unter anderem die Standardauswahl der Algorithmen und Fehlermodelle, die Behandlung ungültiger Eingaben, die Auswahl der Start- und Zielknoten sowie die korrekte Fortsetzung unterbrochener Versuchsreihen. Zwei zusätzliche Regressionstests prüfen die Speicherung der wirksamen CLI-Überschreibungen und den Abbruch bei einer vollständig fehlenden Parametergruppe.

Drei weitere Tests prüfen die Streamlit-Oberfläche ohne einen externen Browser. Sie kontrollieren die Vorauswahl, den Umgang mit unbekannten Grapharten und einen kleinen vollständigen Experimentlauf einschließlich Fortschrittsanzeige und Ergebnisdateien.

In einer separaten Windows-Umgebung mit Python 3.11.3 bestanden alle 74 automatisierten Tests. Damit wurden die von den Tests abgedeckten Funktionen technisch geprüft. Eine erneute Bewertung der Bedienbarkeit durch die drei Testpersonen wurde dadurch jedoch nicht ersetzt.

Die neue Prüfung erkennt vollständig fehlende Parametergruppen. Eine verbleibende Einschränkung betrifft jedoch die Wiederholungsnummern innerhalb vorhandener Rohdatengruppen. Bei der Summary-Prüfung wird ihre Anzahl kontrolliert, aber noch nicht ausdrücklich geprüft, ob jede Nummer von 1 bis zur festgelegten Wiederholungszahl genau einmal vorkommt. Die reguläre Experiment- und Resume-Logik verhindert solche Unstimmigkeiten weitgehend. Sie könnten jedoch bei manuell veränderten oder extern zusammengestellten Rohdaten unentdeckt bleiben.

6.6 Zusammenfassende Bewertung

Die Artefaktevaluation zeigt insgesamt, dass das entwickelte Framework von anderen Informatikstudierenden verwendet und erweitert werden kann. Alle drei Testpersonen konnten nach der Einrichtung der benötigten Abhängigkeiten einen kleinen Experimentlauf erfolgreich durchführen. Die Konfiguration, die Ausführung und die Ablage der Rohdaten, Zusammenfassungen und Plots wurden überwiegend als verständlich bewertet.

Bei den durchgeführten Wiederholungstests entstanden mit derselben Konfiguration und demselben Seed übereinstimmende Ergebnisse. Dies bestätigt die Reproduzierbarkeit für die jeweils geprüften kleinen Versuchsreihen innerhalb der verwendeten Softwareumgebung. Daraus folgt jedoch keine allgemeine Garantie für beliebige Konfigurationen oder abweichende Bibliotheksversionen. Deshalb werden die verwendeten Parameter, Paketversionen und Prüfsummen inzwischen im Manifest festgehalten.

Die größten Schwierigkeiten bestanden bei der erstmaligen Installation. Die daraufhin ergänzten Versions-, Paket- und Einrichtungshinweise betreffen damit einen für externe Benutzer besonders wichtigen Bereich.

Auch die Erweiterbarkeit wurde grundsätzlich bestätigt. Zwei Testpersonen ergänzten einen Algorithmus, ein Fehlermodell und eine Graphart. Die dritte Testperson untersuchte die Einbindung eines zusätzlichen Algorithmus. Die vorgesehenen Erweiterungsstellen ermöglichten diese Änderungen, ohne dass der Kern der bestehenden Routingalgorithmen vollständig verstanden oder verändert werden musste. Allerdings wurden nicht alle Erweiterungsarten von jeder Testperson geprüft.

Die Rückmeldungen führten zu konkreten Verbesserungen an Dokumentation, Validierung, Fortschrittsanzeige und Speicherung der Versuchsmetadaten. Die anschließenden automatisierten Tests bestätigen, dass die davon abgedeckten Funktionen technisch arbeiten und die bisherigen Versuchsstandards erhalten bleiben. Die überarbeitete Fassung wurde jedoch nicht erneut durch Testpersonen evaluiert. Eine Verbesserung der Bedienbarkeit wird daher aus den behobenen Problemen abgeleitet, aber nicht durch eine zweite Artefaktevaluierung bestätigt.

Die Aussagekraft der Evaluation bleibt außerdem durch die geringe Zahl von drei Testpersonen und die verwendeten kleinen Testläufe begrenzt. Alle Testpersonen erhielten eine kurze Einführung in das Thema und bei Bedarf gelegentliche Unterstützung. Die Ergebnisse belegen deshalb keine vollständig unabhängige Nutzung unter allen möglichen Bedingungen. Sie zeigen jedoch, dass die Experimentierumgebung eine geeignete Grundlage für reproduzierbare Versuche und spätere Erweiterungen darstellt.

7 Fazit und Ausblick

Dieses Kapitel fasst die wesentlichen Ergebnisse der Arbeit zusammen und ordnet sie in Bezug auf die zuvor formulierten Ziele ein. Zunächst werden in Abschnitt 7.1 der entwickelte Ansatz und die erreichten Ziele zusammengefasst. Anschließend werden in Abschnitt 7.2 die in Abschnitt 1.4 aufgestellten Forschungsfragen beantwortet. Abschnitt 7.3 behandelt die Grenzen der Arbeit. Abschließend werden in Abschnitt 7.4 mögliche Erweiterungen der Routingverfahren, der untersuchten Graphen und Fehlermodelle sowie der Experimentierumgebung aufgezeigt.

7.1 Zusammenfassung und erreichte Ziele

Ausgangspunkt dieser Arbeit war der TREE-Ansatz, der Fast-Failover-Routingstrukturen auf der Grundlage kantendisjunkter Pfade (EDPs) erzeugt und ursprünglich für Kantenausfälle untersucht wurde. Das zentrale Ziel dieser Arbeit bestand darin, diesen Ansatz auf Knotenausfälle zu erweitern und zusätzlich knotendisjunkte Pfade (NDPs) als Grundlage der Routingstrukturen zu untersuchen. Dadurch sollte bestimmt werden, wie sich die stärkere Disjunktheitsbedingung der NDPs auf die Resilienz und den Routingaufwand auswirkt.

Hierfür wurden mit NDP Paths, NDP One Tree und NDP Multiple Trees drei zusätzliche Routingverfahren umgesetzt. Gemeinsam mit den ursprünglichen EDP-basierten Varianten stehen damit sechs Verfahren zur Verfügung. Die Konstruktion der One-Tree- und Multiple-Trees-Strukturen wurde auf knotendisjunkte Pfade übertragen. Knotenausfälle werden dabei als Ausfall aller inzidenten Kanten des betroffenen Knotens modelliert. Auf diese Weise können alle Verfahren über eine gemeinsame Routingroutine ausgeführt und unter denselben Bedingungen verglichen werden.

Für die Experimente wurden zufällige und geclusterte Kanten- sowie Knotenausfälle umgesetzt. Bei synthetischen Datensätzen steuert ein Seed die Erzeugung der Graphinstanzen. Bei Real-World-Datensätzen wird dagegen in jeder Wiederholung dieselbe eingelesene Topologie verwendet. Die Auswahl der Start- und Zielknoten sowie der Ausfälle erfolgt bei beiden Graphkategorien Seed-gesteuert. Stabile Sortierungen sorgen zusätzlich dafür, dass gleiche Konfigurationen eine reproduzierbare Folge von Versuchsszenen erzeugen. Innerhalb einer Szene erhalten alle sechs Routingverfahren denselben Graphen, dasselbe Start-Ziel-Paar und dieselbe Ausfallmenge.

Die Evaluation auf drei Erdős-Rényi-Konfigurationen und den Real-World-Topologien ATT-MPLS, Oteglobe und Cogentco zeigt, dass kein Verfahren unter allen Bedingungen zugleich die höchste Resilienz, die niedrigste Hopanzahl und den niedrigsten Stretch erreicht. Die beobachteten Unterschiede hängen vom Pfadtyp, der Baumerweiterung, der Topologie und dem Fehlermodell ab.

Neben den Routingverfahren wurde eine Experimentierumgebung entwickelt, die den vollständigen Versuchsablauf automatisiert. Die Pipeline kann über die Kommandozeile oder die Streamlit-Oberfläche bedient werden. Bei der Kommandozeile wird eine vorhandene JSON-Konfiguration als Grundlage eingelesen und kann durch zusätzliche Parameter überschrieben werden. Die GUI erstellt aus den gewählten Einstellungen eine `gui_config.json` und übergibt diese anschließend an denselben Pipeline-Kern.

Vor der Ausführung wird die tatsächlich verwendete Konfiguration in `used_config.json` gespeichert. Nach Abschluss des Laufs hält die Datei `manifest.json` unter anderem die Versuchsparameter, die Softwareumgebung und die erzeugten Dateien fest. Die Pipeline erstellt außerdem Rohdaten, Zusammenfassungen und Vergleichsplots und unterstützt das Fortsetzen unterbrochener Versuchsreihen. Definierte Erweiterungsstellen ermöglichen die Ergänzung neuer Algorithmen, Fehlermodelle und Grapharten.

Die Umgebung wurde von drei Informatikstudierenden anhand kleiner Experimentläufe und vorgegebener Erweiterungsaufgaben erprobt. Ihre Rückmeldungen führten insbesondere zu Verbesserungen der Installation, Dokumentation, Validierung und Fortschrittsanzeige. Damit wurden sowohl die algorithmischen Erweiterungen als auch eine reproduzierbare und erweiterbare Grundlage für weitere Experimente umgesetzt. Die daraus abgeleiteten Antworten auf die Forschungsfragen werden im folgenden Abschnitt 7.2 zusammengefasst.

7.2 Beantwortung der Forschungsfragen

Im Folgenden werden die vier in Abschnitt 1.4 formulierten Forschungsfragen anhand der Implementierung, der Evaluation und der Erprobung der Experimentierumgebung beantwortet.

7.2.1 Übertragung der TREE-Verfahren

Wie lassen sich die für kantendisjunkte Pfade entwickelten TREE-Verfahren auf Knotenausfälle und knotendisjunkte Pfade übertragen?

Die TREE-Verfahren lassen sich übertragen, indem zunächst die Berechnung der Ausgangspfade ausgetauscht wird. Anstelle kantendisjunkter Pfade werden knotendisjunkte Pfade zwischen dem Start- und Zielknoten berechnet. Bei NDP One Tree wird der längste dieser Pfade zum Grundgerüst eines Baums erweitert. Bei NDP Multiple Trees bildet jeder knotendisjunkte Pfad das Grundgerüst eines eigenen Baums. Nach der Konstruktion werden alle sechs Basisverfahren in der vorliegenden Implementierung über die gemeinsamen Funktionen `route_algorithm(...)` und `route_structure(...)` ausgeführt. Reine Pfade und erweiterte Bäume werden dafür einheitlich als Routingstrukturen dargestellt. Die Routingroutine orientiert sich an den Grundprinzipien der TREE-

Referenz, verwendet jedoch eine eigene deterministische Reihenfolge für die Strukturen und ihre Äste.

Knotenausfälle werden als gleichzeitiger Ausfall aller inzidenten Kanten des betroffenen Knotens dargestellt. Dadurch können die vorhandenen Routingfunktionen weiterhin mit einer Menge nicht verfügbarer Kanten arbeiten. Start- und Zielknoten werden bei der Auswahl der Knotenausfälle geschützt.

Die Übertragung auf NDP-basierte Verfahren und Knotenausfälle ist damit grundsätzlich möglich. Die vollständigen NDP-Multiple-Trees-Strukturen sind in der vorliegenden Implementierung jedoch nicht durchgehend intern knotendisjunkt. Diese Einschränkung wird in Abschnitt 7.3 näher betrachtet.

7.2.2 Vergleich der EDP- und NDP-Verfahren

Wie unterscheiden sich die EDP- und NDP-basierten Routingverfahren hinsichtlich der Resilienz, der durchschnittlichen Hopanzahl und des durchschnittlichen Stretchs?

Die Evaluation zeigt keine allgemeine Überlegenheit eines der beiden Pfadtypen. Auf den synthetischen Random-Graphen liegen die Resilienzwerte der entsprechenden EDP- und NDP-Varianten häufig nahe beieinander. Bei geclusterten Knotenausfällen erreichen die NDP-Verfahren dort teilweise etwas höhere Werte. Auf den Real-World-Graphen hängt die Reihenfolge dagegen stärker von der jeweiligen Topologie und dem Ausfallmodell ab. Bei zufälligen Kantenausfällen besitzen die EDP-Verfahren beispielsweise häufig eine höhere Resilienz als ihre NDP-Gegenstücke.

Bei der durchschnittlichen Hopanzahl und beim Stretch zeigen sich ebenfalls topologieabhängige Unterschiede. Auf den Real-World-Graphen benötigen die NDP-Verfahren in mehreren untersuchten Konfigurationen weniger Hops als die entsprechenden EDP-Verfahren. Auf den Random-Graphen fällt dieser Unterschied kleiner und nicht durchgehend zugunsten der NDP-Verfahren aus.

Die stärkere Disjunktheitsbedingung der NDPs führt somit nicht automatisch zu einer höheren Resilienz. Sie kann die Basispfade besser gegen gemeinsame innere Knotenausfälle trennen, zugleich können jedoch weniger NDPs als EDPs vorhanden sein. Für die Interpretation der Hopanzahl und des Stretchs gelten die in Abschnitt 7.3 beschriebenen Einschränkungen.

7.2.3 Einfluss der Baumerweiterungen

Welchen Einfluss haben die Erweiterungen One Tree und Multiple Trees gegenüber dem Routing über die ursprünglichen disjunkten Pfade bei zufälligen und geclusterten Kanten- und Knotenausfällen?

Die Baumerweiterungen erhöhen in vielen untersuchten Konfigurationen die Resilienz. Zusätzliche Äste ermöglichen es, weitere Ausgänge zum Ziel zu erreichen, wenn ein Teil des ursprünglichen Pfads ausgefallen ist. Der Vorteil ist besonders bei geclusterten Ausfällen sichtbar, da dort mehrere Zugänge in derselben Umgebung betroffen sein können.

Innerhalb der NDP-Verfahren erreichte NDP Multiple Trees in allen ausgewerteten Konfigurationen mindestens die Resilienz von NDP One Tree. NDP One Tree erreichte wiederum mindestens die Resilienz von NDP Paths. Diese Reihenfolge gilt für die untersuchten Graphen und Parameter, stellt jedoch keine allgemeine Garantie für beliebige Netzwerke dar.

Der Resilienzgewinn ist häufig mit einem höheren Routingaufwand verbunden. Die Baumverfahren können zusätzliche Äste ausprobieren und bei Sackgassen zurückgehen. Dadurch steigen die durchschnittliche Hopanzahl und der durchschnittliche Stretch teilweise an. One Tree bildet häufig einen Zwischenwert zwischen dem kurzen Routing über die ursprünglichen Pfade und der höheren Resilienz von Multiple Trees. Die Baumerweiterung verbessert damit vor allem die Zahl erfolgreicher Routings, führt aber nicht gleichzeitig in allen Metriken zum besten Ergebnis.

7.2.4 Artefaktevaluation der Experimentierumgebung

In welchem Umfang ermöglicht die entwickelte Experimentierumgebung eine reproduzierbare Durchführung und eine Erweiterung der Versuche durch andere Personen?

Die Umgebung ermöglicht reproduzierbare Versuchsreihen, wenn dieselbe Konfiguration, derselbe Seed, dieselben Eingabegraphen sowie dieselbe Code- und Softwareversion verwendet werden. Der Seed steuert die Erzeugung synthetischer Graphen, die Auswahl der Start- und Zielknoten und die Bestimmung der Ausfälle. Die tatsächlich verwendete Konfiguration wird in `used_config.json` gespeichert. Das Manifest ergänzt unter anderem Angaben zur Softwareumgebung sowie Prüfsummen des Codes und der verwendeten GraphML-Dateien.

Die Pipeline automatisiert die Durchführung, Aggregation und Darstellung der Experimente. Definierte Erweiterungsstellen ermöglichen es, zusätzliche Algorithmen, Fehlermodelle und Grapharten einzubinden. Zwei der drei Testpersonen setzten alle drei vorgegebenen Erweiterungsarten um. Die dritte Testperson ergänzte einen weiteren Routingalgorithmus. Die Erweiterungen konnten anschließend in kleinen Experimentläufen ausgeführt werden.

Damit wurden Reproduzierbarkeit und Erweiterbarkeit für den geprüften Umfang grundsätzlich bestätigt. Die Aussagekraft der Artefaktevaluation und die noch bestehenden Einschränkungen werden in folgenden Abschnitt eingeordnet.

7.3 Grenzen der Arbeit

Die Ergebnisse dieser Arbeit gelten zunächst für die untersuchten Graphen, Fehlermodelle und Parameterbereiche. Die Evaluation umfasst drei Erdős-Rényi-Konfigurationen und die Real-World-Topologien ATT-MPLS, Oteglobe und Cogentco. Andere Graphgrößen, Dichten oder Netzwerkstrukturen können zu anderen Ergebnissen führen. Die beobachtete Reihenfolge der Verfahren lässt sich deshalb nicht ohne weitere Versuche auf beliebige Netzwerke übertragen.

Alle Graphen werden als ungerichtete und ungewichtete Graphen behandelt. Eigenschaften wie Kantengewichte, Übertragungskapazitäten, Verzögerungen oder mehrere gleichzeitig übertragene Datenströme werden nicht berücksichtigt. Auch die Ausfälle werden durch vereinfachte Modelle erzeugt. Untersucht werden zufällige und geclusterte Kanten- beziehungsweise Knotenausfälle. Gezielte Angriffe, zeitlich aufeinanderfolgende Ausfälle oder geographisch abhängige Störungen sind nicht Bestandteil der Evaluation. Bei Knotenausfällen werden Start- und Zielknoten aus der Kandidatenmenge ausgeschlossen. Bei Kantenausfällen können dagegen abhängig vom Fehlermodell auch inzidente Kanten dieser Knoten ausfallen. Im geclusterten Modell werden Kanten zum Startknoten geschützt, während direkte Kanten zum Zielknoten in der ersten Schicht als Kandidaten zugelassen sind.

Bei zufälligen Ausfällen stellt der Parameter `failFactor` keinen direkten prozentualen Anteil ausgefallener Kanten oder Knoten dar. Die Ausfallanzahl wird bei Kantenausfällen aus der Anzahl der EDPs und bei Knotenausfällen aus der Anzahl der NDPs abgeleitet. Besitzt eine Szene nur einen NDP, werden nach der verwendeten Formel keine zufälligen Knotenausfälle erzeugt. Die Werte verschiedener Fehlermodelle können daher nicht unmittelbar als gleiche Ausfallstärke interpretiert werden.

Start- und Zielknoten werden vor der Platzierung der Ausfälle so gewählt, dass sie verbunden und nicht direkt benachbart sind. Nach der Erzeugung der Ausfälle wird jedoch nicht vorausgesetzt, dass im verbleibenden Graphen noch ein Pfad zwischen ihnen existiert. Die Resilienz enthält damit sowohl unvermeidbare Fehlschläge aufgrund einer vollständigen Trennung als auch Fehlschläge, bei denen noch ein Pfad vorhanden ist, dieser aber von der vorbereiteten Routingstruktur nicht gefunden wird. Eine getrennte Auswertung dieser beiden Fälle wurde nur beispielhaft für eine ausgewählte Konfiguration durchgeführt.

Auch die durchschnittliche Hopanzahl und der durchschnittliche Stretch sind nur eingeschränkt direkt vergleichbar. Beide Metriken werden ausschließlich über die erfolgreichen Routings eines Verfahrens berechnet. Zwei Algorithmen können daher unterschiedliche Mengen von Versuchsszenen zum jeweiligen Mittelwert beitragen. Ein niedrigerer Hopwert bedeutet nicht zwangsläufig, dass ein Verfahren auf denselben Szenen kürzere Routen erzeugt.

Die Implementierung orientiert sich an den grundlegenden Konstruktions- und Routingprinzipien der TREE-Referenz, übernimmt deren Routingreihenfolge jedoch nicht vollständig. Anstelle von `rankTree(...)`, `rankDfs(...)` und `getTreeOrder(...)` wird eine eigene deterministische Ordnung verwendet. Die gemessenen Ergebnisse stellen daher keine direkte Reproduktion sämtlicher Werte der Referenzimplementierung dar.

Eine weitere Einschränkung betrifft NDP Multiple Trees. Die ursprünglichen Ausgangspfade sind knotendisjunkt. Beim anschließenden Baumwachstum werden jedoch nur bereits verwendete Kanten global reserviert. Die erweiterten Bäume können deshalb gemeinsame innere Knoten enthalten. Untersucht wurden somit NDP-basierte Bäume, aber keine Variante, bei der auch die vollständigen Baumstrukturen untereinander knotendisjunkt bleiben.

Die Routingverfahren werden innerhalb eines Python-Graphmodells untersucht. Die Implementierung berechnet die Pfad- und Baumstrukturen und simuliert anschließend die Weiterleitung zwischen Start- und Zielknoten. Eine Installation der daraus abgeleiteten Weiterleitungsregeln auf realen oder virtuellen Netzwerkgeräten ist nicht Bestandteil dieser Arbeit. Der Speicherbedarf der Routingstrukturen, ihre Berechnungszeit und die Auswirkungen auf reale Netzwerkgeräte wurden nicht systematisch untersucht. Die Evaluation konzentriert sich auf Resilienz, durchschnittliche Hopanzahl und durchschnittlichen Stretch.

Schließlich beruht die Bewertung der Experimentierumgebung auf drei Informatikstudierenden und kleinen Testläufen. Die Testpersonen erhielten eine Einführung in das Thema und bei Bedarf gelegentliche Unterstützung. Zudem wurden nicht alle Erweiterungsarten von jeder Person getestet. Die nach den Rückmeldungen überarbeitete Fassung wurde technisch durch automatisierte Tests geprüft, aber nicht erneut durch dieselben Personen praktisch erprobt. Die Ergebnisse liefern daher Hinweise auf Bedienbarkeit und Erweiterbarkeit, stellen jedoch keine umfassende Nutzerstudie dar.

Diese Grenzen beeinträchtigen nicht den Vergleich der sechs Verfahren innerhalb der festgelegten Versuchsbedingungen, da sie jeweils dieselben Graphen, Start- und Zielknoten sowie Ausfallmengen erhalten. Sie begrenzen jedoch, in welchem Umfang die Ergebnisse auf andere Netzwerke, Fehlermodelle und praktische Einsatzbedingungen übertragen werden können.

7.4 Ausblick und mögliche Erweiterungen

Die in dieser Arbeit entwickelte Umgebung bietet mehrere Möglichkeiten für weitere Untersuchungen. Diese betreffen sowohl die Fehlermodelle und Grapharten als auch die Routingalgorithmen, die Ausführung der Experimente und die Auswertung der Ergebnisse.

7.4.1 Weitere Fehlermodelle

Die Evaluation betrachtet zufällige und geclusterte Kanten- und Knotenausfälle. Zukünftige Arbeiten könnten weitere Ursachen und Verteilungen von Ausfällen untersuchen. Beispielsweise könnten Knoten mit einem hohen Grad oder einer hohen Bedeutung für die Netzwerkkonnektivität gezielt ausgewählt werden. Dadurch ließen sich Angriffe oder besonders kritische Komponentenausfälle modellieren.

Das bisherige geclusterte Fehlermodell beginnt in der Umgebung des Zielknotens. Eine mögliche Erweiterung wäre die zufällige Auswahl des Clusterzentrums. Darüber hinaus könnten geographisch benachbarte Knoten und Kanten gemeinsam ausfallen, um regionale Störungen abzubilden. Auch zeitlich aufeinanderfolgende Ausfälle könnten berücksichtigt werden, bei denen sich die verfügbare Topologie während des Routings verändert.

Eine weitere Möglichkeit besteht in einer einheitlicheren Festlegung der Ausfallstärke. Die Anzahl der zufälligen Ausfälle wird aktuell aus dem `failFactor` und der Anzahl der disjunkten Pfade berechnet. Alternativ könnte direkt ein Anteil der vorhandenen Kanten oder zulässigen Knoten ausgewählt werden. Dadurch ließen sich unterschiedliche Graphen und Fehlermodelle leichter anhand derselben tatsächlichen Ausfallrate vergleichen.

7.4.2 Weitere Graphtypen

Die Untersuchung könnte auf zusätzliche synthetische und reale Netzwerktopologien ausgeweitet werden. Die Implementierung unterstützt bereits Random-Regular-Graphen über JSON und CLI, diese wurden jedoch nicht in die Evaluation aufgenommen. Weitere mögliche Modelle sind Small-World-Graphen, skalenfreie Graphen oder geographische Graphen. Sie besitzen andere Gradverteilungen und Zusammenhangsstrukturen als die untersuchten Erdős-Rényi-Graphen.

Zusätzlich könnten weitere Topologien aus öffentlichen Netzwerkdatensammlungen verwendet werden. Eine größere Zahl von Real-World-Graphen würde zeigen, ob die beobachteten Ergebnisse auch auf andere Netzgrößen und Strukturen übertragbar sind. Für gewichtete oder gerichtete Graphen müssten die Pfadberechnung, das Baumwachstum und die Routingroutine entsprechend erweitert werden.

7.4.3 Optimierung der Implementierung

Die Experimente werden derzeit nacheinander ausgeführt. Da die einzelnen Versuchsszenen weitgehend unabhängig voneinander sind, könnten sie zukünftig auf mehrere Prozesse verteilt werden. Jeder Prozess würde eine durch Dataset, Failure-Konfiguration und Wiederholungsnummer eindeutig bestimmte Szene berechnen. Die Ergebnisse könn-

ten anschließend von einem zentralen Prozess sortiert in die CSV-Dateien geschrieben werden. Dadurch wären keine gleichzeitigen Schreibzugriffe auf dieselbe Datei notwendig.

Für die Reproduzierbarkeit müsste jeder Auftrag einen eigenen deterministisch abgeleiteten Seed erhalten. Außerdem müsste die Ausgabe unabhängig von der Reihenfolge gespeichert werden, in der die parallelen Berechnungen abgeschlossen werden. Threads wären ebenfalls möglich, bei den überwiegend rechenintensiven Graphoperationen könnten mehrere Prozesse jedoch eine größere Beschleunigung erreichen.

Weitere Optimierungen könnten nach einer gezielten Laufzeitanalyse an der Pfadberechnung, der Konstruktion der Baumstrukturen und dem Routing ansetzen. Zusätzlich könnten Berechnungszeit, Speicherbedarf und Größe der erzeugten Routingstrukturen automatisch erfasst werden. Dadurch ließe sich neben der Resilienz und der Routenlänge auch der Aufwand der Verfahren vergleichen.

Die Prüfung der Ergebnisdateien kann ebenfalls erweitert werden. Aktuell wird kontrolliert, ob alle erwarteten Parametergruppen und die richtige Anzahl an Ergebniszeilen vorhanden sind. Zukünftig könnte zusätzlich geprüft werden, ob jede Wiederholungsnummer innerhalb einer Gruppe genau einmal vorkommt.

7.4.4 Zusätzliche Routingalgorithmen

Die vorhandenen Erweiterungsstellen ermöglichen die Einbindung weiterer Routingverfahren. Eine naheliegende Variante wäre ein NDP-Multiple-Trees- Verfahren, bei dem beim Baumwachstum nicht nur verwendete Kanten, sondern auch verwendete innere Knoten global reserviert werden. Dadurch könnte verhindert werden, dass die erweiterten Bäume neben dem gemeinsamen Startknoten weitere innere Knoten gemeinsam verwenden. Zu untersuchen wäre, ob die stärkere Trennung den Schutz vor Knotenausfällen verbessert oder das Baumwachstum zu stark einschränkt.

Zwischen One Tree und Multiple Trees könnten weitere Varianten untersucht werden. Anstatt nur den längsten Pfad oder alle vorhandenen Pfade zu erweitern, könnte eine festgelegte Anzahl n der disjunkten Pfade zu Bäumen erweitert werden. Dadurch ließe sich untersuchen, ob bereits wenige zusätzliche Bäume einen großen Teil des Resilienzgewinns von Multiple Trees erreichen, während die durchschnittliche Hopanzahl näher an One Tree bleibt.

Zusätzlich könnte das Wachstum eines einzelnen Baums begrenzt werden. Die Erweiterung müsste nicht von allen inneren Knoten des Ausgangspfads ausgehen, sondern könnte auf die ersten n inneren Knoten oder auf eine bestimmte Entfernung zum Start- beziehungsweise Zielknoten beschränkt werden. Auch eine Begrenzung der Wachstumstiefe oder der Anzahl zusätzlicher Äste wäre möglich. Diese Varianten würden einen einstellbaren Kompromiss zwischen der Größe der Routingstruktur, der Resilienz und der benötigten Hopanzahl ermöglichen.

Zusätzlich könnten hybride Verfahren entwickelt werden, die abhängig von der Topologie zwischen EDPs und NDPs wählen oder beide Pfadtypen gemeinsam verwenden. Ein kürzester noch verfügbarer Pfad könnte außerdem als Vergleichsverfahren dienen. Da ein solches Verfahren alle Ausfälle und den vollständigen Restgraphen kennt, wäre es jedoch nicht als gleichwertiges Fast-Failover-Verfahren, sondern als Referenzwert einzuordnen.

7.4.5 Erweiterung der Auswertung

Eine wichtige Erweiterung besteht in der getrennten Betrachtung topologischer und algorithmischer Fehlschläge. Nach der Platzierung der Ausfälle könnte für jede Versuchsszene gespeichert werden, ob im Restgraphen noch ein Pfad zwischen Start- und Zielknoten vorhanden ist. Die Erfolgsquote der Routingverfahren könnte anschließend zusätzlich nur über die weiterhin verbundenen Szenen berechnet werden. Dadurch würde deutlicher, wie häufig ein Algorithmus einen grundsätzlich vorhandenen Weg tatsächlich nutzen kann.

Für den Vergleich der Hopanzahl und des Stretchs könnten außerdem nur Szenen betrachtet werden, die von allen miteinander verglichenen Verfahren erfolgreich geroutet werden. Dies würde einen direkten Vergleich auf einer gemeinsamen Szenenmenge ermöglichen. Die bisherige Auswertung über die jeweils eigenen erfolgreichen Teilmengen sollte daneben erhalten bleiben, da sie den tatsächlichen Aufwand aller Erfolge eines Verfahrens beschreibt.

Schließlich könnten Konfidenzintervalle, mehrere unabhängige Seed-Reihen und weitere statistische Auswertungen ergänzt werden. Auch eine erneute praktische Erprobung der überarbeiteten Experimentierumgebung mit mehr Testpersonen und ohne vorherige Unterstützung könnte die Aussagen zur Bedienbarkeit und Erweiterbarkeit weiter absichern.

Literatur

[1] James P. G. Sterbenz u. a. „Resilience and survivability in communication networks: Strategies, principles, and survey of disciplines“. In: *Comput. Networks* 54.8 (2010), S. 1245–1265. DOI: 10.1016/J.COMNET.2010.03.005. URL: https://doi.org/10.1016/j.comnet.2010.03.005 (siehe S. 1).

[2] Oliver Schweiger, Klaus-Tycho Foerster und Stefan Schmid. „Improving the Resilience of Fast Failover Routing: TREE (Tree Routing to Extend Edge disjoint paths)“. In: *ANCS '21: Symposium on Architectures for Networking and Communications Systems, Layfette, IN, USA, December 13 - 16, 2021*. ACM, 2021, S. 1–7. DOI: 10.1145/3493425.3502747. URL: https://doi.org/10.1145/3493425.3502747 (siehe S. 1, 4–8, 11–14, 16, 19, 21, 23).

[3] NetworkX Developers. *node_disjoint_paths. NetworkX Documentation*. URL: https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.connectivity.disjoint_paths.node_disjoint_paths.html (siehe S. 1).

[4] Paul Erdős und Alfréd Rényi. „On Random Graphs. I“. In: *Publicationes Mathematicae Debrecen* 6 (1959), S. 290–297 (siehe S. 5).

[5] Edgar N. Gilbert. „Random Graphs“. In: *The Annals of Mathematical Statistics* 30.4 (1959), S. 1141–1144 (siehe S. 5).

[6] Ankit Singla u. a. „Jellyfish: Networking Data Centers Randomly“. In: *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2012, San Jose, CA, USA, April 25-27, 2012*. Hrsg. von Steven D. Gribble und Dina Katabi. USENIX Association, 2012, S. 225–238. URL: https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/singla (siehe S. 5).

[7] Simon Knight u. a. „The Internet Topology Zoo“. In: *IEEE J. Sel. Areas Commun.* 29.9 (2011), S. 1765–1775. DOI: 10.1109/JSAC.2011.111002. URL: https://doi.org/10.1109/JSAC.2011.111002 (siehe S. 5, 72).

[8] Marco Chiesa u. a. „A Survey of Fast-Recovery Mechanisms in Packet-Switched Networks“. In: *IEEE Commun. Surv. Tutorials* 23.2 (2021), S. 1253–1301. DOI: 10.1109/COMST.2021.3063980. URL: https://doi.org/10.1109/COMST.2021.3063980 (siehe S. 5, 6, 8).

[9] Jacek Rak und David Hutchison, Hrsg. *Guide to Disaster-Resilient Communication Networks*. Computer Communications and Networks. Springer, 2020. ISBN: 978-3-030-44684-0. DOI: 10.1007/978-3-030-44685-7. URL: https://doi.org/10.1007/978-3-030-44685-7 (siehe S. 9).

[10] János Tapolcai u. a. „List of shared risk link groups representing regional failures with limited size“. In: *2017 IEEE Conference on Computer Communications, INFOCOM 2017, Atlanta, GA, USA, May 1-4, 2017*. IEEE, 2017, S. 1–9. DOI: 10.1109/INFOCOM.2017.8057040. URL: https://doi.org/10.1109/INFOCOM.2017.8057040 (siehe S. 9).

[11] Pankaj K. Agarwal u. a. „The Resilience of WDM Networks to Probabilistic Geographical Failures“. In: *IEEE/ACM Trans. Netw.* 21.5 (2013), S. 1525–1538. DOI: 10.1109/TNET.2012.2232111. URL: https://doi.org/10.1109/TNET.2012.2232111 (siehe S. 9).

[12] Sebastian Neumayer u. a. „Assessing the Vulnerability of the Fiber Infrastructure to Disasters“. In: *IEEE/ACM Trans. Netw.* 19.6 (2011), S. 1610–1623. DOI: 10.1109/TNET.2011.2128879. URL: https://doi.org/10.1109/TNET.2011.2128879 (siehe S. 9).

[13] J. W. Suurballe und Robert Endre Tarjan. „A quick method for finding shortest pairs of disjoint paths“. In: *Networks* 14.2 (1984), S. 325–336. DOI: 10.1002/NET.3230140209. URL: https://doi.org/10.1002/net.3230140209 (siehe S. 10).

[14] Klaus-Tycho Foerster u. a. „CASA: Congestion and Stretch Aware Static Fast Rerouting“. In: *2019 IEEE Conference on Computer Communications, INFOCOM 2019, Paris, France, April 29 - May 2, 2019*. IEEE, 2019, S. 469–477. DOI: 10.1109/INFOCOM.2019.8737438. URL: https://doi.org/10.1109/INFOCOM.2019.8737438 (siehe S. 10).

[15] Shrinivasa Kini u. a. „Fast Recovery From Dual-Link or Single-Node Failures in IP Networks Using Tunneling“. In: *IEEE/ACM Trans. Netw.* 18.6 (2010), S. 1988–1999. DOI: 10.1109/TNET.2010.2055887. URL: https://doi.org/10.1109/TNET.2010.2055887 (siehe S. 10).

[16] Rihab Maaloul u. a. „Two Node-Disjoint Paths Routing for Energy-Efficiency and Network Reliability“. In: *25th International Conference on Telecommunications (ICT 2018)*. IEEE, 2018, S. 554–560. DOI: 10.1109/ICT.2018.8464840. URL: https://doi.org/10.1109/ICT.2018.8464840 (siehe S. 11).

[17] Aric A. Hagberg, Daniel A. Schult und Pieter J. Swart. „Exploring Network Structure, Dynamics, and Function using NetworkX“. In: *Python in Science Conference* (2008). DOI: 10.25080/TCWV9851. URL: https://doi.org/10.25080/TCWV9851 (siehe S. 11).

[18] Frank Kammer und Hanjo Täubig. „Connectivity“. In: *Network Analysis: Methodological Foundations [outcome of a Dagstuhl seminar, 13-16 April 2004]*. Hrsg. von Ulrik Brandes und Thomas Erlebach. Bd. 3418. Lecture Notes in Computer Science. Springer, 2005, S. 143–177. DOI: 10.1007/978-3-540-31955-9_7. URL: https://doi.org/10.1007/978-3-540-31955-9_7 (siehe S. 17).

Anhang: Vollständige Ergebnisplots

Dieser Anhang enthält die vollständigen Vergleichsabbildungen des in Kapitel 5 ausgewerteten Laufs. Für Random-Graphen und Real-World-Graphen werden jeweils die Resilienz, die durchschnittliche Hopanzahl und der durchschnittliche additive Stretch für alle vier Ausfallkombinationen dargestellt. Zur vollständigen Gegenüberstellung werden auch die bereits in Kapitel 5 gezeigten Resilienz- und Hop-Abbildungen erneut aufgeführt.

Die Resilienz berücksichtigt alle Versuchsszenen. Die Mittelwerte der Hopanzahl und des additiven Stretchs beziehen sich dagegen nur auf die erfolgreichen Routings des jeweiligen Algorithmus. Die Bedeutung der Achsen und die Hinweise zur Interpretation entsprechen Abschnitt 5.2.

Random-Graphen

Zufällige Kantenausfälle

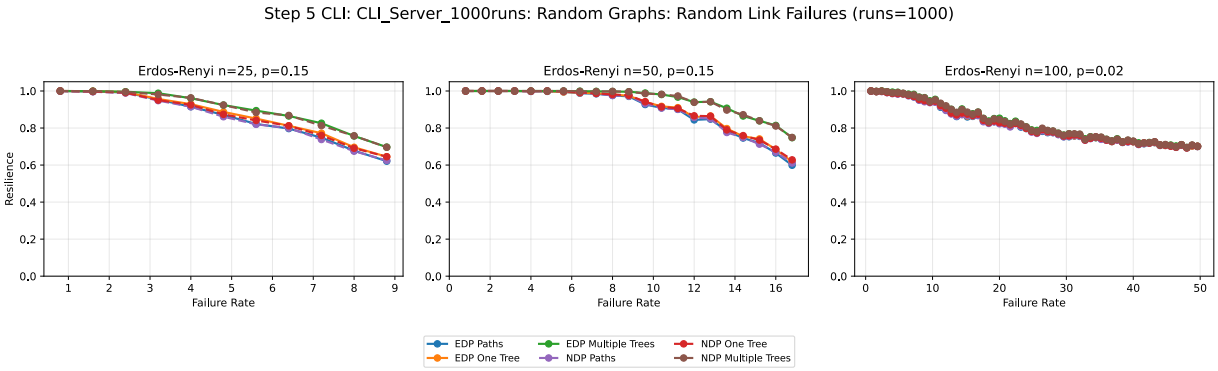


Abbildung 24: Resilienz bei zufälligen Kantenausfällen auf Random-Graphen.

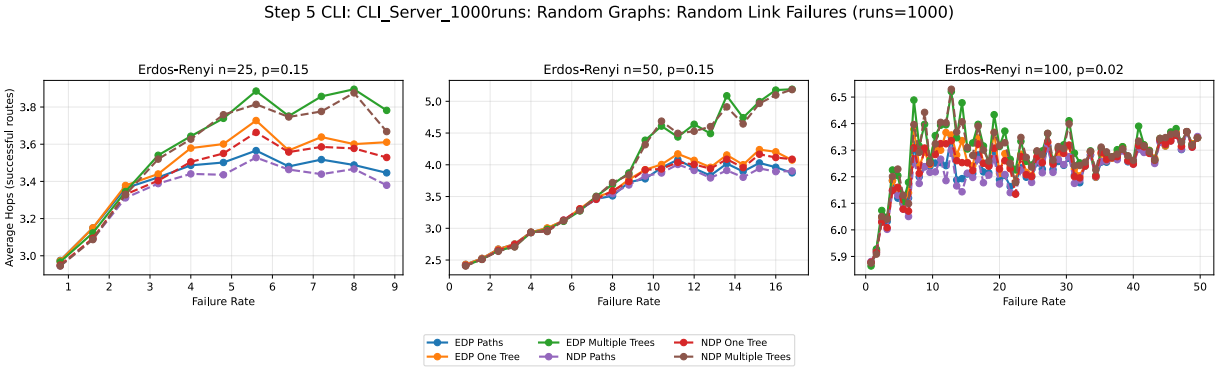


Abbildung 25: Durchschnittliche Hopanzahl erfolgreicher Routings bei zufälligen Kantenausfällen auf Random-Graphen.

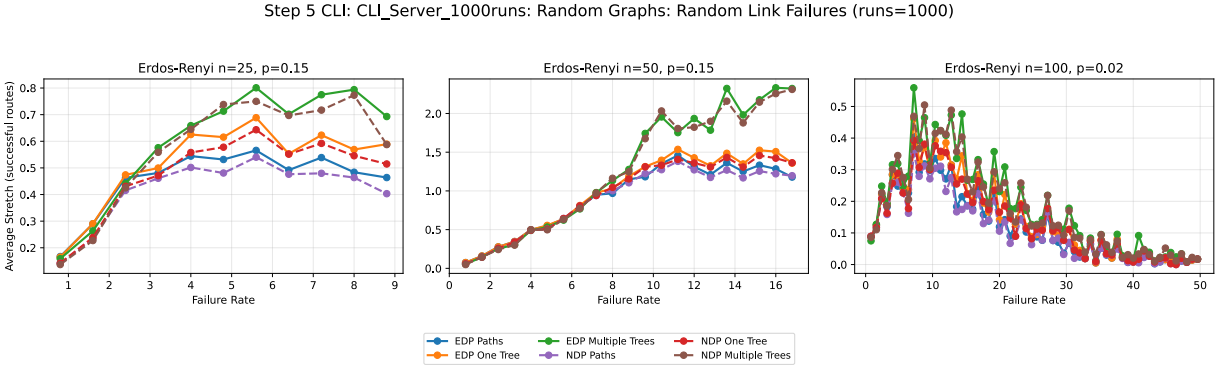


Abbildung 26: Durchschnittlicher additiver Stretch erfolgreicher Routings bei zufälligen Kantenausfällen auf Random-Graphen.

Geclusterte Kantenausfälle

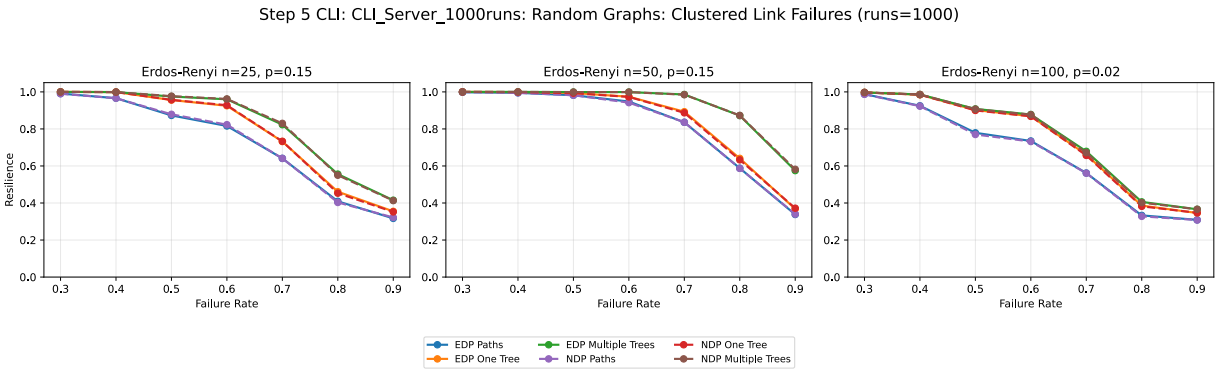


Abbildung 27: Resilienz bei geclusterten Kantenausfällen auf Random-Graphen.

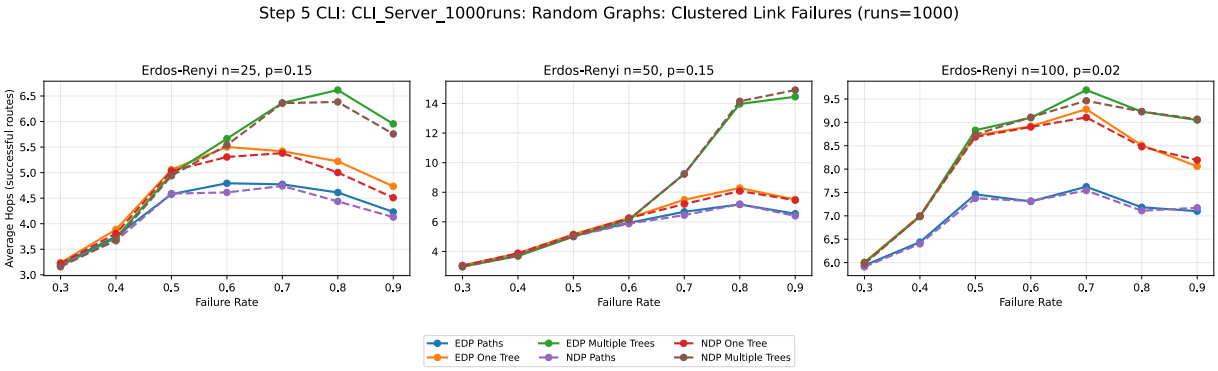


Abbildung 28: Durchschnittliche Hopanzahl erfolgreicher Routings bei geclusterten Kantenausfällen auf Random-Graphen.

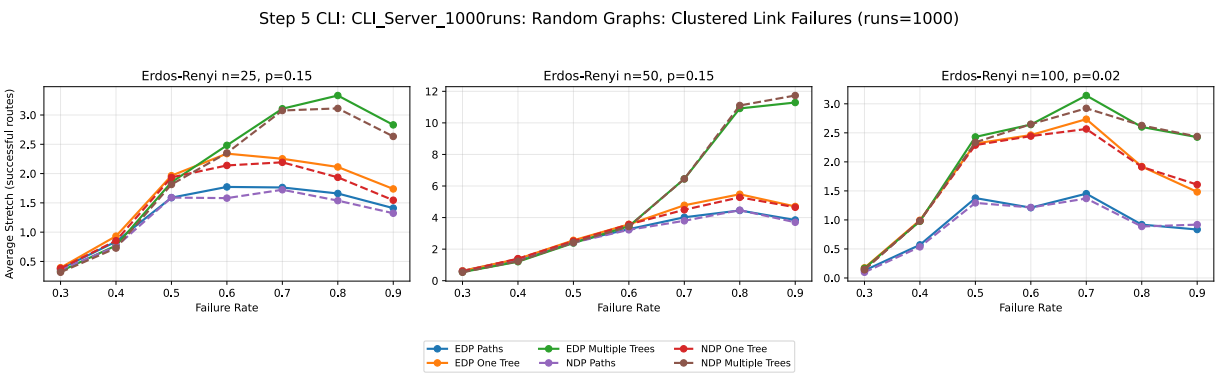


Abbildung 29: Durchschnittlicher additiver Stretch erfolgreicher Routings bei geclusterten Kantenausfällen auf Random-Graphen.

Zufällige Knotenausfälle

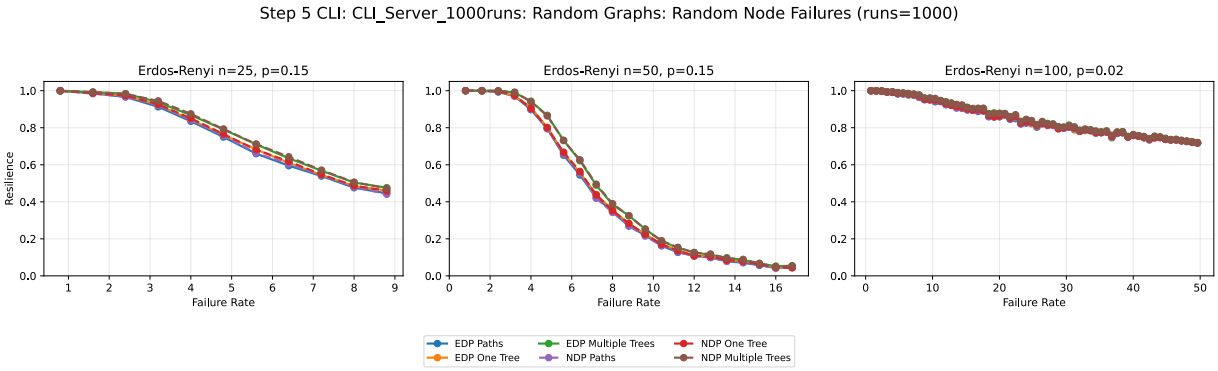


Abbildung 30: Resilienz bei zufälligen Knotenausfällen auf Random-Graphen.

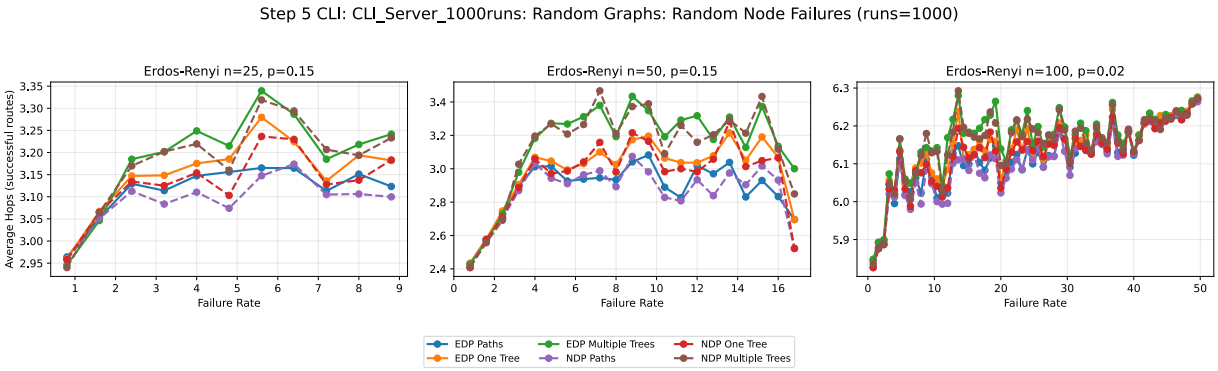


Abbildung 31: Durchschnittliche Hopanzahl erfolgreicher Routings bei zufälligen Knotenausfällen auf Random-Graphen.

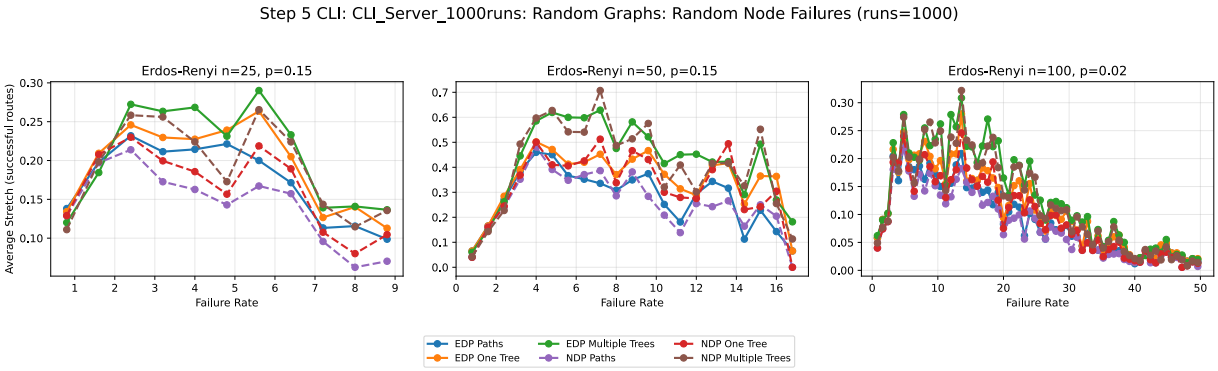


Abbildung 32: Durchschnittlicher additiver Stretch erfolgreicher Routings bei zufälligen Knotenausfällen auf Random-Graphen.

Geclusterte Knotenausfälle

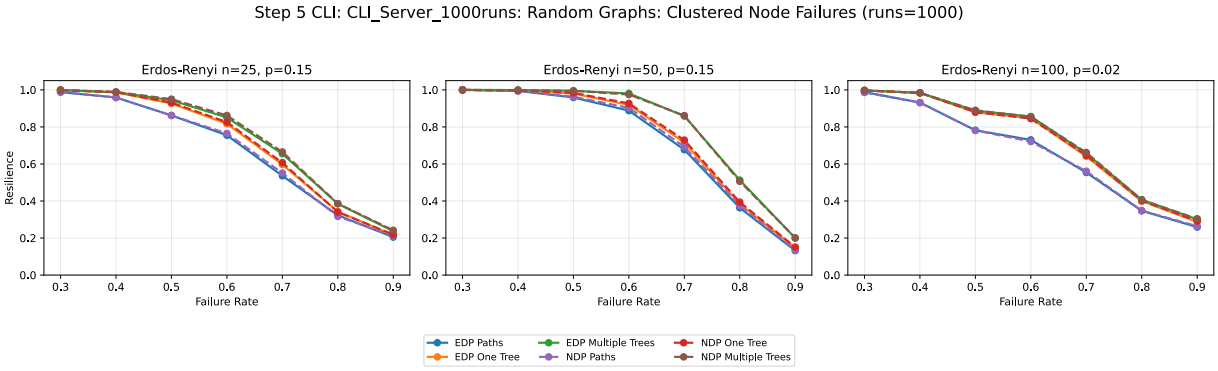


Abbildung 33: Resilienz bei geclusterten Knotenausfällen auf Random-Graphen.

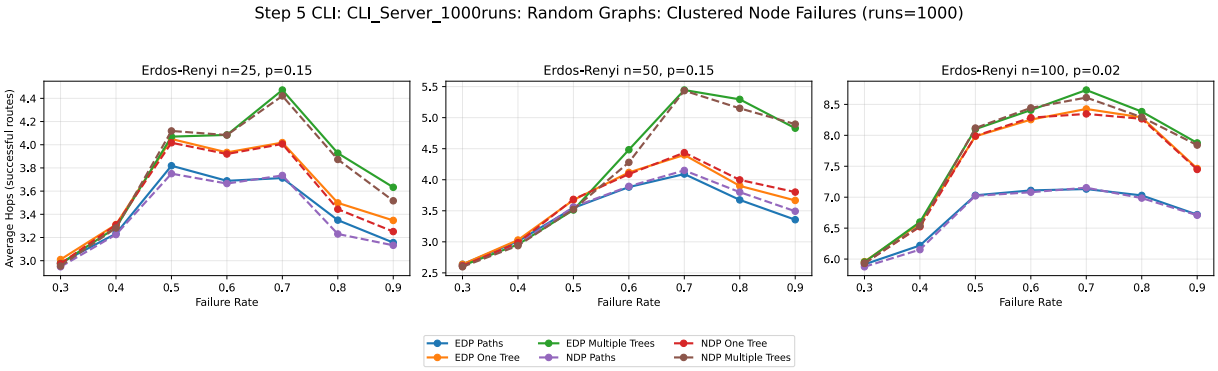


Abbildung 34: Durchschnittliche Hopanzahl erfolgreicher Routings bei geclusterten Knotenausfällen auf Random-Graphen.

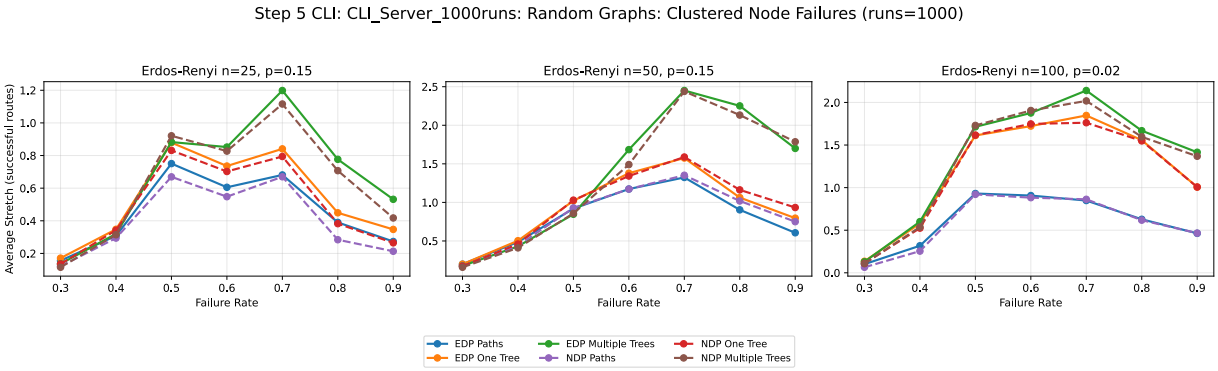


Abbildung 35: Durchschnittlicher additiver Stretch erfolgreicher Routings bei geclusterten Knotenausfällen auf Random-Graphen.

Real-World-Graphen

Zufällige Kantenausfälle

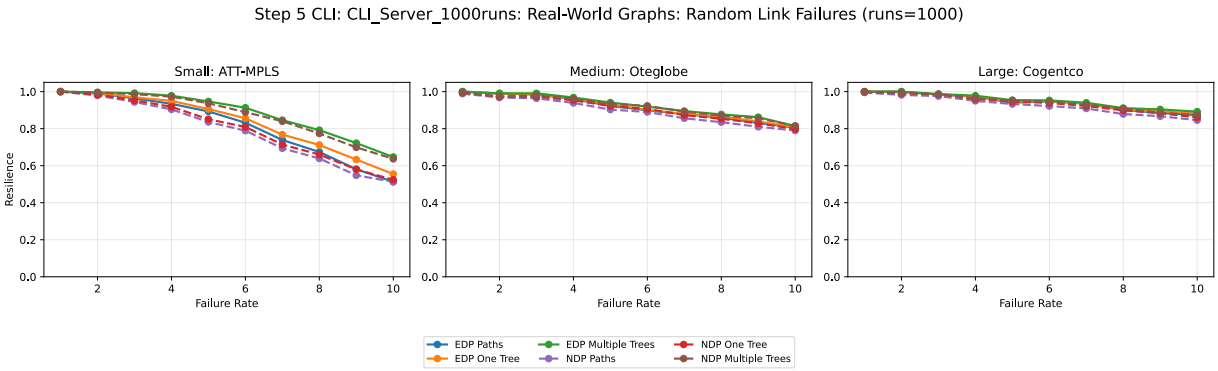


Abbildung 36: Resilienz bei zufälligen Kantenausfällen auf Real-World-Graphen.

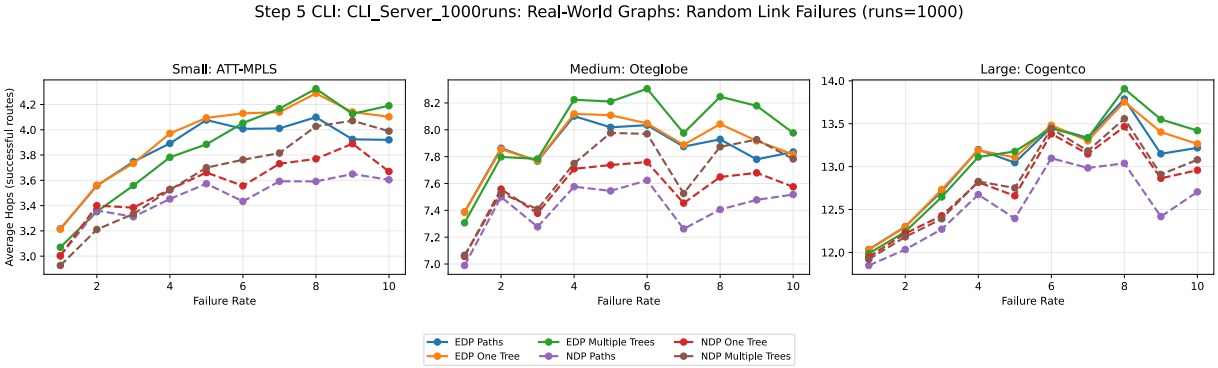


Abbildung 37: Durchschnittliche Hopanzahl erfolgreicher Routings bei zufälligen Kantenausfällen auf Real-World-Graphen.

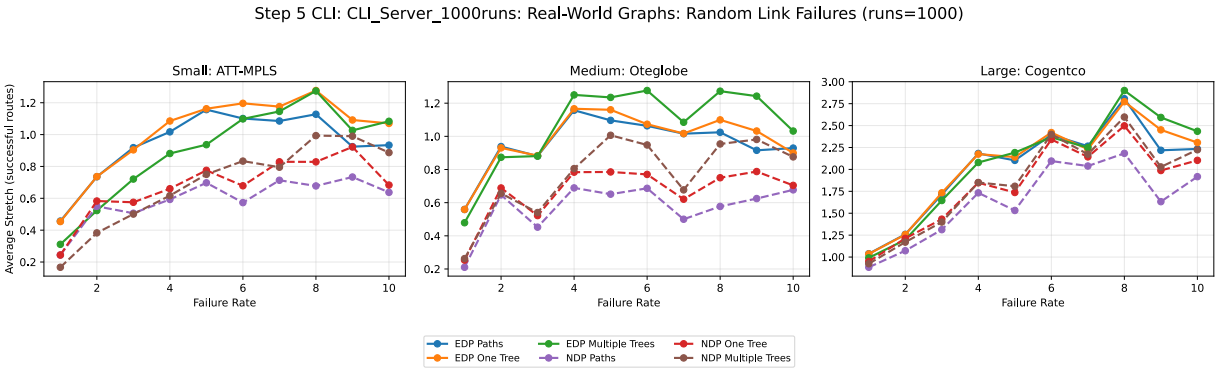


Abbildung 38: Durchschnittlicher additiver Stretch erfolgreicher Routings bei zufälligen Kantenausfällen auf Real-World-Graphen.

Geclusterte Kantenausfälle

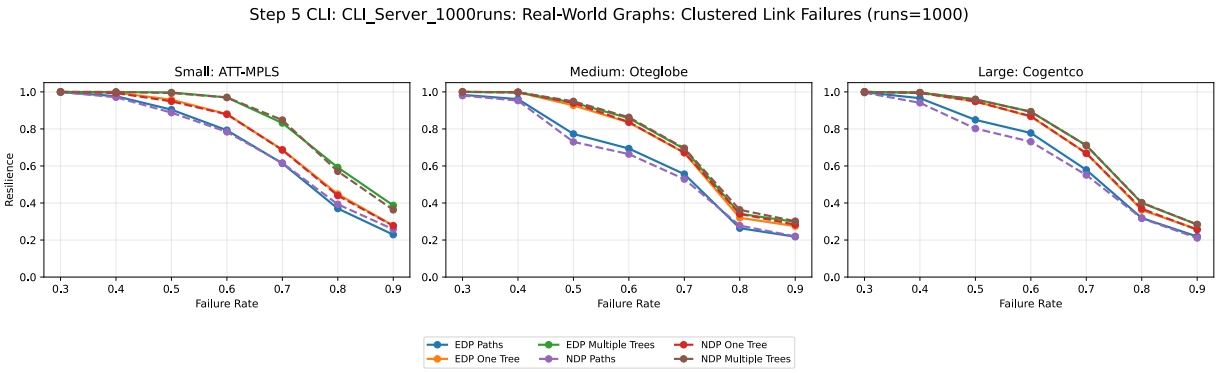


Abbildung 39: Resilienz bei geclusterten Kantenausfällen auf Real-World-Graphen.

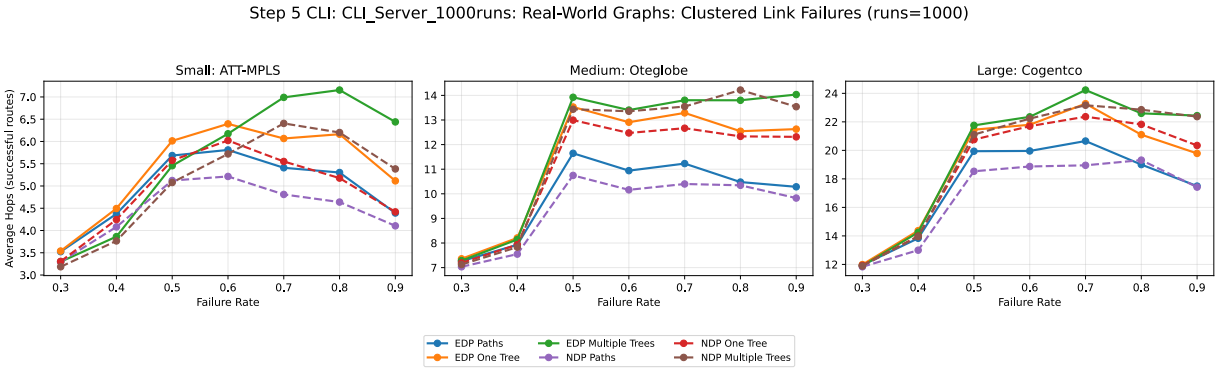


Abbildung 40: Durchschnittliche Hopanzahl erfolgreicher Routings bei geclusterten Kantenausfällen auf Real-World-Graphen.

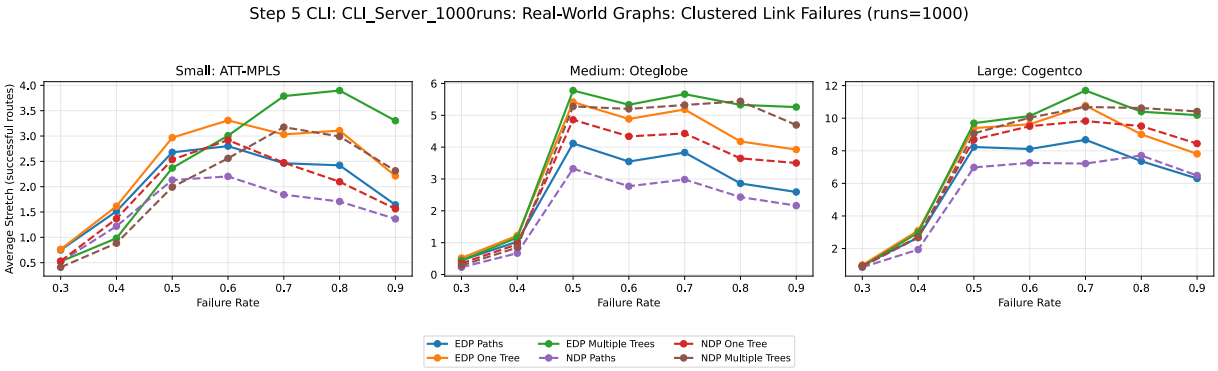


Abbildung 41: Durchschnittlicher additiver Stretch erfolgreicher Routings bei geclusterten Kantenausfällen auf Real-World-Graphen.

Zufällige Knotenausfälle

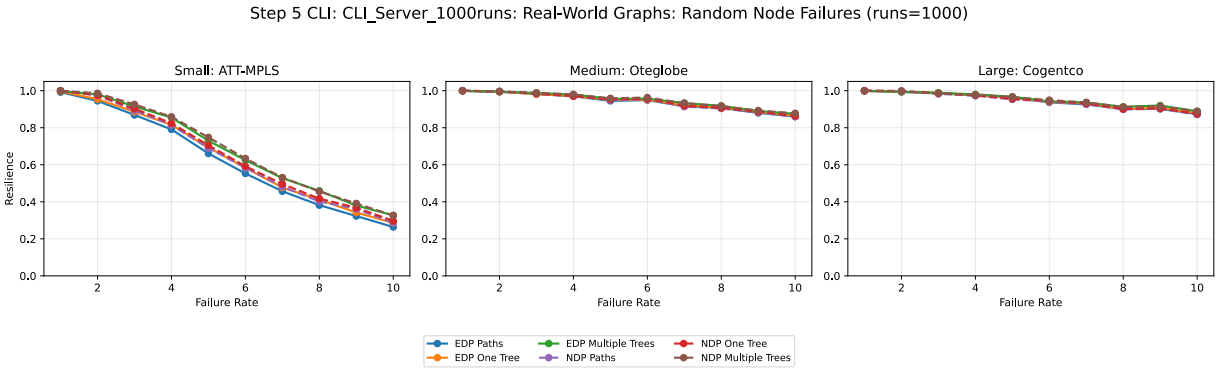


Abbildung 42: Resilienz bei zufälligen Knotenausfällen auf Real-World-Graphen.

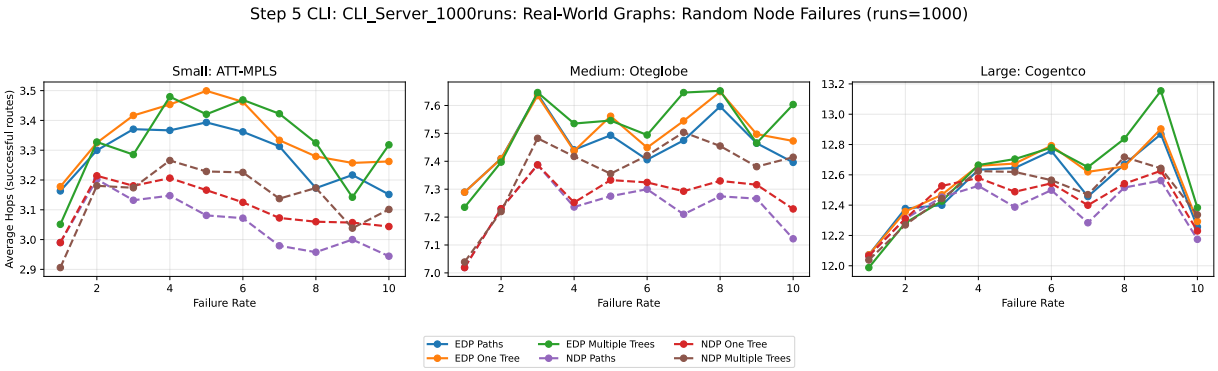


Abbildung 43: Durchschnittliche Hopanzahl erfolgreicher Routings bei zufälligen Knotenausfällen auf Real-World-Graphen.

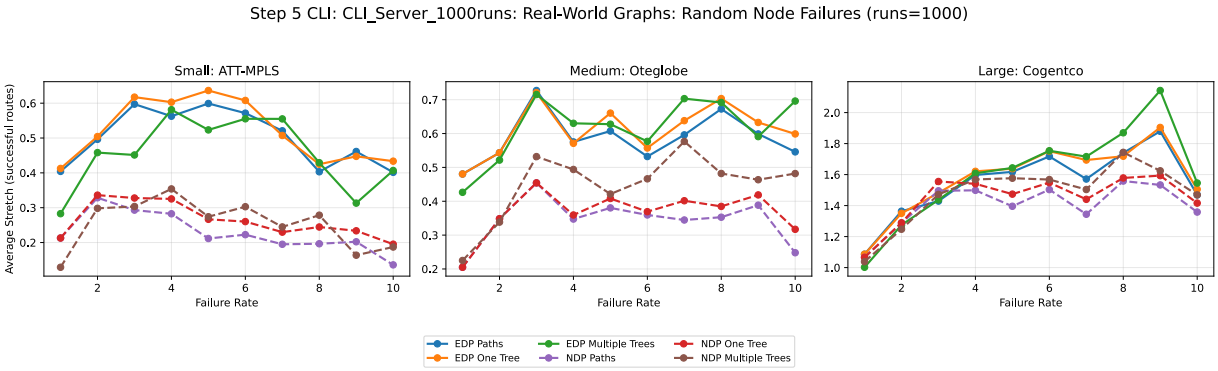


Abbildung 44: Durchschnittlicher additiver Stretch erfolgreicher Routings bei zufälligen Knotenausfällen auf Real-World-Graphen.

Geclusterte Knotenausfälle

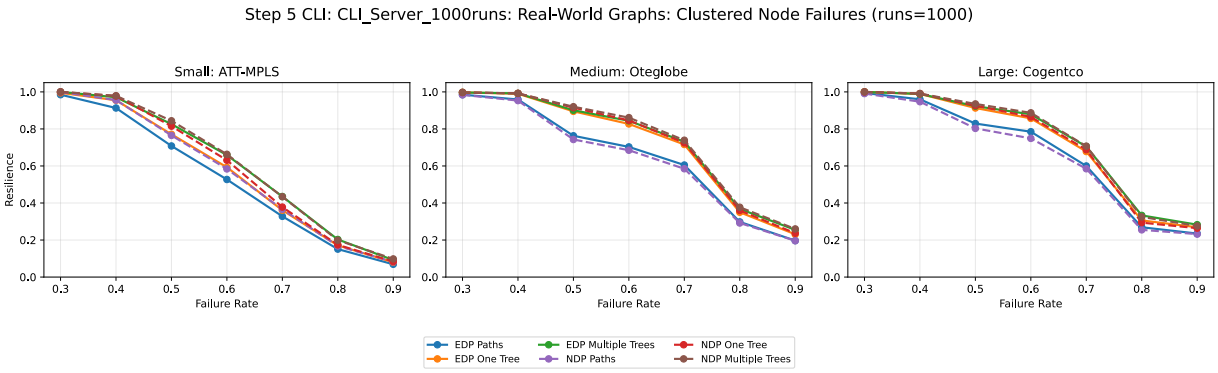


Abbildung 45: Resilienz bei geclusterten Knotenausfällen auf Real-World-Graphen.

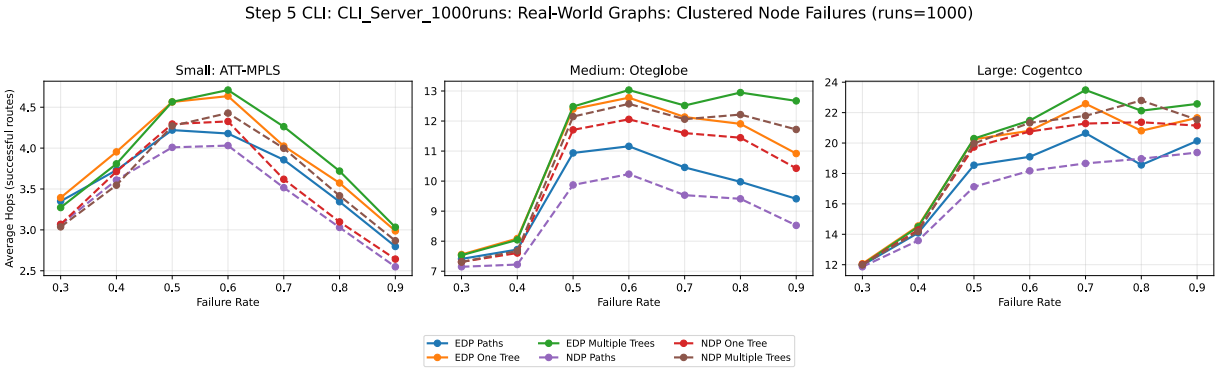


Abbildung 46: Durchschnittliche Hopanzahl erfolgreicher Routings bei geclusterten Knotenausfällen auf Real-World-Graphen.

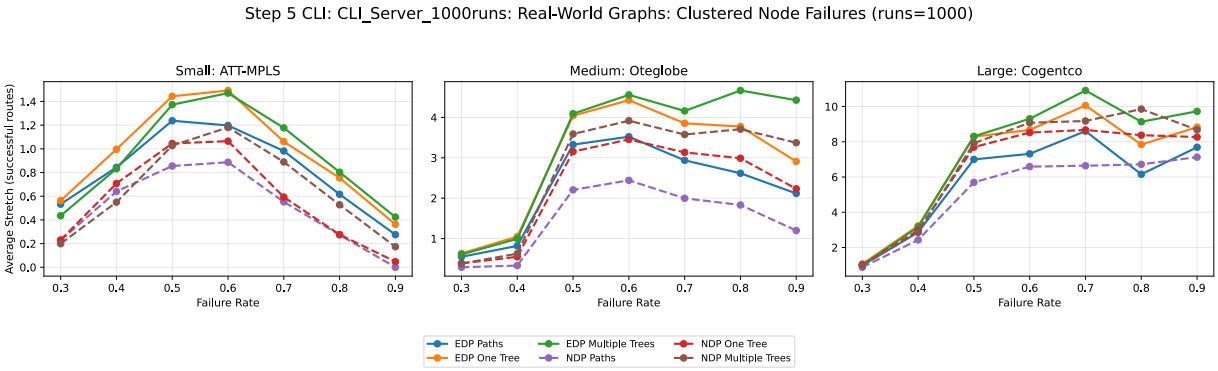


Abbildung 47: Durchschnittlicher additiver Stretch erfolgreicher Routings bei geclusterten Knotenausfällen auf Real-World-Graphen.

Abbildungsverzeichnis

Abbildung 1 Vergleich eines Kantenausfalls (links) und eines Knotenausfalls (rechts) im selben Graphen. Beim Kantenausfall ist eine einzelne Verbindung betroffen. Beim Knotenausfall sind alle zum ausgefallenen Knoten inzidenten Kanten nicht mehr verfügbar. 8

Abbildung 2 Kantendisjunkte Pfade (links) und knotendisjunkte Pfade (rechts) im selben Graphen. Die Farben unterscheiden die einzelnen Pfade. 11

Abbildung 3 EDP One Tree (links) und NDP One Tree (rechts) nach Erweiterung und Trunkierung. Zusätzliche Baumkanten sind rot dargestellt; rot gepunktete Kanten kennzeichnen zusätzliche Verbindungen von Ausgangsknoten zum Ziel d. Diese Zielverbindungen gehören nicht zum gespeicherten Baum. 20

Abbildung 4 EDP Multiple Trees (links) und NDP Multiple Trees (rechts) nach Erweiterung und Trunkierung. Zusätzliche Baumkanten sind rot dargestellt; rot gepunktete Kanten kennzeichnen zusätzliche Verbindungen zum Ziel d. Diese Zielverbindungen gehören nicht zu den gespeicherten Bäumen. 22

Abbildung 5 Seed-gesteuerte Erzeugung einer reproduzierbaren Versuchsszene 45

Abbildung 6 Dry-Run-Ausgabe der GUI mit den geplanten Experimentjobs und der Möglichkeit, den Experimentplan als CSV-Datei herunterzuladen 65

Abbildung 7 Erfolgreiche Preflight-Prüfung der GUI mit der Anzahl der geprüften Datasets und Experimentjobs 66

Abbildung 8 Resilienz bei zufälligen Linkausfällen auf den Random-Graphen 77

Abbildung 9 Durchschnittliche Hopanzahl bei zufälligen Linkausfällen . . . 78

Abbildung 10 Resilienz bei geclusterten Linkausfällen auf den Random-Graphen 79

Abbildung 11 Durchschnittliche Hopanzahl bei geclusterten Linkausfällen . 80

Abbildung 12 Resilienz bei zufälligen Knotenausfällen auf den Random-Graphen 81

Abbildung 13 Durchschnittliche Hopanzahl bei zufälligen Knotenausfällen . 82

Abbildung 14 Resilienz bei geclusterten Knotenausfällen auf den Random-Graphen 83

Abbildung 15 Durchschnittliche Hopanzahl bei geclusterten Knotenausfällen 84

Abbildung 16 Resilienz bei zufälligen Linkausfällen auf den Real-World-Graphen 85
Abbildung 17 Durchschnittliche Hopanzahl bei zufälligen Linkausfällen . . . 86
Abbildung 18 Resilienz bei geclusterten Linkausfällen auf den Real-World-Graphen 87
Abbildung 19 Durchschnittliche Hopanzahl bei geclusterten Linkausfällen . 88
Abbildung 20 Resilienz bei zufälligen Knotenausfällen auf den Real-World-Graphen 89
Abbildung 21 Durchschnittliche Hopanzahl bei zufälligen Knotenausfällen . 90
Abbildung 22 Resilienz bei geclusterten Knotenausfällen auf den Real-World-Graphen 91
Abbildung 23 Durchschnittliche Hopanzahl bei geclusterten Knotenausfällen auf den Real-World-Graphen 92
Abbildung 24 Resilienz bei zufälligen Kantenausfällen auf Random-Graphen. 116
Abbildung 25 Durchschnittliche Hopanzahl erfolgreicher Routings bei zufälligen Kantenausfällen auf Random-Graphen. 117
Abbildung 26 Durchschnittlicher additiver Stretch erfolgreicher Routings bei zufälligen Kantenausfällen auf Random-Graphen. 117
Abbildung 27 Resilienz bei geclusterten Kantenausfällen auf Random-Graphen. 118
Abbildung 28 Durchschnittliche Hopanzahl erfolgreicher Routings bei geclusterten Kantenausfällen auf Random-Graphen. 118
Abbildung 29 Durchschnittlicher additiver Stretch erfolgreicher Routings bei geclusterten Kantenausfällen auf Random-Graphen. 119
Abbildung 30 Resilienz bei zufälligen Knotenausfällen auf Random-Graphen. 120
Abbildung 31 Durchschnittliche Hopanzahl erfolgreicher Routings bei zufälligen Knotenausfällen auf Random-Graphen. 120
Abbildung 32 Durchschnittlicher additiver Stretch erfolgreicher Routings bei zufälligen Knotenausfällen auf Random-Graphen. 121
Abbildung 33 Resilienz bei geclusterten Knotenausfällen auf Random-Graphen. 122
Abbildung 34 Durchschnittliche Hopanzahl erfolgreicher Routings bei geclusterten Knotenausfällen auf Random-Graphen. 122
Abbildung 35 Durchschnittlicher additiver Stretch erfolgreicher Routings bei geclusterten Knotenausfällen auf Random-Graphen. 123

Abbildung 36 Resilienz bei zufälligen Kantenausfällen auf Real-World-Graphen. 124
Abbildung 37 Durchschnittliche Hopanzahl erfolgreicher Routings bei zufälligen Kantenausfällen auf Real-World-Graphen. 124
Abbildung 38 Durchschnittlicher additiver Stretch erfolgreicher Routings bei zufälligen Kantenausfällen auf Real-World-Graphen. 125
Abbildung 39 Resilienz bei geclusterten Kantenausfällen auf Real-World-Graphen. 126
Abbildung 40 Durchschnittliche Hopanzahl erfolgreicher Routings bei geclusterten Kantenausfällen auf Real-World-Graphen. 126
Abbildung 41 Durchschnittlicher additiver Stretch erfolgreicher Routings bei geclusterten Kantenausfällen auf Real-World-Graphen. 127
Abbildung 42 Resilienz bei zufälligen Knotenausfällen auf Real-World-Graphen. 128
Abbildung 43 Durchschnittliche Hopanzahl erfolgreicher Routings bei zufälligen Knotenausfällen auf Real-World-Graphen. 128
Abbildung 44 Durchschnittlicher additiver Stretch erfolgreicher Routings bei zufälligen Knotenausfällen auf Real-World-Graphen. 129
Abbildung 45 Resilienz bei geclusterten Knotenausfällen auf Real-World-Graphen. 130
Abbildung 46 Durchschnittliche Hopanzahl erfolgreicher Routings bei geclusterten Knotenausfällen auf Real-World-Graphen. 130
Abbildung 47 Durchschnittlicher additiver Stretch erfolgreicher Routings bei geclusterten Knotenausfällen auf Real-World-Graphen. 131

Tabellenverzeichnis

Tabelle 1 Verwendete Erdős-Rényi-Graphsets und Random-Failure-Parameter 72

Tabelle 2 Verwendete Real-World-Topologien 73

Tabelle 3 Resilienz bei geclusterten Knotenausfällen mit `failureStart` $= 0{,}7$, `failureDrop` $= 0{,}3$ und 1000 Wiederholungen. Der höhere Wert jedes EDP-NDP-Paars ist fett hervorgehoben. 93

Tabelle 4 Durchschnittliche Hopanzahl erfolgreicher Routings bei geclusterten Knotenausfällen mit `failureStart` $= 0{,}7$ und `failureDrop` $= 0{,}3$. Der niedrigere Wert jedes EDP-NDP-Paars ist fett hervorgehoben. 94

Tabelle 5 Ausführungsumgebungen der dokumentierten Testläufe 99

Tabelle 6 Subjektive Bewertungen aus dem ersten Fragebogen 100

Algorithmenverzeichnis

Algorithmus 1 Extend – NDP One Tree 31
Algorithmus 2 Extend – NDP Multiple Trees 33
Algorithmus 3 Truncation der Baumstruktur 34
Algorithmus 4 Routing über vorbereitete Pfad- und Baumstrukturen 53